

ΠΑΝΕΠΙΣΤΗΜΙΟ ΜΑΚΕΔΟΝΙΑΣ
ΣΧΟΛΗ ΕΠΙΣΤΗΜΩΝ ΠΛΗΡΟΦΟΡΙΑΣ
ΤΜΗΜΑ ΕΦΑΡΜΟΣΜΕΝΗΣ ΠΛΗΡΟΦΟΡΙΚΗΣ



Σχεδίαση και Υλοποίηση Ιεραρχικού Planner σε Java για το πρόβλημα “Barman” (IPC 2023)

Πτυχιακή Εργασία

της

Μαρίας - Νεφέλης Σπυριδοπούλου

Θεσσαλονίκη, Ιούνιος 2026

Σχεδίαση και Υλοποίηση Ιεραρχικού Planner σε Java για το πρόβλημα “Barman” (IPC 2023)

Μαρία - Νεφέλη Σπυριδοπούλου

Προπτυχιακή Φοιτήτρια του Τμήματος Εφαρμοσμένης Πληροφορικής του
Πανεπιστημίου Μακεδονίας

Πτυχιακή Εργασία
υποβάλλεται για τη μερική εκπλήρωση των απαιτήσεων του προπτυχιακού
τίτλου σπουδών στην
Εφαρμοσμένη Πληροφορική - Επιστήμης και Τεχνολογίας Υπολογιστών

Επιβλέπων Καθηγητής
Ιωάννης Ρεφανίδης

Περίληψη

Ο Αυτοματοποιημένος Σχεδιασμός (Automated Planning) αποτελεί έναν από τους καίριους πυλώνες της Τεχνητής Νοημοσύνης, στοχεύοντας στην αυτόνομη λήψη αποφάσεων για την επίτευξη στόχων. Στο πλαίσιο αυτό, ο Ιεραρχικός Σχεδιασμός (Hierarchical Task Network Planning - HTN) διαφοροποιείται από τις κλασικές προσεγγίσεις, οργανώνοντας την διαδικασία σε επίπεδα αφαίρεσης. Μέσω μεθόδων αποσύνθεσης (decomposition methods), οι σύνθετες εργασίες (compound tasks) αποσυντίθενται συστηματικά σε απλές, εκτελέσιμες ενέργειες (primitive actions), έως ότου παραχθεί μια εκτελέσιμη ακολουθία ενεργειών χαμηλού επιπέδου που υλοποιεί τον αρχικό στόχο.

Η παρούσα πτυχιακή εργασία πραγματεύεται την εφαρμογή αυτών των αρχών στο πεδίο “Barman” του Διεθνούς Διαγωνισμού Σχεδιασμού (IPC 2023 - HTN Tracks), παρουσιάζοντας την υλοποίηση ενός εξειδικευμένου συστήματος σχεδιασμού (domain-parsed planner) ολικής διάταξης σε γλώσσα Java. Το σύστημα έχει σχεδιαστεί ώστε να δέχεται ως είσοδο αποκλειστικά το αρχείο του εκάστοτε προβλήματος (pfileXX.hddl), αναγνωρίζοντας αυτόματα τις αρχικές συνθήκες, τους στόχους και τα αντικείμενα του κόσμου, χωρίς να απαιτείται εισαγωγή των κανόνων του πεδίου. Ο αλγόριθμος επίλυσης, βασιζόμενος σε καθιερωμένες τεχνικές αναζήτησης, παράγει σχέδια ολικής διάταξης για τα σενάρια του διαγωνισμού.

Η πειραματική διαδικασία εστιάζει τόσο στην ακριβή μοντελοποίηση των κανόνων του πεδίου, όσο και στην εύρεση ικανοποιητικών πλάνων (Satisficing Plans) αποσύνθεσης των ενεργειών για την κάλυψη όλων των πιθανών σεναρίων, επιβεβαιώνοντας την αποδοτικότητα της εξειδικευμένης προσέγγισης.

Λέξεις Κλειδιά: Τεχνητή Νοημοσύνη, Αυτοματοποιημένος Σχεδιασμός, Ιεραρχικός Σχεδιασμός (HTN), Domain-Parsed Υλοποίηση, Ολική Διάταξη (Total-Order), Satisficing Planning, Διεθνής Διαγωνισμός Σχεδιασμού (IPC 2023), Java.

Abstract

Automated Planning constitutes one of the key pillars of Artificial Intelligence, aiming at autonomous decision-making to achieve specific goals. In this context, Hierarchical Task Network (HTN) distinguishes itself from classical approaches by organizing the process into levels of abstraction. Through decomposition methods, compound tasks are systematically decomposed into simple, executable actions (primitive tasks), until an executable sequence of low-level actions is produced that implements the original goal.

The present thesis addresses the application of these principles to the “Barman” domain of the International Planning Competition (IPC 2023 – HTN Tracks), presenting the implementation of a specialized, total-order planner in Java. The system is designed to accept exclusively the problem file (pfileXX.hddl) as input, automatically recognizing the initial conditions, goals and the objects in the world, without requiring the introduction of domain rules. The solving algorithm, based on established search techniques, produces total-order plans for the competition scenarios.

The experimental procedure focuses both on the accurate modelling of the domain rules and on finding satisficing decomposition plans to cover all possible scenarios, confirming the efficiency of the specialized approach.

Keywords: Artificial Intelligence, Automated Planning, Hierarchical Task Network (HTN), Domain-Parsed Implementation, Total-Order, Satisficing Planning, International Planning Competition (IPC 2023), Java.

Ευχαριστίες

Με την ολοκλήρωση της εκπόνησης της παρούσας εργασίας, αισθάνομαι την ανάγκη να εκφράσω τις ειλικρινείς μου ευχαριστίες σε όλους όσους στάθηκαν αρωγοί σε αυτήν την προσπάθεια.

Πρωτίστως, θα ήθελα να ευχαριστήσω θερμά τον επιβλέποντα καθηγητή μου, κ. Ιωάννη Ρεφανίδη, ο οποίος αποτέλεσε τον κύριο εμπνευστή για την ενασχόληση μου με τον κλάδο της Τεχνητής Νοημοσύνης. Μέσα από τις διαλέξεις του και την επιστημονική του προσέγγιση, μου μετέδωσε το ερέθισμα να εμβαθύνω στο αντικείμενο. Τον ευχαριστώ ιδιαίτερα για την εμπιστοσύνη που μου έδειξε, και που μου ανέθεσε ένα θέμα που ανταποκρινόταν πλήρως στα ενδιαφέροντα μου. Η συμβολή του υπήρξε καθοριστική, παρέχοντας τόσο την απαραίτητη επιστημονική καθοδήγηση όσο και σημαντική ηθική υποστήριξη σε κάθε στάδιο της έρευνας.

Τέλος, οφείλω ένα τεράστιο «ευχαριστώ» στους γονείς μου, Θέμη και Εύη. Η παρουσία τους από την αρχή των σπουδών μου μέχρι και σήμερα είναι ανεκτίμητη. Η αδιάλειπτη στήριξη τους και η διαρκής παρότρυνση τους για την συνεχή εξέλιξη μου, αποτέλεσαν κινητήριο δύναμη για την επιτυχή περάτωση όχι μόνο της παρούσας εργασίας, αλλά και ολόκληρου του κύκλου σπουδών μου.

Περιεχόμενα

1 ΕΙΣΑΓΩΓΗ.....	10
1.1 Το Πεδίο του Αυτόματοποιημένου Σχεδιασμού	10
1.2 Αντικείμενο και Κίνητρο Εργασίας	10
1.3 Σκοπός και Επιμέρους Στόχοι	10
1.4 Συνεισφορά της Εργασίας	11
1.5 Διαθώρωση της Εργασίας	12
2 ΘΕΩΡΗΤΙΚΟ ΥΠΟΒΑΘΡΟ	13
2.1 ΕΙΣΑΓΩΓΗ ΣΤΗΝ ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ ΚΑΙ ΤΟΝ ΣΧΕΔΙΑΣΜΟ	13
2.1.1 Ιστορική Εξέλιξη και Σημασία	13
2.1.2 Κλασικός Σχεδιασμός vs Ιεραρχικός Σχεδιασμός	13
2.1.3 Εφαρμογές του Σχεδιασμού	14
2.2 ΙΕΡΑΡΧΙΚΑ ΔΙΚΤΥΑ ΕΡΓΑΣΙΩΝ (HIERARCHICAL TASK NETWORK)	15
2.2.1 Βασικές Έννοιες του HTN	15
2.2.2 Η Διαδικασία της αποσύνθεσης	16
2.3 ΣΤΡΑΤΗΓΙΚΕΣ ΑΝΑΖΗΤΗΣΗΣ ΚΑΙ ΔΙΑΤΑΞΗΣ	17
2.3.1 Δίκτυο Εργασιών (Task Network)	17
2.3.2 Ολική Διάταξη (Total-Order) vs. Μερική Διάταξη (Partial-Order)	18
2.4 ΓΛΩΣΣΕΣ ΠΕΡΙΓΡΑΦΗΣ ΠΡΟΒΛΗΜΑΤΩΝ	19
2.4.1 Η γλώσσα PDDL (Planning Domain Definition Language)	19
2.4.2 Η επέκταση HDDL (Hierarchical Domain Definition Language)	20
3 ΤΟ ΠΕΔΙΟ ΕΦΑΡΜΟΓΗΣ	21
3.1 Ο ΔΙΕΘΝΗΣ ΔΙΑΓΩΝΙΣΜΟΣ ΣΧΕΔΙΑΣΜΟΥ	21
3.1.1 Ιστορία και Σημασία	21
3.1.2 Οι πτυχές του Διαγωνισμού	22
3.1.3 Το HTN Track του IPC 2023	23
3.2 ΑΝΑΛΥΣΗ ΤΟΥ DOMAIN “BARMAN”	24
3.2.1 Γενική Επισκόπηση	24
3.2.2 Δομή HTN: Tasks και Methods	25
3.2.3 Προκλήσεις, Περιορισμοί και Πολυπλοκότητα του “Barman”	29

4 ΜΕΘΟΔΟΛΟΓΙΑ ΚΑΙ ΥΛΟΠΟΙΗΣΗ ΣΥΣΤΗΜΑΤΟΣ	31
4.1 ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΠΡΟΣΕΓΓΙΣΗ.....	31
4.1.1 Επισκόπηση Ροής Δεδομένων	32
4.2 ΔΟΜΙΚΟΣ ΣΧΕΔΙΑΣΜΟΣ ΚΑΙ ΑΝΑΠΑΡΑΣΤΑΣΗ ΓΝΩΣΗΣ.....	33
4.2.1 Δυναμική Ανάλυση Αρχείων Εισόδου (Parsing).....	34
4.2.2 Μοντελοποίηση Καταστάσεων και Οντοτήτων	35
4.2.3 Σημαντικότερες Κλάσεις του Προγράμματος	37
4.3 Ο ΑΛΓΟΡΙΘΜΟΣ ΤΟΥ PLANNER.....	39
4.3.1 Διαδικασία Αποσύνθεσης Εργασιών	39
4.3.2 Στρατηγική Οπισθοδρόμησης (Backtracking).....	40
4.3.3 Σύνθεση και Εξαγωγή Τελικού Πλάνου	41
4.3.4 Ψευδοκώδικας και δενδρική αναπαράσταση του πρώτου προβλήματος	42
4.4 ΤΕΧΝΙΚΕΣ ΠΡΟΚΛΗΣΕΙΣ ΚΑΙ ΒΕΛΤΙΣΤΟΠΟΙΗΣΕΙΣ ΚΩΔΙΚΑ	44
4.4.1 Διαχείριση Μνήμης και Κλωνοποίηση Καταστάσεων (State Cloning)	44
4.4.2 Αντιμετώπιση Ατέρμονων Βρόχων (Infinite Recursion).....	45
4.4.3 Επιλογή Αποδοτικών Δομών Δεδομένων στη Java.....	45
4.4.4 Βελτιστοποίηση Ελέγχου Προϋποθέσεων (Precondition Checking).....	47
5 ΠΕΙΡΑΜΑΤΙΚΉ ΑΞΙΟΛΟΓΗΣΗ ΚΑΙ ΑΠΟΤΕΛΕΣΜΑΤΑ.....	48
5.1 ΜΕΘΟΔΟΛΟΓΙΑ ΠΕΙΡΑΜΑΤΩΝ ΚΑΙ ΠΕΡΙΒΑΛΛΟΝ.....	48
5.1.1 Προδιαγραφές Συστήματος Εκτέλεσης	48
5.1.2 Το Σύνολο των προβλημάτων Ελέγχου	48
5.1.3 Κριτήρια και Μετρικές Αξιολόγησης.....	48
5.1.4 Διαδικασία Επικύρωσης Πλάνου.....	49
5.1.5 Εργαλεία και Τεχνολογίες Λογισμικού	49
5.2 ΠΑΡΟΥΣΙΑΣΗ ΚΑΙ ΑΝΑΛΥΣΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ	50
5.2.1 Ανάλυση Υπολογιστικής Απόδοσης.....	50
5.2.2 Κλιμάκωση και Διαχείριση Μνήμης	51
5.2.3 Ποιότητα Πλάνων και Μήκος Λύσεων	51
5.3 ΜΕΛΕΤΗ ΠΕΡΙΠΤΩΣΗΣ	52
5.3.1 Ανάλυση Εισόδου και Αρχικοποίηση Συστήματος.....	52
5.3.2 Μηχανισμός Αποσύνθεσης και Διαχείρισης Αδιεξόδων	53
5.3.3 Επικύρωση και Ενδιάμεσος Καθαρισμός Πόρων.....	55

5.3.4 Σύνθεση Τελικού Πλάνου και Αποτελέσματα.....	57
6 ΣΥΜΠΕΡΑΣΜΑΤΑ ΚΑΙ ΜΕΛΛΟΝΤΙΚΗ ΕΡΓΑΣΙΑ.....	59
6.1 ΣΥΜΠΕΡΑΣΜΑΤΑ.....	59
6.2 ΠΕΡΙΟΡΙΣΜΟΙ ΥΛΟΠΟΙΗΣΗΣ	59
6.3 ΠΡΟΤΑΣΕΙΣ ΓΙΑ ΜΕΛΛΟΝΤΙΚΗ ΈΡΕΥΝΑ ΚΑΙ ΕΠΕΚΤΑΣΗ.....	60
ΒΙΒΛΙΟΓΡΑΦΙΑ	61
ΠΑΡΑΡΤΗΜΑ Α	63
ΔΕΙΓΜΑΤΑ ΑΡΧΕΙΩΝ HDDL	63
A.1 Πλήρης Κώδικας του Πεδίου “Barman”.....	63
A.2 Κώδικας Προβλήματος Μέσης Δυσκολίας (pfile10.hddl)	75

ΚΑΤΑΛΟΓΟΣ ΣΧΗΜΑΤΩΝ

Σχήμα 2.1: Αναπαράσταση Κλασικού και Ιεραρχικού Σχεδιασμού	14
Σχήμα 2.2: Ενδεικτική δενδρική αναπαράσταση Ιεραρχικού Σχεδιασμού	15
Σχήμα 2.3: Αναπαράσταση των βασικών εννοιών του HTN	16
Σχήμα 2.4: Κατηγορίες Αλγορίθμων Αναζήτησης	17
Σχήμα 4.1: Διάγραμμα Αρχιτεκτονικής του Συστήματος	31
Σχήμα 4.2: Ιεραρχία Τύπων	37
Σχήμα 4.3: Δενδρική αναπαράσταση της λύσης του πρώτου προβλήματος	44
Σχήμα 5.1: Δείγμα Διαγράμματος Ροής του Δέκατου Προβλήματος	55
Σχήμα 5.2: Διαδικασία Καθαρισμού και Επαναφοράς του Κόσμου	57

ΚΑΤΑΛΟΓΟΣ ΠΙΝΑΚΩΝ

Πίνακας 2.1: Σύγκριση Ολικής και Μερικής Διάταξης στον Ιεραρχικό Σχεδιασμό ...	18
Πίνακας 3.1: Πίνακας Πληροφοριών των tracks του IPC.....	23
Πίνακας 3.2: Επιπτώσεις στον Planner λόγω χρήσης Java	30
Πίνακας 4.1: Πολυπλοκότητα των επιλεγμένων δομών δεδομένων	47
Πίνακας 5.1: Τεχνικά Χαρακτηριστικά Συστήματος	48
Πίνακας 5.2: Συγκεντρωτικά Αποτελέσματα Μήκους Πλάνων	52

1 Εισαγωγή

1.1 Το Πεδίο του Αυτοματοποιημένου Σχεδιασμού

Ο Αυτοματοποιημένος σχεδιασμός (Automated Planning) αποτελεί έναν από τους θεμελιώδεις και παλαιότερους κλάδους της Τεχνητής Νοημοσύνης (TN). Στον πυρήνα του, πραγματεύεται το υπολογιστικό πρόβλημα της επιλογής και οργάνωσης μίας σειράς ενεργειών οι οποίες, όταν εκτελεστούν από έναν ευφυή πράκτορα (agent), μεταβάλλουν την κατάσταση του κόσμου από μία αρχική συνθήκη σε μία επιθυμητή κατάσταση στόχου. Σύμφωνα με τους Russell και Norvig, ο σχεδιασμός γεφυρώνει το χάσμα μεταξύ της αντίληψης και της δράσης επιτρέποντας στον πράκτορα να λειτουργεί ορθολογικά σε πολύπλοκα περιβάλλοντα. Ωστόσο, σε πολύπλοκα πεδία εφαρμογής, όπως αυτό του Barman, που εξετάζεται στην παρούσα εργασία, οι παραδοχές του Κλασικού Σχεδιασμού (Classical Planning) δεν αρκούν για την κατάλληλη εύρεση πλάνου. Εδώ ανακύπτει η ανάγκη για Ιεραρχικό Σχεδιασμό (Hierarchical Task Network Planning – HTN) ο οποίος δεν αρκείται μόνο στις ενέργειες, αλλά εισάγει την έννοια της διαδικαστικής γνώσης μέσω σύνθετων εργασιών (Compound Tasks) και μεθόδων αποσύνθεσης (Decomposition Methods), επιτρέποντας στον πράκτορα να "συλλογίζεται" σε υψηλότερο επίπεδο αφαίρεσης πριν προχωρήσει στην υλοποίηση.

1.2 Αντικείμενο και Κίνητρο Εργασίας

Ο Ιεραρχικός Σχεδιασμός (Hierarchical Task Network Planning – HTN) αποτελεί επέκταση του Κλασικού Σχεδιασμού (Classical Planning), εισάγοντας μια ιεραρχία εργασιών. Σε αντίθεση με την κλασική προσέγγιση, η διαδικασία δεν καθοδηγείται από μία κατάσταση στόχο, αλλά από μία αρχική σύνθετη εργασία (Compound Task) προς αποσύνθεση. Η αποσύνθεση αυτή γίνεται μέσω των μεθόδων αποσύνθεσης (Decomposition Methods) και αποσκοπεί στην εύρεση της κατάλληλης σειράς πρωτογενών εργασιών (Primitive Tasks). Η εκτέλεση των εργασιών αυτών, σε μερική ή πλήρη διάταξη (Partial/Total Order), υλοποιεί την αρχική εργασία υψηλού επιπέδου.

1.3 Σκοπός και Επιμέρους Στόχοι

Ο κύριος σκοπός της παρούσας πτυχιακής εργασίας είναι η σχεδίαση και η ανάπτυξη ενός ολοκληρωμένου συστήματος Ιεραρχικού Σχεδιασμού (HTN Planner) στην αντικειμενοστρεφή γλώσσα προγραμματισμού Java, εξειδικευμένου στην επίλυση προβλημάτων του domain “Barman”, όπως αυτό ορίστηκε στον Διεθνή Διαγωνισμό Σχεδιασμού (IPC 2023).

Πιο συγκεκριμένα, οι επιμέρους στόχοι της έρευνας αναλύονται ως εξής:

1. Μοντελοποίηση Πεδίου (Domain Modelling): Η εις βάθος κατανόηση και στην συνέχεια μεταφορά των φυσικών περιορισμών (π.χ. καθαριότητα ποτηριών, χρήση χεριών) σε ιεραρχικές δομές HTN (Tasks, Methods, Actions).
2. Αλγοριθμική Υλοποίηση: Η ανάπτυξη ενός πλήρως λειτουργικού πλάνου, χρησιμοποιώντας τον αλγόριθμο αναζήτησης Πρώτα σε Βάθος (Depth-First Search) στο χώρο των πλάνων ολικής διάταξης (Total-Order), ικανού να διαχειρίζεται μεθόδους αποσύνθεσης και να παράγει έγκυρες ακολουθίες πρωτογενών ενεργειών.
3. Δυναμική Ανάλυση Δεδομένων (Parsing): Η δημιουργία ενός μηχανισμού ανάλυσης που επιτρέπει στο σύστημα να δέχεται δυναμικά τα αρχεία ορισμού προβλημάτων (.hddl) , αναγνωρίζοντας αυτόματα τις αρχικές καταστάσεις, τα αντικείμενα και τους στόχους.
4. Αξιολόγηση: Η πειραματική επαλήθευση της λειτουργικότητας του συστήματος μέσω της εύρεσης ικανοποιητικών λύσεων (Satisficing Planning) για τα επίσημα σενάρια του διαγωνισμού.

1.4 Συνεισφορά της Εργασίας

Η συνεισφορά της εργασίας στην επιστημονική περιοχή του Αυτοματοποιημένου Σχεδιασμού εντοπίζεται σε τρεις βασικούς άξονες:

1. Σε επίπεδο Υλοποίησης: Παραδίδεται ένας πλήρως λειτουργικός HTN Planner, ο οποίος υλοποιεί τη θεωρία της αποσύνθεσης εργασιών από το μηδέν, προσφέροντας διαφάνεια στον τρόπο λειτουργίας του αλγορίθμου και δυνατότητα επέκτασης.
2. Σε επίπεδο Προγραμματισμού: Η εργασία αναδεικνύει πως η Java, χάρη στη αντικειμενοστρεφή της αρχιτεκτονική, προσφέρει το ιδανικό υπόβαθρο για την αποτύπωση της φυσικής μοντελοποίησης των ιεραρχικών δομών του HTN. Η δυνατότητα ορισμού αφηρημένων κλάσεων επέτρεψε την ακριβή αποτύπωση των εννοιών του πεδίου, οδηγώντας σε έναν Planner που είναι ταυτόχρονα στιβαρός, επεκτάσιμος και εύκολα αναγνώσιμος.
3. Σε επίπεδο Πρακτικής Εφαρμογής: Επιβεβαιώνεται η αποδοτικότητα της εξειδικευμένης προσέγγισης (Domain-Specific Planning) σε ρεαλιστικά περιβάλλοντα, γεφυρώνοντας το χάσμα μεταξύ της θεωρητικής μοντελοποίησης και της πρακτικής επίλυσης προβλημάτων.

1.5 Διάρθρωση της Εργασίας

Η παρούσα εργασία διαρθρώνεται σε έξι κεφάλαια, τα οποία οργανώνουν τη ροή της πληροφορίας από το θεωρητικό υπόβαθρο στην πρακτική υλοποίηση και την αξιολόγηση. Συγκεκριμένα:

Στο Κεφάλαιο 2, τίθεται το απαραίτητο Θεωρητικό Υπόβαθρο. Αναλύονται οι βασικές αρχές του Αυτοματοποιημένου Σχεδιασμού και γίνεται η διάκριση μεταξύ της Κλασικής και Ιεραρχικής προσέγγισης.

Στο Κεφάλαιο 3 πραγματεύεται ο θεσμός του Διεθνή Διαγωνισμού Σχεδιασμού, αναλύεται η ιστορική του αξία και παρουσιάζονται οι πτυχές του. Έπειτα παρουσιάζονται σε βάθος το Domain, αλλά και η δομή των αρχείων ορισμού προβλήματος (HDDL) του “Barman”.

Στο Κεφάλαιο 4 περιγράφεται η μεθοδολογία υλοποίησης, αναλύεται ο τρόπος με τον οποίο αναπτύχθηκε το σύστημα με χρήση της Java. Επιπλέον αναλύεται η αρχιτεκτονική των κλάσεων και ο αλγόριθμος αναζήτησης που εφαρμόστηκε.

Στο Κεφάλαιο 5 παρουσιάζονται και αναλύονται τα αποτελέσματα της έρευνας, με ιδιαίτερη αναφορά στις δυσκολίες και τις προκλήσεις που προέκυψαν, κατά την ανάπτυξη του συστήματος.

Τέλος, στο Κεφάλαιο 6 συνοψίζονται τα βασικά συμπεράσματα της έρευνας και της υλοποίησης, επισημαίνονται τα δυνατά σημεία και οι περιορισμοί του συστήματος, ενώ καταγράφονται προτάσεις για μελλοντικές βελτιώσεις.

2 Θεωρητικό Υπόβαθρο

2.1 Εισαγωγή στην Τεχνητή Νοημοσύνη και τον Σχεδιασμό

Η Τεχνητή Νοημοσύνη (Artificial Intelligence) αποτελεί έναν από τους πιο ραγδαία εξελισσόμενους κλάδους της επιστήμης των υπολογιστών, με στόχο τη δημιουργία μηχανών και συστημάτων που επιδεικνύουν ορθολογική συμπεριφορά. Κέντρο ενδιαφέροντος της σύγχρονης Τεχνητής Νοημοσύνης (TN) αποτελεί η έννοια του ευφυούς πράκτορα (intelligent agent). Ευφυή πράκτορα αποτελούν όλες οι οντότητες που αντιλαμβάνονται το περιβάλλον τους με αισθητήρες και επιδρούν σε αυτό μέσω ορθολογικά συλλογισμένων πράξεων, με σκοπό την επίτευξη συγκεκριμένων στόχων.

Ο Αυτοματοποιημένος Σχεδιασμός (Automated Planning) αποτελεί υποκατηγορία της Τεχνητής Νοημοσύνης, ο οποίος εστιάζει στη διαδικασία λήψης αποφάσεων (Decision Making). Το πεδίο Planning πραγματεύεται το θεμελιώδες ζήτημα της εύρεσης μιας κατάλληλης γραμμικής ακολουθίας ενεργειών, η οποία όταν εκτελεστεί στην τρέχουσα κατάσταση θα καταλήξει σε μία κατάσταση στόχο, όπου βάση αυτής, ορίστηκε το συγκεκριμένο πλάνο.

2.1.1 Ιστορική Εξέλιξη και Σημασία

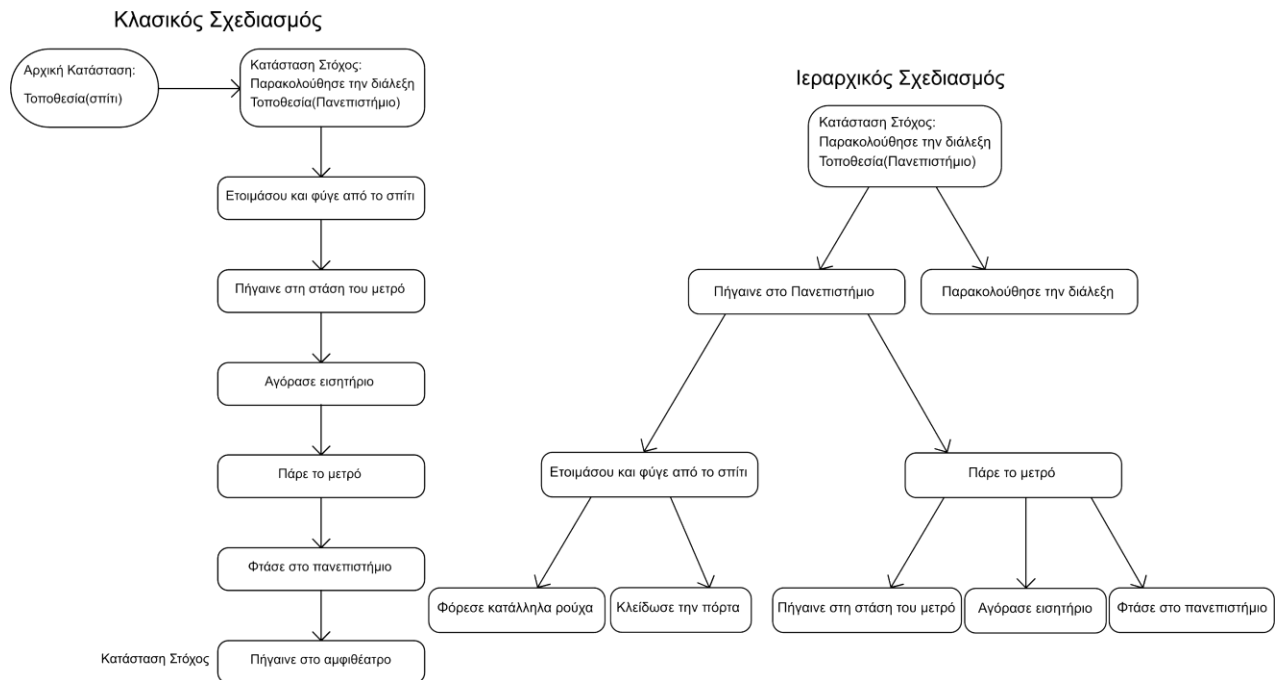
Οι ρίζες του Αυτοματοποιημένου Σχεδιασμού εντοπίζονται στα τέλη της δεκαετίας του 1950, πρώτη σημαντική προσπάθεια αποτέλεσε το σύστημα General Problem Solver (GPS) από τους Newell, Shaw και Simon (1957), το οποίο εισήγαγε την έννοια της means-ends analysis.

Βέβαια, το θεμελιώδες πλαίσιο για τον σύγχρονο σχεδιασμό τέθηκε το 1971 με την ανάπτυξη του συστήματος Stanford Research Institute Problem Solver (STRIPS) των Fikes και Nilsson. Το STRIPS εισήγαγε έναν τυπικό τρόπο περιγραφής ενεργειών με την χρήση προϋποθέσεων (Preconditions) και επιδράσεων (Effects). Η λογική αυτή παραμένει κυρίαρχη μέχρι και σήμερα, καθώς επίσης αποτελεί βάση για τις γλώσσες περιγραφής PDDL και HDDL, οι οποίες χρησιμοποιούνται στον Διεθνή Διαγωνισμό Σχεδιασμού (International Planning Competition).

2.1.2 Κλασικός Σχεδιασμός vs Ιεραρχικός Σχεδιασμός

Η επιλογή μεταξύ του Κλασικού και του Ιεραρχικού Σχεδιασμού καθορίζεται από τη φύση του προβλήματος και το επίπεδο διαθέσιμης γνώσης που έχουμε για τον τομέα της εφαρμογής. Η θεμελιώδης διαφορά τους έγκειται στη δομή του παραγόμενου πλάνου: ο Κλασικός Σχεδιασμός παράγει μία επίπεδη αλληλουχία ενεργειών μέσω αναζήτησης στον χώρο καταστάσεων, ενώ ο Ιεραρχικός Σχεδιασμός οργανώνει το πλάνο σε μία δεντρική δομή, αποσυνθέτοντας σταδιακά μια γενική εργασία υψηλού

επιπέδου σε απλούστερες υπο-εργασίες. Αυτή η δομική διαφοροποίηση επηρεάζει άμεσα την υπολογιστική αποδοτικότητα, καθώς ο Ιεραρχικός Σχεδιασμός περιορίζει τον χώρο αναζήτησης κατάστασης χρησιμοποιώντας προκαθορισμένες μεθόδους επίλυσης σε αντίθεση με τον Κλασικό Σχεδιασμό, όπου χρειάζεται να ανακαλύψει τη λύση βασιζόμενος αποκλειστικά στις προϋποθέσεις και τα αποτελέσματα των ενεργειών.



Σχήμα 2.1: Αναπαράσταση Κλασικού και Ιεραρχικού Σχεδιασμού

2.1.3 Εφαρμογές του Σχεδιασμού

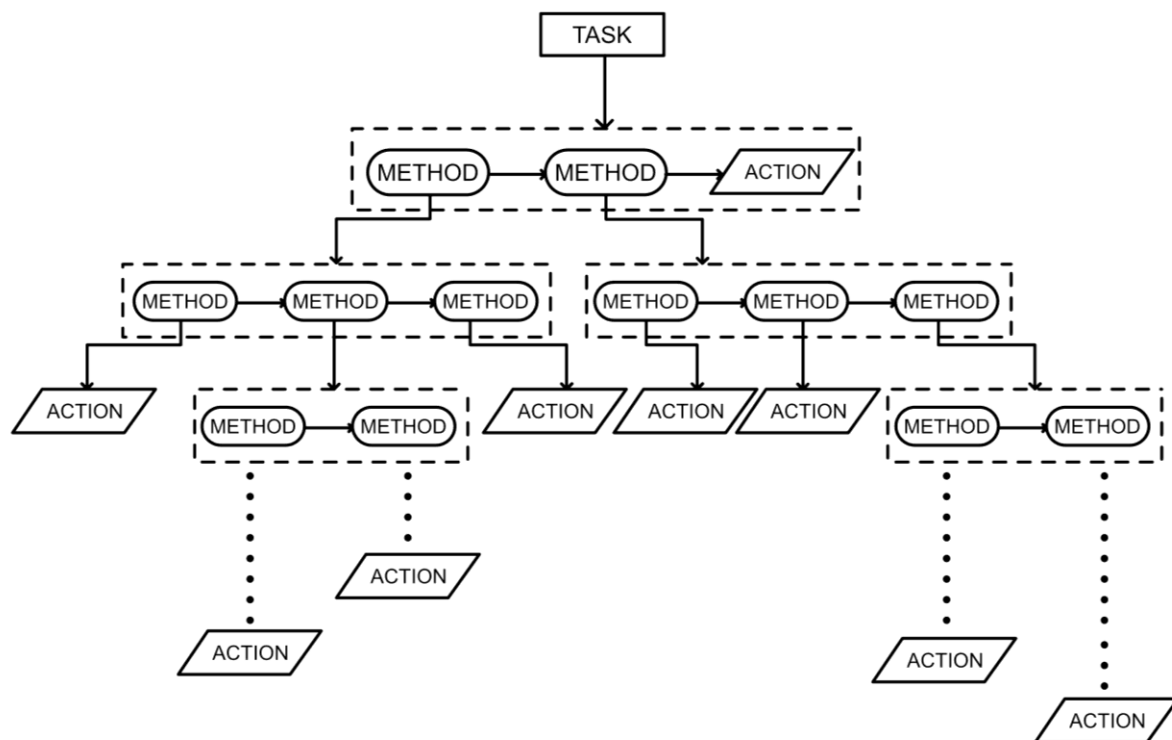
Ο Σχεδιασμός (Planning) εφαρμόζεται σε ένα ευρύ φάσμα πεδίων, με την επιλογή της μεθοδολογίας να εξαρτάται από τη δομή του εκάστοτε προβλήματος. Στην περίπτωση του Κλασικού Σχεδιασμού (Classical Planning), η κυριότερη εφαρμογή εντοπίζεται στα Logistics, όπου ένας πράκτορας παράγει μια γραμμική αλληλουχία ενεργειών, όπως «μετακίνηση στο ράφι», «φόρτωση δέματος» και «παράδοση» με στόχο τη βέλτιστη διαδρομή σε ένα ελεγχόμενο και πλήρες παρατηρήσιμο περιβάλλον.

Αντίθετα, ο Ιεραρχικός Σχεδιασμός (HTN Planning) χρησιμοποιείται κυρίως για προβλήματα που απαιτούν οργάνωση σε επίπεδα, όπως ο Βιομηχανικός σχεδιασμός παραγωγής (Manufacturing Planning). Αντί το σύστημα να σχεδιάζει κάθε κίνηση ενός μηχανήματος ξεχωριστά, αποσυνθέτει μια σύνθετη εργασία, όπως η «Κατασκευή Εξαρτήματος», σε υποεργασίες: «Αγορά Υλικού», «Προετοιμασία Μηχανών», «Μηχανική Κατεργασία» και έπειτα «Ποιοτικός Έλεγχος». Αυτή η προσέγγιση επιτρέπει την αποτελεσματική επίλυση σύνθετων προβλημάτων, καθώς περιορίζει την

αναζήτηση λύσεων μόνο σε λογικούς συνδυασμούς που έχουν οριστεί εκ των προτέρων, αποφεύγοντας έτσι την άσκοπη επεξεργασία κάθε πιθανής κίνησης.

2.2 Ιεραρχικά Δίκτυα Εργασιών (Hierarchical Task Network)

Τα Ιεραρχικά Δίκτυα Εργασιών (HTN) αποτελούν μία μέθοδο αυτόματου σχεδιασμού στο πεδίο της Τεχνητής Νοημοσύνης. Σε αντίθεση με τις μεθόδους σχεδιασμού «από κάτω προς τα πάνω», η Ιεραρχική Σχεδίαση ξεκινάει από ένα αφηρημένο στόχο υψηλού επιπέδου (Task) και στη συνέχεια αναλύεται ιεραρχικά μέσω μεθόδων αποσύνθεσης (Decomposition Methods), μέχρι να φτάσει σε πρωτογενείς ενέργειες (Primitive Actions) που το σύστημα μπορεί να εκτελέσει απευθείας. [13] [15]



Σχήμα 2.2: Ενδεικτική δενδρική αναπαράσταση Ιεραρχικού Σχεδιασμού

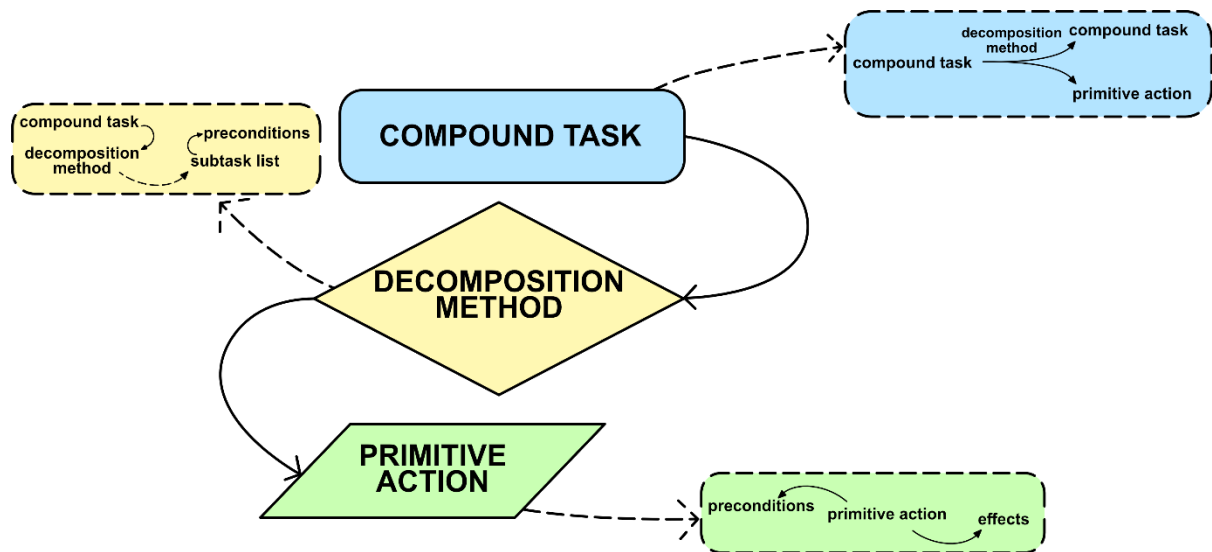
2.2.1 Βασικές Έννοιες του HTN

Τα θεμελιώδη συστατικά του Ιεραρχικού Σχεδιασμού (HTN) είναι οι Σύνθετες Ενέργειες (Compound Tasks). Αυτές αποτελούν ενέργειες υψηλού επιπέδου που αναπαριστούν αφηρημένους στόχους και δεν μπορούν να εκτελεστούν άμεσα. Αντίθετα, πρέπει να διασπαστούν περαιτέρω σε εργασίες (Subtasks), οι οποίες μπορεί να είναι είτε άλλες Σύνθετες Ενέργειες είτε Πρωτογενείς Ενέργειες.

Η διαδικασία διάσπασης υλοποιείται μέσω των μεθόδων αποσύνθεσης (Decomposition Methods). Κάθε Μέθοδος συνδέεται με μία συγκεκριμένη Σύνθετη

Ενέργεια (Compound Task) και περιγράφει έναν πιθανό τρόπο διάσπασης της. Για να είναι εφαρμόσιμη μία Μέθοδος σε μία δεδομένη κατάσταση, οφείλει να ικανοποιεί ένα σύνολο Προϋποθέσεων (Preconditions), οι οποίες βασίζονται στα Κατηγορούμενα (Predicates) του πεδίου (Domain).

Οι Πρωτογενείς Ενέργειες (Primitive Actions) αποτελούν μη περαιτέρω διασπώμενα στοιχεία και το κατώτερο επίπεδο της ιεραρχίας, καθώς επίσης είναι τα μοναδικά στοιχεία που επιτρέπεται να εμφανίζονται στο τελικό πλάνο. Κάθε Primitive Action, εκτός από τις Προϋποθέσεις της, περιλαμβάνει και ένα σύνολο Επιδράσεων (Effects). Οι Επιδράσεις αυτές τροποποιούν την τρέχουσα κατάσταση του κόσμου και η κατάλληλη σειριακή εκτέλεση τους οδηγεί τελικά στην επίτευξη του αρχικού στόχου.



Σχήμα 2.3: Αναπαράσταση των βασικών εννοιών του HTN

2.2.2 Η Διαδικασία της αποσύνθεσης

Στον Ιεραρχικό Σχεδιασμό, ο στόχος δεν ορίζεται αποκλειστικά από μία επιθυμητή τελική κατάσταση, όπως στον Κλασικό Σχεδιασμό, αλλά από ένα αρχικό σύνολο Σύνθετων Εργασιών (Compound Tasks), οι οποίες πρέπει να αποσυντεθούν σε μία ακολουθία Ενεργειών χαμηλού επιπέδου (Primitive Actions).

Αυτό γίνεται σταδιακά μέσω της εύρεσης κατάλληλων Μεθόδων Αποσύνθεσης (Method). Η αποσύνθεση των Tasks δεν αποτελεί εύκολη διαδικασία, απαιτείται πλήρης κατανόηση της τρέχουσας κατάστασης, καθώς επίσης των κανόνων του πεδίου (Domain). Κάθε Method έχει προϋποθέσεις (Preconditions) που πρέπει να ικανοποιεί προτού εφαρμοστεί, και αφού εφαρμοστεί αντικαθιστά το συγκεκριμένο Task με μία 'λίστα' ενδιάμεσων εργασιών που μπορεί να αποτελείται είτε από άλλα Tasks, είτε από Actions.

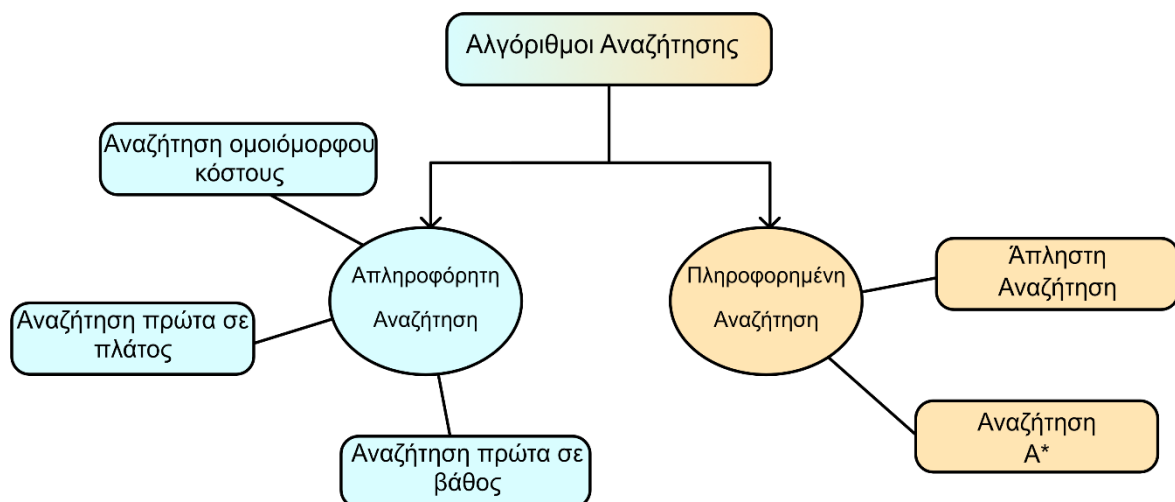
Η διαδικασία "ελέγχου προϋποθέσεων και αποσύνθεσης" εφαρμόζεται αναδρομικά, έως ότου παραχθεί ένα τελικό πλάνο που θα αποτελείται αποκλειστικά

από Πρωτογενείς Ενέργειες (Primitive Actions). Η σειριακή εκτέλεση αυτών των ενεργειών επιτυγχάνει την υλοποίηση της αρχικής Εργασίας υψηλού επιπέδου.

2.3 Στρατηγικές Αναζήτησης και Διάταξης

Το ουσιαστικό πρόβλημα που αντιμετωπίζει ένα Planner δεν είναι απλώς η περιγραφή του κόσμου, αλλά η πλοήγηση μέσα σε αυτόν. Το κεντρικό ζητούμενο είναι ο μηχανισμός που θα χρησιμοποιηθεί για να διαχειριστεί τον τεράστιο αριθμό πιθανών συνδυασμών αποσύνθεσης, ώστε να εντοπίσει την κατάλληλη ακολουθία ενεργειών που ικανοποιεί τον στόχο. Η διαδικασία αυτή ανάγεται στο πρόβλημα της Αναζήτησης (Search).

Στον HTN ο αλγόριθμος καλείται να λάβει αποφάσεις σε κάθε βήμα της αναζήτησης σε σχέση με το ποια Εργασία να επεξεργαστεί πρώτα και ποια Μέθοδο να χρησιμοποιήσει για την αποσύνθεση. Οι απαντήσεις σε αυτά τα ερωτήματα καθορίζονται από τον αλγόριθμο αναζήτησης και το είδος της διάταξης, που επιβάλλει ο Planner.



Σχήμα 2.4: Κατηγορίες Αλγορίθμων Αναζήτησης

2.3.1 Δίκτυο Εργασιών (Task Network)

Στο πλαίσιο του HTN, το Δίκτυο Εργασιών (Task Network) αποτελεί τη θεμελιώδη δομή δεδομένων που αναπαριστά την τρέχουσα κατάσταση της διαδικασίας σχεδιασμού. Σε αντίθεση με τον κλασικό σχεδιασμό που εστιάζει πρωτίστως στις καταστάσεις του κόσμου (States), ο Ιεραρχικός Σχεδιασμός διαχειρίζεται πρωτίστως συλλογές εργασιών και τους μεταξύ τους συσχετισμούς.

Τυπικά, ένα Δίκτυο Εργασιών μπορεί να οριστεί ως ένα ζεύγος $v = (T, C)$ όπου T ορίζονται τα Tasks (Compound και Primitive) και C τα αντίστοιχα Constraints. Τα

Constraints καθορίζουν τους περιορισμούς των Tasks του συνόλου T που πρέπει να ικανοποιηθούν κατά τη διάρκεια της διαδικασίας σχεδιασμού και από τη λύση.

Η διαδικασία σχεδιασμού ξεκινά με ένα αρχικό δίκτυο που περιέχει Εργασίες-Στόχους υψηλού επιπέδου, μέσω της εφαρμογής Μεθόδων Αποσύνθεσης, το δίκτυο αυτό τροποποιείται σταδιακά. Οι σύνθετοι “κόμβοι” αντικαθίστανται από υπό-δίκτυα, ενώ νέοι περιορισμοί προστίθενται στο σύνολο C . Ένα δίκτυο εργασιών θεωρείται λύση όταν περιέχει αποκλειστικά πρωτογενείς ενέργειες και δεν παραβιάζει κανέναν από τους περιορισμούς του συνόλου C . [15]

2.3.2 Ολική Διάταξη (Total-Order) vs. Μερική Διάταξη (Partial-Order)

Μία από τις κρίσιμες διακρίσεις στον σχεδιασμό αλγορίθμων HTN αφορά τον τρόπο με τον οποίο διατάσσονται χρονικά οι Ενέργειες μέσα στο δίκτυο. Διάκριση γίνεται σε δύο κατηγορίες. Την Ολική Διάταξη (Total-Order HTN), την οποία ακολουθεί και η υλοποίηση της παρούσας εργασίας και την Μερική Διάταξη (Partial-Order).

Στην Ολική Διάταξη ο αλγόριθμός γνωρίζει ακριβώς ποια είναι η επόμενη Εργασία που πρέπει να εκτελεστεί, η δομή του δικτύου μοιάζει κάπως $T1 < T2 < T3$. Σημαντικό πλεονέκτημα της προσέγγισης αυτής είναι πως επειδή οι Ενέργειες εκτελούνται με αυστηρή σειρά, υπάρχει πάντοτε βεβαιότητα για την τρέχουσα κατάσταση μετά από κάθε βήμα. Αυτό καθιστά τον αλγόριθμο ταχύτερο και πιο αποδοτικό στα περισσότερα σενάρια.

Στην Μερική Διάταξη οι περιορισμοί είναι πιο χαλαροί, μπορεί να ορίζονται οι ενέργειες $T1, T2$ και $T3$, και να υπάρχει ο περιορισμός $T1 < T3$, αλλά επίσης $T2 < T3$, χωρίς να υπάρχει κάποια οδηγία για την σειρά εκτέλεσης της $T1$ σε σχέση με την $T2$. Αυτό επιτρέπει την παράλληλη εκτέλεση και εναλλαγή ενεργειών, παρόλο που φαίνεται να παρέχεται μεγαλύτερη ευελιξία, με την Μερική Διάταξη η πολυπλοκότητα αναζήτησης αυξάνεται εκθετικά, καθώς ο Planner πρέπει να εξετάσει όλους τους πιθανούς συνδυασμούς αλληλουχίας για να διασφαλίσει την ορθή λειτουργία του.

Χαρακτηριστικό	Ολική Διάταξη (Total-Order)	Μερική Διάταξη (Partial-Order)
Σειρά Εκτέλεσης	Καθορισμένη (Γραμμική)	Ελαστική (Παράλληλη)
Πολυπλοκότητα	Χαμηλότερη Μικρότερος χώρος αναζήτησης	Υψηλή Εκθετική αύξηση συνδυασμών
Γνώση Κατάστασης	Πλήρης βεβαιότητα σε κάθε βήμα	Αβεβαιότητα λόγω πολλαπλών διαδρομών
Αποδοτικότητα	Ταχύτερη εύρεση πλάνου	Πιο ευέλικτο αλλά αργό πλάνο

Πίνακας 2.1: Σύγκριση Ολικής και Μερικής Διάταξης στον Ιεραρχικό Σχεδιασμό

2.4 Γλώσσες Περιγραφής Προβλημάτων

Για την επιτυχή επίλυση ενός προβλήματος σχεδιασμού απαιτείται ένας τυπικός και αυστηρός τρόπος περιγραφής του. Ο Planner πρέπει να γνωρίζει τους κανόνες του πεδίου, τις διαθέσιμες ενέργειες, τους στόχους και τα αντικείμενα του κόσμου, χωρίς ασάφειες. Για τον σκοπό αυτό έχουν αναπτυχθεί ειδικές γλώσσες μοντελοποίησης προβλημάτων, οι οποίες λειτουργούν ως ενδιάμεσο επίπεδο επικοινωνίας μεταξύ του σχεδιαστή του προβλήματος και του αλγορίθμου επίλυσης.

2.4.1 Η γλώσσα PDDL (*Planning Domain Definition Language*)

Η κυρίαρχη γλώσσα περιγραφής προβλήματος στο πλαίσιο του Κλασικού Αυτοματοποιημένου Σχεδιασμού είναι η PDDL (*Planning Domain Definition Language*). Δημιουργήθηκε το 1998, με αφορμή τις ανάγκες του πρώτου Διεθνούς Διαγωνισμού Σχεδιασμού (*International Planning Competition – IPC*), και έκτοτε καθιερώθηκε ως το διεθνές πρότυπο για την αναπαράσταση προβλημάτων σχεδιασμού. Η βασική αρχή της PDDL είναι ο διαχωρισμός της αναπαράστασης του κόσμου σε δύο διακριτά αρχεία, το αρχείο Πεδίου (*Domain file*) και το αρχείο Προβλήματος (*Problem file*). [1] [2]

Το *Domain file* σε PDDL ορίζει τους αμετάβλητους κανόνες που διέπουν το περιβάλλον. Τα βασικά συστατικά του είναι τα εξής:

- (*:types*): Ιεραρχία τύπων αντικειμένων (π.χ. *robot - vehicle*)
- (*:constants*): Σταθερά αντικείμενα (*constants C - physics*).
- (*:predicates*): Λογικά κατηγορήματα που περιγράφουν τις ιδιότητες και τις σχέσεις των αντικειμένων (π.χ. *at(?r ?l)*).
- (*:action*): Σχήματα ενεργειών που περιλαμβάνουν παραμέτρους (*parameters*), τις προϋποθέσεις (*precondition*) και τα αποτελέσματα (*effect*) (*add/delete*).

Ένα *Problem file* σε PDDL ορίζει το συγκεκριμένο στιγμιότυπο του προβλήματος. Τα βασικά συστατικά του είναι τα εξής:

- (*:objects*): Αντικείμενα (π.χ. *car1 – car*).
- (*:init*): Αρχική Κατάσταση του Κόσμου
- (*:goal*): Στόχος (*conjunction/disjunction/exists*).

Ωστόσο, η κλασική PDDL σχεδιάστηκε για να περιγράφει ενέργειες και καταστάσεις, όχι ιεραρχίες. Με την άνοδο του Ιεραρχικού Σχεδιασμού (*HTN Planning*), δημιουργήθηκε η ανάγκη για μία γλώσσα που μπορεί να εκφράσει την έννοια της ιεραρχίας μέσω Ενεργειών με τη διάκριση υψηλού και χαμηλού επιπέδου (*Compound Tasks / Primitive Actions*) και Μεθόδων Αποσύνθεσης (*Decomposition Methods*).

2.4.2 Η επέκταση *HDDL (Hierarchical Domain Definition Language)*

Η HDDL (Hierarchical Domain Definition Language) αποτελεί επέκταση της PDDL, είναι ειδικά σχεδιασμένη για Ιεραρχικό Σχεδιασμό τύπου HTN. Βασίζεται στο STRIPS-fragment της PDDL 2.1 και προσθέτει έννοιες όπως Compound Tasks, Primitive Actions, Decomposition Methods και Task Networks [3].

Το Domain file επεκτείνει την PDDL στα εξής βασικά συστατικά:

- (:task): Ορισμός Compound Tasks, παρόμοια με τα Actions, χωρίς Preconditions/Effects, αλλά με parameters.
- (:method): Αποσύνθεση Compound Tasks, με parameters, decomposed task, προαιρετικά preconditions και task-network (ordered-unordered subtasks) και constraints (not ?x ?y)
- (:action): Primitive tasks, ίδια έννοια όπως PDDL STRIPS

Το Problem file σε HDDL διαφοροποιείται με:

- (:htn): Αντί για απλό στόχο κατάστασης, ορίζεται Αρχικό Δίκτυο Εργασιών, το οποίο περιέχει τις Εργασίες υψηλού επιπέδου που πρέπει να αποσυνθέσει ο πράκτορας, καθώς και τους περιορισμούς διάταξης μεταξύ τους (π.χ. root compound task).

3 Το Πεδίο Εφαρμογής

3.1 Ο Διεθνής Διαγωνισμός Σχεδιασμού

Ο Διεθνής Διαγωνισμός Σχεδιασμού (International Planning Competition - IPC) αποτελεί έναν κορυφαίο θεσμό σύγκρισης αλγορίθμων και συστημάτων αυτοματοποιημένου σχεδιασμού (Automated Planning). Ο IPC διοργανώνεται περιοδικά στο πλαίσιο του Διεθνούς Συνεδρίου Σχεδιασμού και Προγραμματισμού (International Conference on Automated Planning and Scheduling - ICAPS), όπου εκεί αξιολογούνται εμπειρικά, συστήματα σχεδιασμού τελευταίας τεχνολογίας σε μία σειρά από προβλήματα αναφοράς, καθώς λειτουργεί ως πρότυπο της κοινότητας σχεδιασμού. Οι στόχοι του IPC είναι η προώθηση της έρευνας σχεδιασμού, η ανάδειξη προκλήσεων στην επιστημονική κοινότητα, καθώς επίσης και η παροχή νέων και ενδιαφερόντων προβλημάτων ως σημεία αναφοράς για μελλοντική έρευνα. Η συγκεκριμένη ενότητα έχει ως στόχο την παρουσίαση του θεσμού του IPC, θέτοντας έτσι το πλαίσιο, το οποίο πραγματεύεται η παρούσα πτυχιακή εργασία. [4] [5] [6]

3.1.1 Ιστορία και Σημασία

Ο Διεθνής Διαγωνισμός Σχεδιασμού (IPC) ως θεσμός ξεκίνησε το 1998 με σκοπό να τυποποιήσει την εμπειρική αξιολόγηση για την σύγκριση των διαφορετικών συστημάτων σχεδιασμού. Πριν την θέσπιση του IPC, οι ερευνητές δεν είχαν κάποια κοινή βάση με την οποία θα αξιολογούσαν την ποιότητα ενός σχεδιαστικού συστήματος, γεγονός το οποίο καθιστούσε ασαφή τη μέτρηση της προόδου στον τομέα. Ο διαγωνισμός έθεσε μία κοινή βάση αξιολόγησης, συστήνοντας μια ομάδα προβλημάτων με σαφή και αυστηρά κριτήρια αξιολόγησης της απόδοσης, λύνοντας έτσι το πρόβλημα της αξιολογικής ασάφειας του τομέα.

Η σημασία του IPC έχει αποδειχθεί καίρια για την επιστημονική κοινότητα σχεδιασμού, καθώς πριν καν πραγματοποιηθεί ο πρώτος διαγωνισμός είχε ήδη γίνει πρόοδος στον τομέα με πρόφαση το IPC. Πρόκειται για τη δημιουργία της PDDL στην οποία αφιερώθηκε το υποκεφάλαιο 2.4, βέβαια η άνοδος δεν σταμάτησε εκεί. Σε κάθε νέα εκδήλωση του διαγωνισμού, τα προβλήματα που εισάγονται είναι όλο και πιο απαιτητικά, γεγονός που ωθεί την κοινότητα να δημιουργήσει νέες τεχνικές και να αναπτύξει νέους μηχανισμούς για την επίλυση των προβλημάτων. Ο IPC φαίνεται να ωθεί την κοινότητα σχεδιασμού σε συνεχή ανάπτυξη και πρόοδο, επιβραβεύοντας φυσικά τις αποδοτικότερες προσεγγίσεις. Αρκετές από τις γνωστές τεχνικές στον αυτοματοποιημένο σχεδιασμό, όπως τον ορίζουμε σήμερα, αναδείχθηκαν χάρη στην επιτυχία τους σε διάφορους τομείς του συγκεκριμένου θεσμού, χαρακτηριστικά παραδείγματα αποτελούν οι Relaxed Planning Graph Heuristics (1998), Enforced Hill Climbing και Relaxed Plans (2000), καθώς επίσης και Landmark Heuristics (2008).

3.1.2 Οι πτυχές του Διαγωνισμού

Είναι σημαντικό να σημειωθεί πως στα αρχικά στάδια του θεσμού, το ενδιαφέρον του International Planning Competition (IPC) μονοπωλούσε σχεδόν αποκλειστικά η Κλασική Σχεδίαση (Classical Planning). Οι Planners καλούνταν κυρίως να αναπτύξουν συστήματα ικανά να επιλύσουν προβλήματα ντετερμινιστικής φύσης, εστιάζοντας στη αναζήτηση λύσεων σε πλήρως παρατηρήσιμα περιβάλλοντα.

Ωστόσο, καθώς η έρευνα Τεχνητής Νοημοσύνης εξελισσόταν, προέκυψαν ανάγκες για πιο ρεαλιστικές και απαιτητικότερες εφαρμογές, έτσι ο διαγωνισμός διεύρυνε σταδιακά τους ορίζοντες του. Πλέον, στο πλαίσιο του IPC 2023, η δομή του διαγωνισμού έχει οργανωθεί έτσι ώστε να καλύπτει ένα ευρύ φάσμα ερευνητικών περιοχών, παρουσιάζοντας τα εξής Tracks:

- Classical Tracks: Αποτελεί την συνέχεια της παραδοσιακής μορφής του διαγωνισμού και χωρίζεται σε υποκατηγορίες, βέλτιστο (optimal), ευέλικτο (agile) και ικανοποιητικό (satisficing) σχεδιασμό. Οι Planners καλούνται να λύσουν ντετερμινιστικά προβλήματα αναζήτησης σε χώρο καταστάσεων. [6]
- Learning Tracks: Εστιάζει σε αλγορίθμους που εκπαιδεύονται σε ένα συγκεκριμένο πεδίο, αποκτούν γνώση και έπειτα την χρησιμοποιούν για την αποδοτικότερη επίλυση νέων, άγνωστων προβλημάτων ίδιου πεδίου. [7]
- Probabilistic Tracks: Πραγματεύεται προβλήματα όπου υπάρχει αβεβαιότητα στα αποτελέσματα των ενεργειών, προσομοιώνοντας στοχαστικά περιβάλλοντα. Οι Planners καλούνται να λάβουν αποφάσεις που μεγιστοποιούν την αναμενόμενη ανταμοιβή υπό καθεστώς αβεβαιότητας. [8]
- Numeric Tracks: Ασχολείται με προβλήματα σχεδιασμού που περιλαμβάνουν αριθμητικές μεταβλητές και περιορισμούς, απαιτώντας χειρισμό αριθμητικών πράξεων ως μέρος της διαδικασίας λήψης αποφάσεων. [9]
- HTN Tracks: Αφορά τον ιεραρχικό Σχεδιασμό, όπου ένας σύνθετος στόχος αναλύεται σταδιακά σε απλούστερες υποεργασίες, μέχρι να προκύψει ένα εκτελέσιμο πλάνο αποτελούμενο αποκλειστικά από πρωτογενείς ενέργειες. Εξετάζονται σενάρια τόσο με ολική όσο και με μερική διάταξη των εργασιών. Αξίζει να σημειωθεί ότι αυτό είναι και το Track το οποίο εξετάζει η παρούσα εργασία. [10]

Η συγκεκριμένη κατηγοριοποίηση των Tracks, δεν εξυπηρετεί μόνο οργανωτικούς σκοπούς, αλλά επιτρέπει την αξιοκρατική σύγκριση των συστημάτων, καθώς κάθε Planner αξιολογείται στο πεδίο ειδικότητας του. Παράλληλα χαρτογραφεί με σαφήνεια

την πρόοδο της επιστημονικής κοινότητας σε κάθε επιμέρους κλάδο του Αυτοματοποιημένου Σχεδιασμού.

Track	Γλώσσα Περιγραφής	Είδος Προβλήματος
Classical	PDDL	Ντετερμινιστικό / Λογικό
Learning	PDDL	Άγνωστο
Probabilistic	PPDDL & RDDDL	Πιθανοτικό / Στοχαστικό
Numeric	PDDL w/ numeric fluents	Ντετερμινιστικό / Αριθμητικό
HTN	HDDL	Ιεραρχική Αποσύνθεση

Πίνακας 3.1: Πίνακας Πληροφοριών των tracks του IPC

3.1.3 To HTN Track του IPC 2023

Το Track του Ιεραρχικού Σχεδιασμού (HTN Track) του IPC 2023 αποτέλεσε τη δεύτερη κατά σειρά διοργάνωση στην ιστορία του θεσμού που εστιάζει αποκλειστικά στον Ιεραρχικό Σχεδιασμό, ακολουθώντας την εναρκτήρια προσπάθεια του 2020. Στόχος του συγκεκριμένου Track είναι η αξιολόγηση συστημάτων σχεδιασμού που δεν αρκούνται στην απλή αλληλουχία εργασιών, αλλά βασίζονται στην αποδόμηση σύνθετων αφηρημένων εργασιών (Tasks) σε απλούστερες υποεργασίες, μέχρι την παραγωγή ενός εκτελέσιμου πλάνου αποτελούμενου αποκλειστικά από πρωτογενείς Ενέργειες (Actions) [10].

Στο συγκεκριμένο Track, ο διαγωνισμός χωρίζεται σε κύριους κλάδους με βάση την δομή του δικτύου εργασιών:

- Total-Order (Ολική Διάταξη): Αφορά προβλήματα όπου τα δίκτυα εργασιών αποτελούν πλήρως διατεταγμένες ακολουθίες.
- Partial-Order (Μερική Διάταξη): Αφορά προβλήματα όπου τα δίκτυα επιτρέπουν οποιαδήποτε αυθαίρετη μερική διάταξη, προσφέροντας ευελιξία στη σειρά εκτέλεσης.

Ο κάθε ένας από τους παραπάνω κλάδους περιλαμβάνει τρεις υποκατηγορίες αξιολόγησης:

- Optimal Track: Απαιτείται η εγγυημένη εύρεση του βέλτιστου πλάνου, και η βαθμολογία καθορίζεται από το πλήθος των διαφορετικών προβλημάτων που λύθηκαν βέλτιστα.
- Satisficing Track: Στόχος είναι η εύρεση πλάνων υψηλής ποιότητας και χαμηλού κόστους. Η βαθμολογία βασίζεται στη σχέση μεταξύ του κόστους του βέλτιστου γνωστού πλάνου έναντι του παραγόμενου πλάνου.

- Agile Track: Εστιάζει στην ταχύτητα εύρεσης λύσης. Κατά την αξιολόγηση αγνοείται το κόστος του πλάνου, και η βαθμολογία βασίζεται αποκλειστικά στον χρόνο.

Ως γλώσσα περιγραφής προβλημάτων χρησιμοποιήθηκε η γλώσσα HDDL (Hierarchical Domain Definition Language). Το σετ προβλημάτων βασίζεται κυρίως στα domains του 2020, εμπλουτισμένο ωστόσο με νέα domains για αύξηση της πολυπλοκότητας.

3.2 Ανάλυση του Domain “Barman”

Για την συγκεκριμένη εργασία, επιλέχθηκε η δημιουργία Planner για το πρόβλημα που ορίζει το πεδίο “Barman” σε Satisficing Tracks με διάταξη Total-Order. Πρόκειται για ένα κλασικό πρόβλημα σχεδιασμού, που μοντελοποιεί τη διαδικασία παρασκευής κοκτέιλ από έναν μπάρμαν, προσαρμοσμένο για Ιεραρχικό Σχεδιασμό (HTN) [11].

3.2.1 Γενική Επισκόπηση

Το Domain ορίζεται ως `barman_htn` και χρησιμοποιεί τη γλώσσα HDDL. Απαιτεί μηχανισμούς διαχείρισης αρνητικών προϋποθέσεων (negative-preconditions), ιεραρχίας (hierarchy) και ισότητας (equality).

Ο βασικός στόχος του μπάρμαν είναι να συνδυάσει συγκεκριμένα συστατικά (ingredients) για να δημιουργήσει κοκτέιλ (cocktail), μέσω ανάμειξης (shaker) και να τα σερβίρει σε σφηνάκια (shots).

3.2.1.1 Ιεραρχία Τύπων (Types)

Το πεδίο ορίζει μία σαφή οντολογία αντικειμένων:

- Containers: Περιλαμβάνει όλα τα αντικείμενα του τύπου shaker και shot
- Beverage: Περιλαμβάνει όλα τα αντικείμενα τύπου ingredient και cocktail.
- Anything: Η γενική κλάση, η οποία περιλαμβάνει όλους τους τύπους, όπως container, dispenser, level, beverage και hand.

3.2.1.2 Κατηγορήματα (Predicates) και Καταστάσεις (States)

Το σύστημα παρακολουθεί την κατάσταση του κόσμου, μέσω μιας σειράς κατηγορημάτων [11]:

- Κατηγορήματα κατάστασης δοχείων και υγιεινής: Ορίζουν την λειτουργική κατάσταση των σκευών:
 - clean(?c - container): Υποδηλώνει ότι το δοχείο ?c είναι καθαρό.
 - empty(?c - container): Υποδηλώνει ότι το δοχείο ?c είναι άδειο.
 - ontable(?c - container): Υποδηλώνει ότι το δοχείο ?c βρίσκεται τοποθετημένο στο τραπέζι.
 - used(?c - container ?b - beverage): Υποδηλώνει ότι το δοχείο ?c έχει χρησιμοποιηθεί για το ποτό ?b, και απαιτεί καθαρισμό.
- Κατηγορήματα Περιεχομένου και Συνταγών:
 - cocktailPart1(?c - cocktail ?i - ingredient): Ορίζει το ?i ως το πρώτο συστατικό της συνταγής του κοκτέιλ ?c.
 - cocktailPart2(?c - cocktail ?i - ingredient): Ορίζει το ?i ως το δεύτερο συστατικό που απαιτείται για τη συνταγή του κοκτέιλ ?c.
 - contains(?c - container ?b - beverage): Υποδηλώνει ότι το δοχείο ?c περιέχει το ποτό ή το συστατικό ?b.
 - dispenses(?d - dispenser ?i - ingredient): Συνδέει έναν διανεμητή ?d με ένα συστατικό ?i που παρέχει.
- Κατηγορήματα Χειρισμού και Στάθμης:
 - handEmpty(?h - hand): Δηλώνει ότι το χέρι ?h είναι ελεύθερο.
 - holding(?h - hand ?c - container): Υποδηλώνει ότι ο μπάρμαν κρατάει το δοχείο ?c με το χέρι ?h.
 - shakerLevel(?s - shaker ?l - level): Καθορίζει ότι το επίπεδο ?l αντιστοιχεί στην τρέχουσα κατάσταση πληρότητας του shaker ?s.
 - shakerEmptyLevel(?s - shaker ?l - level): Καθορίζει ότι το επίπεδο ?l αντιστοιχεί στην άδεια κατάσταση του shaker ?s.
 - next(?l1 - level ?l2 - level): Ορίζει την διαδοχή των επιπέδων, δηλώνοντας ότι το επίπεδο ?l2 έρχεται μετά το επίπεδο ?l1.
 - shaken(?s - shaker) / unshaken(?s - shaker): Υποδηλώνουν αν το περιεχόμενο του shaker έχει ανακατευτεί κατάλληλα.

3.2.2 Δομή HTN: Tasks και Methods

Η επίλυση του προβλήματος στον Ιεραρχικό Σχεδιασμό βασίζεται στην αποδόμηση Σύνθετων Ενεργειών στόχων σε απλούστερες Εργασίες. Το domain οργανώνει αυτή τη διαδικασία με τον ορισμό Compound Tasks, Decomposition Methods και Primitive Actions [1], [13], [14].

3.2.2.1 Ενέργειες Υψηλού Επιπέδου (Compound Tasks)

Το domain “Barman” ορίζει μία σειρά αποτελούμενο από δέκα (10) Ενέργειες υψηλού επιπέδου (Tasks) και αντιπροσωπεύουν τους στόχους που επιδιώκει ο Planner [11]:

- Εργασίες Παρασκευής και Σερβιρίσματος:
 - AchieveContainsShotCocktail(?s – shot ?c – cocktail): Ο τελικός στόχος για τη ύπαρξη ενός κοκτέιλ σε σφηνάκι.
 - AchieveContainsShakerCocktail(?s – shaker ?c - cocktail): Η διαδικασία ανάμειξης του κοκτέιλ εντός του δοχείου (shaker).
 - DoPourShakerToShot(?s – shaker ?t – shot ?c - cocktail): Η μεταφορά περιεχομένου του shaker στο shot.
- Εργασίες Διαχείρισης Συστατικών:
 - AchieveContainsShakerIngredient(?s – shaker ?i – ingredient)
 - AchieveContainsShotIngredient(?s – shot ?i – ingredient)

Εξασφαλίζουν την τοποθέτηση του συγκεκριμένου συστατικού στο αντίστοιχο δοχείο (shaker/shot).
- Εργασίες Υποδομής και Καθαριότητας:
 - AchieveOnTable(?c - container): Διασφαλίζει ότι το σκεύος βρίσκεται τοποθετημένο στο τραπέζι.
 - AchieveHolding(?h – hand ?c- container): Εξασφαλίζει ότι το δοχείο (container) κρατιέται από το συγκεκριμένο χέρι (hand).
 - AchieveHandEmpty(?h – hand): Διασφαλίζει πως το εν λόγω χέρι είναι ελεύθερο.
 - AchieveCleanShaker(?s – shaker)
 - AchieveCleanShot(?s – shot)

Εξασφαλίζουν ότι τα δοχεία (shaker/shot) είναι καθαρά.

3.2.2.2 Μέθοδοι Αποσύνθεσης (Decomposition Methods)

Οι μέθοδοι αποσύνθεσης αποτελούν την κύρια “στρατηγική” του domain, καθορίζοντας τον τρόπο με τον οποίο κάθε Σύνθετη Ενέργεια (Task) αναλύεται σε Υποεργασίες (Subtasks) ανάλογα με την τρέχουσα κατάσταση του κόσμου. Το domain “Barman” ορίζει είκοσι-δύο (22) Μεθόδους Αποσύνθεσης, ενδεικτικά οι κυριότερες εκ των οποίων είναι [11]:

- Στρατηγικές Παρασκευής Κοκτέιλ:
 - MakeAndPourCocktail(?t – shot, ?c – cocktail, ?s – shaker, ?h – hand): Η κύρια μέθοδος που συνδυάζει παρασκευή του shaker, καθαρισμό του shot και το σερβίρισμα του ποτού.
 - MakeCocktail (?s - shaker ?c - cocktail ?i - ingredient ?h - hand ?h - hand ?i - ingredient): Περιγράφει τα βήματα ανάμειξης του κοκτέιλ.
- Διαχείριση Ροής Συστατικών:
 - AddIngredientToEmptyShaker(?s – shaker ?i – ingredient ?l – level ?l – level ?s – shot ?h – hand)
 - AddIngredientToUsedShaker(?s – shaker ?i – ingredient ?l – level, ?l – level ?s – shot ?h – hand)

Επιλέγουν την κατάλληλη ενέργεια πλήρωσης του δοχείου, ανάλογα με το αν το shaker είναι άδειο και καθαρό ή χρησιμοποιημένο.

 - AddIngredientToShot (?s - shot ?i- ingredient ?d - dispenser ?h - hand ?h - hand): Διασφαλίζει τον καθαρισμό του shot και το γέμισμα του από το dispenser.
- Μέθοδοι Καθαρισμού και Συντήρησης:
 - CleanFullShot (?s - shot ?h – hand ?b – beverage ?h - hand)
 - CleanEmptyShaker(?s – shaker ?h – hand ?h - hand)

Περιλαμβάνει την διαδικασία αδειάσματος, εφόσον αυτό είναι απαραίτητο, και την εκτέλεση του καθαρισμού του δοχείου.
- Μέθοδοι Βελτιστοποίησης (Null Methods):

Περιλαμβάνουν μεθόδους όπως η CleanShotNull, η HandEmptyNull και η MakeCocktailNull. Αυτές οι μέθοδοι ενεργοποιούνται όταν η επιθυμητή κατάσταση ισχύει ήδη, επιτρέποντας έτσι στον Planner να αποσυνθέσει ένα Task χωρίς την εκτέλεση περιττών Ενεργειών.

3.2.2.3 Πρωτογενείς Ενέργειες (Primitive Actions)

Οι Ενέργειες αυτές ορίζονται ως χαμηλού επιπέδου Ενέργειες (ή Πρωτογενείς) και η εύρεση τους σε κατάλληλη ακολουθία αποτελεί κύριο στόχο στον Ιεραρχικό Σχεδιασμό. Ορίζονται από Προϋποθέσεις (preconditions) που πρέπει να ικανοποιούνται για να εκτελεστούν και από Αποτελέσματα (effects) που περιγράφουν τις αλλαγές που επιφέρει η εκτέλεση της εκάστοτε Ενέργειας. Το domain ορίζει έντεκα (11) actions [11] :

- Ενέργειες Χειρισμού και Μετακίνησης:
 - grasp (?h - hand ?c – container): Ο μπάρμαν χρησιμοποιεί το χέρι ?h για να πιάσει το δοχείο ?c από το τραπέζι. Προϋποθέτει ότι το δοχείο βρίσκεται επί του τραπεζιού και πως το χέρι είναι ελεύθερο.
 - drop (?h - hand ?c – container): Ο μπάρμαν αφήνει το δοχείο ?c στο τραπέζι, απελευθερώνοντας το χέρι του ?h, αποδεσμεύοντας παράλληλα το δοχείο από το χέρι.
- Ενέργειες Προετοιμασίας και Καθαρισμού:
 - fill-shot (?s – shot ?i – ingredient ?h – hand ?h -hand ?d - dispenser): Ο μπάρμαν γεμίζει ένα άδειο, καθαρό σφηνάκι ?s με το συστατικό ?i του αντίστοιχου διανεμητή ?d. Η ενέργεια αυτή καθιστά το shot χρησιμοποιημένο και όχι πλέον καθαρό.
 - clean-shot (?s – shot ?h – hand ?h - hand)
 - clean-shaker (?s – shaker ?h – hand ?h - hand)
Ο μπάρμαν καθαρίζει ένα ήδη άδειο δοχείο (shot/shaker) και απαιτεί και τα δύο χέρια για την συγκεκριμένη ενέργεια.
 - empty-shaker(?h – hand ?s – shaker ?c – cocktail ?l - level ?l-level)
 - empty-shot(?h – hand ?s – shot ?b - beverage)
Ο μπάρμαν αδειάζει το περιεχόμενο των αντίστοιχων δοχείων (shot/shaker) χωρίς να τα καθαρίσει.
- Ενέργειες Ανάμειξης και Μεταφοράς Ποτού:
 - pour-shot-to-clean-shaker (?s - shot ?i - ingredient ?s - shaker ?h - hand ?l - level ?l - level)
 - pour-shot-to-used-shaker (?s - shot ?i - ingredient ?s - shaker ?h - hand ?l - level ?l - level)
Μεταφέρει το περιεχόμενο ενός shot στο shaker. Η ενέργεια ενημερώνει τη στάθμη του shaker από το τρέχον επίπεδο στο επόμενο.
 - pour-shaker-to-shot (?c - cocktail ?s - shot ?h - hand ?s - shaker ?l - level ?l - level): Μεταφέρει το έτοιμο κοκτέιλ από το shaker σε ένα καθαρό shot. Χαμηλώνει τη στάθμη του shaker και καθιστά το shot γεμάτο με το κοκτέιλ.
 - shake (?c - cocktail ?i - ingredient ?i - ingredient ?s - shaker ?h - hand ?h - hand): Η βασική ενέργεια παρασκευής όπου τα συστατικά αναμειγνύονται στο shaker για να δημιουργήσουν το κοκτέιλ. Το shaker μεταπίπτει από την κατάσταση unshaked σε shaken.

3.2.3 Προκλήσεις, Περιορισμοί και Πολυπλοκότητα του “Barman”

Το domain “Barman” παρουσιάζει ιδιαίτερο ενδιαφέρον στον Ιεραρχικό Σχεδιασμό, καθώς η επίλυση ενός προβλήματος δεν είναι μια απλή γραμμική ακολουθία ενεργειών, αλλά μία σύνθετη διαδικασία διαχείρισης πόρων και καταστάσεων. [11]

3.2.3.1 Διαχείριση Πολλαπλών Στόχων και Επαναφορά Κατάστασης

Στην συντριπτική πλειονότητα των σεναρίων, ο Planner έρχεται αντιμέτωπος με παραπάνω από έναν στόχο προς αποσύνθεση. Αυτό δημιουργεί την ανάγκη για μία κυκλική και αναδρομική διαδικασία.

- Αποσύνθεση και Καθαρισμός: Μόλις ολοκληρωθεί η αποσύνθεση του Task στόχου, δηλαδή μόλις ένα κοκτέιλ παρασκευαστεί και σερβιριστεί στο σφηνάκι, ο κόσμος βρίσκεται σε μία αναστατωμένη κατάσταση. Το shaker είναι χρησιμοποιημένο, ο μπάρμαν κρατάει αντικείμενα, οι στάθμες των υγρών έχουν αλλάξει και τα δοχεία δεν είναι καθαρά.
- Επαναφορά του Κόσμου: Με βάση την κατάσταση του κόσμου έχοντας ήδη αποσυνθέσει ένα Task, έχοντας ήδη την ακολουθία από Primitive Actions για την αποδόμηση του Task, για να ξεκινήσει η επόμενη Παρασκευή, ο Planner πρέπει να “καθαρίσει” το περιβάλλον. Οφείλουν να ληφθούν νέες Ενέργειες, να προστεθούν δηλαδή στην λίστα εκτέλεσης των Actions, περισσότερες Πρωτογενείς Ενέργειες για την επαναφορά του κόσμου σε ένα στάδιο που δεν εμποδίζει την έναρξη επίλυσης του επόμενου Task-Στόχου.
- Διατήρηση Στόχων: Ο αλγόριθμος πρέπει να επιτύχει τον επόμενο στόχο χωρίς να αναιρέσει τους προηγούμενους. Για παράδειγμα, ενώ καθαρίζει το shaker για το δεύτερο κοκτέιλ, πρέπει να διασφαλίσει ότι δεν θα αδειάσει το ήδη γεμάτο σφηνάκι που αποτελεί μέρος του πρώτου επιτυχούς στόχου.

3.2.3.2 Περιορισμοί Πόρων και Χειρισμού

Η πολυπλοκότητα αυξάνεται δραματικά λόγω των φυσικών περιορισμών του barman.

- Περιορισμός Χεριών: Ο μπάρμαν διαθέτει μόνο δύο χέρια, Ενέργειες όπως ο καθαρισμός απαιτούν το ένα χέρι να κρατά το σκεύος και το άλλο χέρι να είναι ελεύθερο. Αυτό επιβάλλει στον Planner να “σχεδιάζει” συνεχώς κινήσεις απελευθέρωσης και ανάκτησης σκευών.
- Διαδοχή Επιπέδων: Το κατηγορημα next επιβάλλει μια αυστηρά γραμμική αύξηση της στάθμης. Ο Planner δεν μπορεί να προσθέσει

συστατικά αν δεν έχει προηγηθεί η σωστή μετάβαση από το προηγούμενο επίπεδο, καθιστώντας τον χώρο αναζήτησης ευαίσθητο στη σειρά ενεργειών.

3.2.3.3 Υλοποίηση με Java και DFS

Η επιλογή της Java και του αλγορίθμου DFS για την επίλυση του domain επηρεάζει τον τρόπο που εξερευνάται η πολυπλοκότητα:

- Αναζήτηση σε Βάθος: Ο DFS δεν είναι ο πρώτος αλγόριθμος που επιλέγεται σε τέτοιου είδους συστήματα, αλλά εν τέλει αποδείχθηκε ταιριαστός για Total-Order HTN, καθώς ακολουθεί το μονοπάτι μίας μεθόδου μέχρι να φτάσει σε Πρωτογενείς Ενέργειες. Αυτό επιτρέπει τη γρήγορη εύρεση μίας πρώτης λύσης, αρκεί οι Μέθοδοι να είναι καλά ορισμένες.
- Backtracking και Αδιέξοδα: Λόγω των περιορισμών, ο DFS μπορεί να οδηγηθεί σε αδιέξοδα. Αν ο μπάρμαν κρατάει δύο λερωμένα σκεύη και δεν έχει ελεύθερο χέρι για καθαρισμό, ο αλγόριθμος πρέπει να οπισθοδρομήσει σε προηγούμενο στάδιο επιλογής χεριού ή σκεύους.
- Διαχείριση Μνήμης: Η Java παρέχει τη δυνατότητα αναδρομικής κλήσης των Μεθόδων Αποσύνθεσης, επιτρέποντας στον Planner να διαχειρίζεται τα Σύνθετα Tasks ως μία στοίβα (stack) Εργασιών, κάτι που ταιριάζει απόλυτα με τη φύση του DFS.

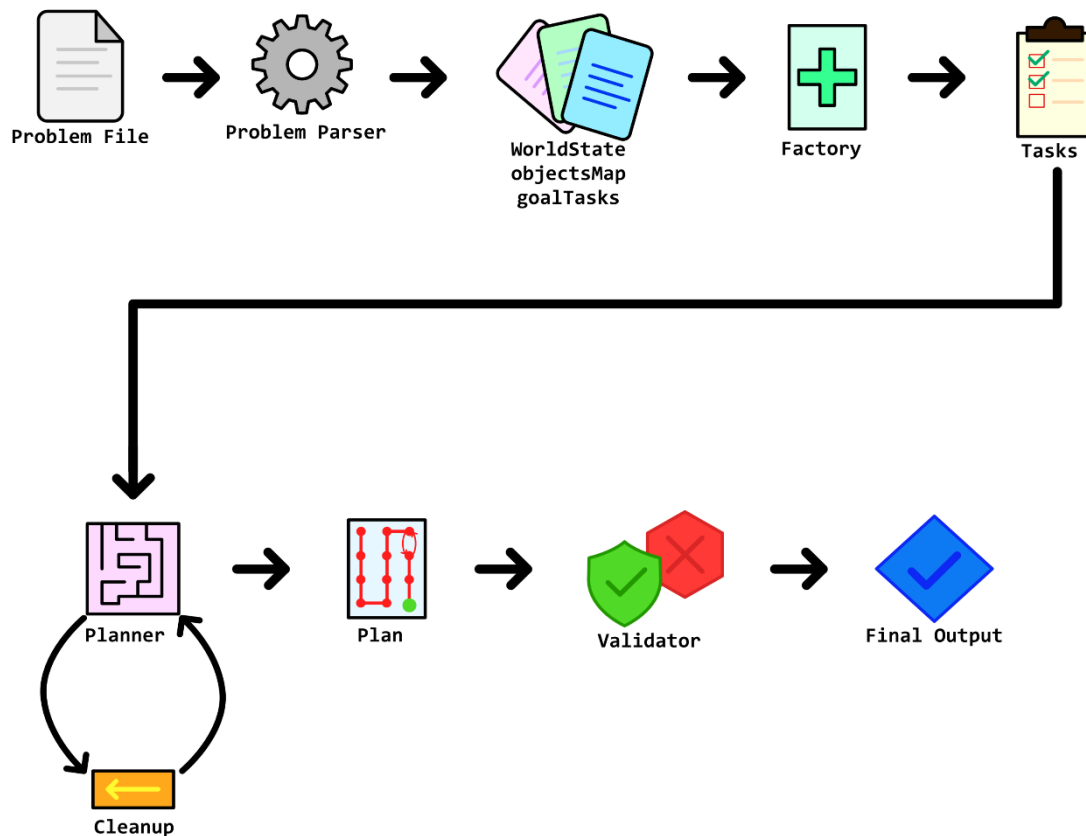
Παράμετρος	Επίπτωση στον Planner
Αναδρομικότητα	Υποχρεωτική επαναφορά του κόσμου σε «αρχική» μορφή μετά από την αποσύνθεση κάποιου Task-Στόχου.
Περιορισμός Χεριών	Συχνές ενέργειες grasp/drop που επιμηκύνουν το τελικό πλάνο.
DFS Backtracking	Ανάγκη για επιστροφή σε προηγούμενα βήματα αν δεν επιλεγεί η κατάλληλη Μέθοδος Αποσύνθεσης.
Total-Order	Αυστηρή σειρά εκτέλεσης που δεν επιτρέπει παραλληλισμό Εργασιών.

Πίνακας 3.2: Επιπτώσεις στον Planner λόγω χρήσης Java

4 Μεθοδολογία και Υλοποίηση Συστήματος

4.1 Αρχιτεκτονική Προσέγγιση

Η αρχιτεκτονική σχεδίαση του project «Barman» υλοποιεί έναν εξειδικευμένο στο πεδίο (Domain-specific) σχεδιαστή ιεραρχικών δικτύων εργασιών (HTN Planner), ο οποίος αναπτύχθηκε εξ ολοκλήρου σε γλώσσα προγραμματισμού Java. Το σύστημα είναι σχεδιασμένο ώστε να έχει την δυνατότητα να αναγνώσει στιγμιότυπα προβλημάτων, να διατηρεί αυστηρά την κατάσταση του περιβάλλοντος, να αναλύει αναδρομικά τις αφηρημένες Εργασίες σε εκτελέσιμες Πρωτογενείς Ενέργειες (Primitive Actions) και να επικυρώνει τεχνικά το παραγόμενο πλάνο εκτέλεσης έναντι των αυστηρών κανόνων του πεδίου. Η αρχιτεκτονική του σχεδιαστή είναι ιδιαίτερα αρθρωτή, διαχωρίζοντας σαφώς την ανάλυση εισόδου (Input-Parsing), την αναπαράσταση κατάστασης, τον αλγόριθμο σχεδιασμού, των εργασιών επικύρωσης που ακολουθούν την εύρεση λύσης, καθώς επίσης και την διαδικασία επαναφοράς του περιβάλλοντος στην αρχική του κατάσταση.



Σχήμα 4.1: Διάγραμμα Αρχιτεκτονικής του Συστήματος

4.1.1 Επισκόπηση Ροής Δεδομένων

Η ροή δεδομένων του συγκεκριμένου συστήματος ακολουθεί μία σαφή και διαδοχική γραμμή επεξεργασίας (Pipeline), από την αρχική εισαγωγή του προβλήματος έως την τελική παραγωγή του πλάνου. Αποτελείται από πολλαπλά άμεσα συνδεδεμένα υποσυστήματα, τα οποία μεταβιβάζουν συγκεκριμένες δομές δεδομένων κατά την «αλυσίδα» της εκτέλεσης.

Το Στάδιο της Εισαγωγής των Δεδομένων και της Αρχικοποίησης (Data Input and Initialization Phase): Η διαδικασία αρχικοποιείται στην κεντρική κλάση “Main”, η οποία επιτρέπει στον χρήστη να εισάγει όποιο διαθέσιμο αρχείο προβλήματος θέλει (pfile01.hddl – pfile20.hddl). Έπειτα, το υποσύστημα “ProblemParser ” λαμβάνει τη διαδρομή του επιλεγμένου αρχείου και το αναλύει σειριακά. Η λειτουργία του συγκεκριμένου αναλυτή αποτελεί τον θεμέλιο λίθο ολόκληρου του προγράμματος, καθώς εξάγει τους λογικούς περιορισμούς και τα δεδομένα του εκάστοτε αρχείου προβλήματος, διανέμοντάς τα σε τρεις κεντρικές δομές:

- Στη δομή “WorldState”, η οποία αποτυπώνει τρέχουσα την κατάσταση του περιβάλλοντος, στην δοσμένη στιγμή (π.χ. ποια ποτήρια είναι καθαρά / ποια shaker είναι τοποθετημένα στο τραπέζι κ.ο.κ.).
- Στη δομή-λεξικό “objectsMap”, η οποία συλλέγει και καταγράφει όλα τα διαθέσιμα αντικείμενα (οντότητες) του προβλήματος.
- Στην λίστα των επιθυμητών στόχων “goalTask”, η οποία περιέχει τις Ενέργειες προς αποσύνθεση, αρχικά με την μορφή απλού κειμένου.

Όταν ο έλεγχος επιστρέφει στη “Main”, οι παράμετροι των στόχων προωθούνται στην κλάση Factory (Εργοστάσιο Δημιουργίας). Η “Factory” χρησιμοποιεί πλέον όλες αυτές τις παραμέτρους για να δημιουργήσει δυναμικά δομημένα αντικείμενα τύπου Task, τα οποία αποτελούν τους πρωταρχικούς στόχους του προβλήματος που πρέπει να αποσυντεθούν.

Το Στάδιο του Σχεδιασμού (Planning Phase): Με την ολοκλήρωση των εργασιών προετοιμασίας, το σύστημα κατασκευάζει τους τρεις κύριους μηχανισμούς:

- Τον Σχεδιαστή “Planner”,
- Τον Επικυρωτή “Validator”
- Τον Μηχανισμό Καθαρισμού “Cleanup”

Σε περίπτωση που το πρόβλημα απαιτεί την αποσύνθεση περισσότερων του ενός στόχων, το σύστημα εισέρχεται σε έναν επαναληπτικό βρόχο για να διεκπεραιώσει τον κάθε στόχο διαδοχικά, εφόσον έχουμε ολική διάταξη (Total-Order). Ο ελεγκτής “Main” τροφοδοτεί το αντικείμενο “WorldState”, καθώς και τις Εργασίες-Στόχους προς αποσύνθεση στον “Planner”. Ο κεντρικός μηχανισμός διατηρεί δύο βασικές δομές:

- Την ατζέντα “agenda”, η οποία παρακολουθεί την ενεργή αλληλουχία των Εργασιών που εκκρεμούν και καταγράφει κάθε κίνηση που σημειώνεται ως σωστή.

- Το πλάνο “plan”, το οποίο καταγράφει την τελική ακολουθία μόνο των Πρωτογενών Ενεργειών (Primitive Actions).

Με την υλοποίηση μίας αναζήτησης Πρώτα-σε-Βάθος με οπισθοδρόμηση (Depth-First Search with Backtracking), το σύστημα καταφθάνει στην εύρεση μίας γραμμικής ακολουθίας που συντάσσεται μονάχα από Πρωτογενείς Ενέργειες, αυτές επιδρούν απευθείας σε μία αντιγραφή του εκάστοτε αντικειμένου “WorldState”, προσομοιώνοντας έτσι την εξέλιξη του περιβάλλοντος.

Ενδιάμεση Επαναφορά Περιβάλλοντος (Cleanup): Μία κρίσιμη πρόκληση κατά την επίλυση πολλαπλών διαδοχικών στόχων στο domain “Barman” είναι η αναστάτωση του περιβάλλοντος. Για την αντιμετώπιση αυτού του φαινομένου, παρεμβάλλεται η διαδικασία του Cleanup ανάμεσα σε κάθε ξεχωριστή αποσύνθεση μίας Ενέργειας υψηλού επιπέδου. Το σύστημα εντοπίζει και απομονώνει τα “goalShots” (τα ποτήρια που περιέχουν τα κοκτέιλ, τα οποία αποτελούν το τελικό στάδιο των προηγούμενων Task) και επαναφέρει αυτόματα το υπόλοιπο περιβάλλον στην αρχική του κατάσταση, προετοιμάζοντας το “WorldState” για την επόμενη αποσύνθεση Ενέργειας.

Τελική Επαλήθευση και Εξαγωγή Αποτελέσματος (Validate): Αφού ο “Planner” επιβεβαιώσει μια έγκυρη αλληλουχία, το πλάνο δεν οριστικοποιείται άμεσα. Αφού λάβει η “Main” τα αποτελέσματα τα μεταφέρει στο υποσύστημα “Validate”, το οποίο εκτελεί το πλάνο δοκιμαστικά ξεκινώντας από ένα καθαρό αντίγραφο της αρχικής κατάστασης. Με την επιτυχή επαλήθευση, τα οριστικοποιημένο πλάνο αποθηκεύεται και εκτυπώνεται χρονολογικά στον χρήστη.

4.2 Δομικός Σχεδιασμός και Αναπαράσταση Γνώσης

Έχοντας ήδη αναλύσει τη γενικότερη αρχιτεκτονική προσέγγιση και τη ροή των δεδομένων του συστήματος, κρίνεται πλέον αναγκαία η παρουσίαση του τρόπου με τον οποίο η θεωρητική προσέγγιση αποτυπώνεται σε λειτουργικό κώδικα. Η παρούσα ενότητα επικεντρώνεται στη δομή και στον σχεδιασμό του προγράμματος, δίνοντας έμφαση στις τεχνικές επιλογές και στις αντικειμενοστρεφείς δομές που υιοθετήθηκαν για την προγραμματιστική μοντελοποίηση του domain “Barman”.

Η μετάβαση από την αφηρημένη αναπαράσταση του Ιεραρχικού Σχεδιασμού (HTN Planning) σε εκτελέσιμη υλοποίηση στην Java οργανώθηκε γύρω από τρεις βασικούς άξονες. Αρχικά, εξετάζεται η διαδικασία ανάγνωσης και μετατροπής των δεδομένων ενός αρχείου HDDL σε διαχειρίσιμες δομές μνήμης. Στη συνέχεια, αναλύεται η προγραμματιστική αναπαράσταση των βασικών οντοτήτων του προβλήματος, καθώς και οι εξειδικευμένες δομές δεδομένων που χρησιμοποιήθηκαν για τη διασφάλιση της μοναδικότητας και την αποδοτική διαχείριση της κατάστασης του κόσμου. Τέλος, παρουσιάζονται οι κύριες κλάσεις του συστήματος, οι οποίες αποτελούν τα βασικά δομικά στοιχεία πάνω στα οποία στηρίζεται ο Σχεδιαστής (Planner) για την πλοήγηση στο δέντρο αναζήτησης.

4.2.1 Δυναμική Ανάλυση Αρχείων Εισόδου (Parsing)

Το πρόγραμμα δέχεται και αναλύει τυποποιημένα αρχεία προβλημάτων εισόδου (Problem Files) γραμμένα σε γλώσσα HDDL, τα οποία μεταφέρουν τις αρχικές δηλώσεις και τους κανόνες του εκάστοτε σεναρίου-προβλήματος. Για να γεφυρωθεί το χάσμα μεταξύ της κειμενικής μορφής της πληροφορίας σε αντικείμενο της Java δημιουργήθηκε η κλάση “ProblemParser”. Ο ρόλος της περιορίζεται στη συντακτική και σημασιολογική ανάλυση των δεδομένων εισόδου, αναλαμβάνοντας αποκλειστικά το στάδιο του tokenization και της εξαγωγής συμβολοσειρών (Strings).

Κατά τη φάση της ανάγνωσης (“parseFile”) το αρχείο φορτώνει στη μνήμη το περιεχόμενο του γραμμή προς γραμμή για ανάλυση. Ο αναλυτής “ProblemParser” λειτουργεί ουσιαστικά ως μια μηχανή καταστάσεων, ελέγχοντας τα πρώτα γράμματα κάθε γραμμής προκειμένου να εντοπίσει τις χαρακτηριστικές ετικέτες της HDDL, όπως για παράδειγμα “(:objects”, “(:htn” και “(:init”. Ο εντοπισμός αυτών των λέξεων (tokens) λειτουργεί ως διακόπτης, καθοδηγώντας έτσι το πρόγραμμα να καλέσει την κατάλληλη Μέθοδο Αποσύνθεσης για την εκάστοτε ενότητα (“currentSection”), οι ορισμένες ενότητες αναλύονται ως εξής:

1. Ανάλυση Δηλώσεων Οντοτήτων (parseObjectLine): Όταν ο αναλυτής διατρέχει το περιεχόμενο της ενότητας “(:objects”, εντοπίζει γραμμές της μορφής “name1 name2 – type”. Η ομώνυμη μέθοδος υλοποιεί έναν μηχανισμό διαχωρισμού συμβολοσειρών προκειμένου να απομονώσει τα ονόματα των οντοτήτων από το είδος στο οποίο ανήκουν, χρησιμοποιώντας την παύλα (-) ως χαρακτήρα διαχωριστικό ανάμεσα στα αντικείμενα και τον τύπο. Τα απομονωμένα πλέον “Strings” ονομάτων χωρίζονται περαιτέρω για να ξεχωρίσουμε τα μεμονωμένα αντικείμενα. Έπειτα με την σειρά τους προωθούνται στον κώδικα και μέσω της (“createObject”) πυροδοτείται η δέσμευση μνήμης και η δημιουργία της αντίστοιχης υποκλάσης δημιουργώντας έτσι το αντικείμενο.
2. Ανάλυση Αρχικών Γεγονότων (parseInitLine): Στο “currentSection (:init”, τα αρχικά δεδομένα έχουν την μορφή “(predicate param1 param2)”. Η επεξεργασία εδώ χαρακτηρίζεται από τον λεξιλογικό καθαρισμό της γραμμής, αρχικά αφαιρούνται οι ακριανές παρενθέσεις και το απομείναν κείμενο κόβεται σε “Tokens” με βάση τον κενό χαρακτήρα (\\s+). Η πρώτη συμβολοσειρά που προκύπτει λειτουργεί ως κατηγορημα “Predicate”, ενώ οι υπόλοιπες αντιμετωπίζονται ως παράμετροι. Χρησιμοποιώντας αυστηρούς ελέγχους (Type-Casting), το σύστημα τακτοποιεί αν τα “Strings” των ορισμάτων ανήκουν πράγματι σε οντότητες που έχουν ήδη καταγραφεί στην μνήμη, προετοιμάζοντας έτσι τα δεδομένα που θα συντάξουν την αρχική κατάσταση του κόσμου.

3. Ανάλυση Εργασιών προς Αποσύνθεση (parseHTNLine): Στο τμήμα ελέγχου “(:htn”, η πληροφορία είναι ελαφρώς πιο περίπλοκη. Το σύστημα αγνοεί σκόπιμα της ετικέτες δομής (όπως το :parameters), εστιάζοντας μονάχα στην ορθή εξαγωγή των Ενεργειών-Στόχων. Οι γραμμές αυτές απαλλάσσονται από τους ειδικούς χαρακτήρες (χρήση της `replaceAll(" [()]", "")`) και αποθηκεύονται προσωρινά σε μία λίστα συμβολοσειρών. Εφόσον είναι ήδη αποθηκευμένες, σε δεύτερο χρόνο θα μετατραπούν σε δομημένα αντικείμενα Εργασίας υψηλού επιπέδου (Compound Task).

4.2.2 Μοντελοποίηση Καταστάσεων και Οντοτήτων

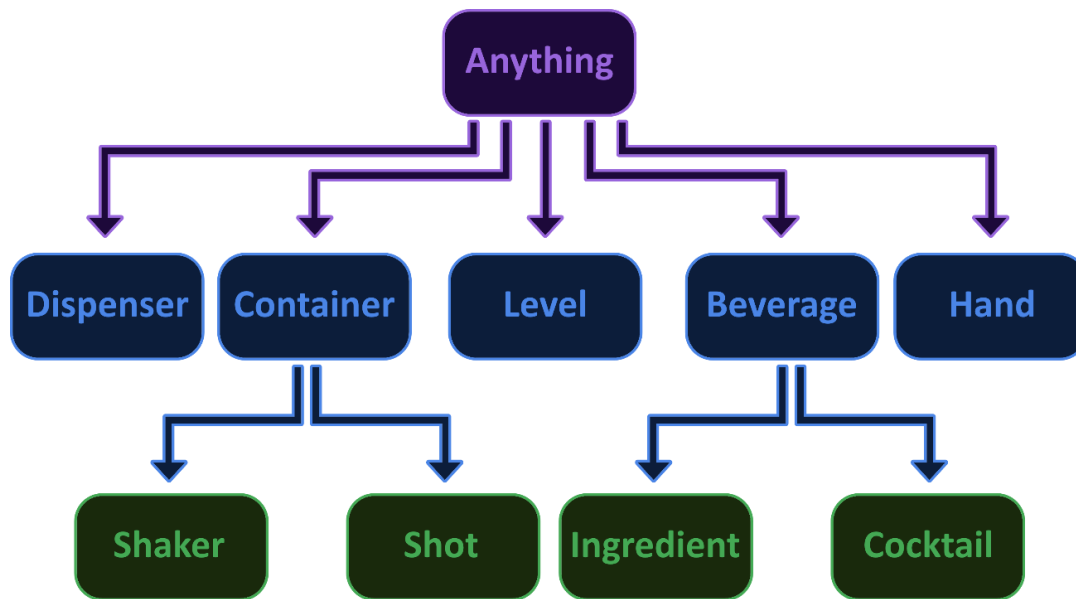
Για την επιτυχή ανάλυση και επίλυση προβλημάτων Σχεδιασμού (Planning) καθίσταται απολύτως απαραίτητη η συστηματική και αποδοτική αναπαράσταση τόσο των αντικειμένων που ίστανται στον εκάστοτε κόσμο, όσο και των συνθηκών που διαμορφώνουν την τρέχουσα κατάσταση του κόσμου. Στο εν λόγω σύστημα, η αρχιτεκτονική διαχωρίζει ξεκάθαρα την έννοια των οντοτήτων από την έννοια του περιβάλλοντος, προκειμένου να διευκολυνθεί η διαχείριση της πληροφορίας.

1. Ιεραρχία και Αναπαράσταση Οντοτήτων (Κλάση Types): Όλα τα αντικείμενα του Domain “Barman” μοντελοποιούνται μέσω ενός αυστηρού ιεραρχικού συστήματος κλάσεων, το οποίο υλοποιείται εντός της κλάσης “Types”. Εντός της κλάσης ορίζεται η αφηρημένη υποκλάση-τύπος “Anything”, η οποία προσδίδει σε κάθε οντότητα το πλέον θεμελιώδες χαρακτηριστικό της, το όνομα, όπως αυτό εξήχθη κατά την φάση του “Parsing”. Από αυτό το σημείο πηγάζουν νέες αφηρημένες κατηγορίες-πυλώνες “Abstract Classes”, οι οποίες επεκτείνουν την κλάση “Anything”, όπως είναι η “Container” και η “Beverage”. Αυτές οι αφηρημένες κλάσεις αποτελούν κλάσεις-γονείς για τις τελικές υποκλάσεις-τύπους “Shaker” και “Shot” της “Container” και “Ingredient”, “Cocktail” της “Beverage” αντίστοιχα. Βέβαια, υπάρχουν και τελικές υποκλάσεις απευθείας υπό της κλάσης “Anything”, αυτές είναι η “Hand”, “Dispenser” και “Level”. Η κλάση “Hand” μπορεί να έχει μόνο δύο αντικείμενα (left-right), η “Level” μπορεί να πάρει μόνο τις τιμές “level1”, “level2” και “level3”, αλλά ιδιαίτερο ενδιαφέρον παρουσιάζει η κλάση “Dispenser”, ή οποία πέραν του ονόματος της, διατηρεί εσωτερικά και μία άμεση αναφορά προς το συγκεκριμένο συστατικό τύπου “Ingredient” που παράγει.
2. Διαχείριση της Κατάστασης του Κόσμου (Κλάση WorldState): Η δυναμική φύση του εν λόγω προβλήματος, δηλαδή το πως αλληλεπιδρούν μεταξύ τους οι παραπάνω οντότητες σε πραγματικό χρόνο αποτυπώνεται

μέσα από την κλάση “WorldState”. Η κλάση αυτή λειτουργεί προγραμματιστικά ως στιγμιότυπο του κόσμου σε μία δεδομένη χρονική συγκυρία, συγκεντρώνοντας το σύνολο των γεγονότων που αληθεύουν. Για την αποδοτική διαχείριση της μνήμης και την ταχύτερη δυνατή προσπέλαση των δεδομένων ελέγχου, η “WorldState” κάνει εκτενή και στοχευμένη χρήση δομών δεδομένων της Java, κατηγοριοποιώντας τα γεγονότα σε δύο άξονες:

- Σύνολα (Sets): Γίνεται χρήση της δομής “HashSet” για να καταγραφούν μονοδιάστατες ιδιότητες. Παραδείγματα τέτοιων συνόλων συνιστούν τα καθαρά σκεύη (clean), τα άδεια δοχεία (empty) και τα διαθέσιμα χέρια (handEmpty). Η επιλογή του τύπου “Set” είναι στρατηγική, ώστε να εξασφαλίζεται η αποφυγή διπλότυπων εγγραφών και η ταχύτατη δυνατή πολυπλοκότητα αναζήτησης $O(1)$.
- Χάρτες (Maps): Για τις πιο σύνθετες σχέσεις που επικαλούνται εξαρτήσεις μεταξύ δύο διαφορετικών οντοτήτων, επιστρατεύονται οι δομές “HashMap”. Ενδεικτικά το “Predicate” “holding” αντιστοιχίζει ένα αντικείμενο τύπου χεριού “Hand” κατευθείαν με το δοχείο “Container”, το οποίο αυτό κρατάει. Αντίστοιχα, προσδιορίζονται οι δείκτες γεμίσματος, για παράδειγμα το “shakerLevel” συνδέει άμεσα ένα αντικείμενο δοχείου “Shaker” με το κατάλληλο επίπεδο “Level”.

Τέλος, απόλυτα καθοριστικός είναι ο ρόλος της μεθόδου “copyState()” εντός της κλάσης “WorldState”. Επειδή οι αλγόριθμοι αναζήτησης της Τεχνητής Νοημοσύνης τείνουν να δοκιμάζουν πολλαπλά διαφορετικά μονοπάτια διερεύνησης, το σύστημα πρέπει να είναι ευέλικτο και σε θέση να ανακαλεί σταθερές προηγούμενες καταστάσεις στην περίπτωση όπου κάποιο βήμα αποτύχει διαδρομικά. Έτσι, μέσω της εν λόγω μεθόδου, δημιουργείται και επιστρέφεται ένα απόλυτα ανεξάρτητο, σε βάθος αντίγραφο “Deep Copy” όλων των δομών (Set/Maps) της εκάστοτε τρέχουσας κατάστασης, προστατεύοντας τη δραστηριότητα της μνήμης από παράπλευρες και ανεπιθύμητες αλλοιώσεις (Side-Effects) άλλων μη έγκυρων ενεργειών.



Σχήμα 4.2: Ιεραρχία Τύπων

4.2.3 Σημαντικότερες Κλάσεις του Προγράμματος

Η καρδιά του συστήματος ιεραρχικού σχεδιασμού (HTN) βασίζεται σε μια σειρά από αλληλένδετες κλάσεις, οι οποίες υλοποιούν πρακτικά τη θεωρητική λογική της συνεχούς αποσύνθεσης Εργασιών και της αναζήτησης πλάνου. Οι κλάσεις αυτές σχεδιάστηκαν με γνώμονα τον αυστηρό διαχωρισμό αρμοδιοτήτων, μεταφράζοντας τις αφηρημένες έννοιες ενός ιεραρχικού προβλήματος σε λειτουργικό κώδικα Java μέσω των εξής δομικών στοιχείων:

1. Σύνθετες Εργασίες (Αφηρημένη Κλάση Task): Αποτελεί την κεντρική, αφηρημένη (abstract) οντότητα του προγράμματος, αντιπροσωπεύοντας μια οποιαδήποτε Εργασία-Στόχο που καλείται να αποσυνθέσει το σύστημα. Κάθε αντικείμενο τύπου “Task” φέρει ως βασικά χαρακτηριστικά ένα όνομα ορισμού και μία λίστα παραμέτρων (τα αντικείμενα στα οποία εφαρμόζεται). Ένα ιδιαίτερο αρχιτεκτονικό γνώρισμα της εν λόγω κλάσης είναι η ενσωμάτωση της τεχνικής “lazy generation” για τον υπολογισμό των Μεθόδων της. Συγκεκριμένα, το σύστημα δεν παράγει όλους τους πιθανούς συνδυασμούς Μεθόδων εξαρχής, καθώς αυτό θα απαιτούσε τεράστια υπολογιστική δύναμη, αλλά τους υπολογίζει δυναμικά και κατ’ απαίτηση, μέσω της μεθόδου “generateMethods()”, μόλις ζητηθούν ενεργά από τον αλγόριθμο αναζήτησης.
2. Πρωτογενείς Ενέργειες (Αφηρημένη Κλάση Action): Η “Action” κληρονομεί άμεσα από την “Task” και οι κλάσεις αυτού του τύπου αντικατοπτρίζουν τις Πρωτογενείς Ενέργειες (π.χ. άδειασμα δοχείου), οι οποίες δεν υποστηρίζουν καμία επιπλέον διάσπαση. Σε αντίθεση με τις

Σύνθετες Εργασίες, μία “Action” υλοποιεί τη διεπαφή (Interface) “hasPreconditions” ενσωματώνοντας στη δομή της:

- Τις προϋποθέσεις (Preconditions): Οι συνθήκες που πρέπει να αληθεύουν στην εκάστοτε κατάσταση “WorldState” προκειμένου η δράση να είναι εκτελέσιμη.

Πέρα από την ιδιότητα των Πρωτογενών Ενεργειών ως αδιάσπαστες, ένα ακόμα χαρακτηριστικό, το οποίο τις καθιστά μοναδικές είναι το ότι έχουν επιδράσεις “Effects”. Οι επιδράσεις υπαγορεύουν τις προσθετικές ή αφαιρετικές τροποποιήσεις που θα επιφέρει η εκτέλεση της δράσης στον κόσμο.

3. Μέθοδοι Αποσύνθεσης (Αφηρημένη Κλάση Method): Η αφηρημένη κλάση “Method” ενσαρκώνει την ιεραρχική λογική του συστήματος λειτουργώντας ως θεμελιώδης μηχανισμός που επιτρέπει στο πρόγραμμα να οργανώσει σύνθετα προβλήματα σε διαχειρίσιμα μέρη. Πρακτικά, κάθε αντικείμενο αυτής της κλάσης αναφέρει τον τρόπο με τον οποίο μία αφηρημένη Εργασία μπορεί να διασπαστεί σε μία συγκεκριμένη ακολουθία μικρότερων υπό-εργασιών (Subtasks). Ωστόσο, η “Method” δεν περιορίζεται απλώς στην εννοιολογική αποσύνθεση “Decomposition”. Ενσωματώνοντας της διεπαφή (Interface) “hasPreconditions”, επιβάλλει αυστηρούς κανόνες ελέγχου. Προτού ο παραπάνω μηχανισμός του “Planner” εγκρίνει τη διάσπαση μέσω της εκάστοτε Μεθόδου, οφείλει να πιστοποιήσει εάν οι συνδεδεμένες προϋποθέσεις “Preconditions” της ικανοποιούνται από την τρέχουσα κατάσταση του κόσμου “WorldState”. Με αυτόν τον τρόπο, οι Μέθοδοι κατευθύνουν τον αλγόριθμο αυτόματα προς τα έγκυρα μονοπάτια επίλυσης, απορρίπτοντας νωρίς τις διασπάσεις που ενδέχεται να είναι μη εκτελέσιμες στο παρόν στάδιο.

4. Σχεδιαστής (Κλάση Planner): Η κλάση “Planner” αποτελεί το κεντρικό υποσύστημα σχεδιασμού με την ορθή χρήση όλων των προαναφερθέντων οντοτήτων και κλάσεων. Υλοποιώντας έναν αλγόριθμο αναζήτησης Πρώτα-σε-Βάθος (Depth-First Search - DFS), μέσω της αυτοαναφορικής επανάληψης, ο “Planner” διαχειρίζεται μία λίστα εκκρεμών εργασιών “agenda” με σκοπό να παραδώσει ένα ολοκληρωμένο πλάνο “plan”. Εάν κάποιο μονοπάτι αποδειχθεί αδιέξοδο, αναλαμβάνει δράση ο ενσωματωμένος μηχανισμός οπισθοδρόμησης “backtracking”, αναιρώντας τις προσωρινές καταχωρήσεις και δοκιμάζοντας εναλλακτικές. Σημαντικό στοιχείο καλής μηχανικής αποτελεί και η ενσωμάτωση ορίων ασφαλείας, όπως εκφράζονται μέσα από τις σταθερές “MaxNodes” (Περιορισμός 20.000 κόμβων) και “MaxDepth” (Μέγιστο βάθος 60). Οι έλεγχοι αυτοί θωρακίζουν το σύστημα ενάντια σε εγκλωβισμούς λόγω υπερβάλλουσας ή άπειρης αναδρομικής

αναζήτησης, εξασφαλίζοντας τον τερματισμό του προγράμματος ακόμα και αν δεν βρεθεί η βέλτιστη λύση.

4.3 Ο Αλγόριθμος του Planner

Αναμφίβολα, το κυριότερο κομμάτι της παρούσας εργασίας και την κεντρική ιδέα του συστήματος, αποτελεί ο σχεδιαστής “Planner” ενσωματώνοντας την λογική του Ιεραρχικού Σχεδιασμού Δικτύων Εργασιών (HTN Planning). Ο αλγόριθμος αναλαμβάνει τον κρίσιμο ρόλο στο να γεφυρώσει το χάσμα μεταξύ της αρχικής κατάστασης και της επιτυχημένης εύρεσης πλάνου. Η επίλυση αυτής της διαδρομής δεν είναι γραμμική, αλλά υφίσταται ως μια δυναμική και εξονυχιστική εξερεύνηση πιθανών μονοπατιών στο δέντρο αναζήτησης του προβλήματος, με απώτερο σκοπό την παραγωγή ενός έγκυρου πλάνου εκτέλεσης. Στην συνέχεια θα αναλυθεί μεθοδικά ο τρόπος λειτουργίας του Σχεδιαστή, ξεκινώντας από την αποσύνθεση των Σύνθετων Εργασιών, περνώντας στον μηχανισμό οπισθοδρόμησης και καταλήγοντας στον τρόπο με το οποίο συντίθεται και εξάγεται το τελικό αποτέλεσμα.

4.3.1 Διαδικασία Αποσύνθεσης Εργασιών

Η θεμελιώδης αρχή της προσέγγισης του Ιεραρχικού Σχεδιασμού επιτάσσει τον συνεχή διαχωρισμό των Σύνθετων Εργασιών (Compound Tasks), σε συνεχώς μικρότερες και απλούστερες ενέργειες, έως ότου καταλήξουμε σε εκτελέσιμες Πρωτογενείς Ενέργειες (Primitive Actions).

Προτού όμως αναλυθεί η διαδικασία αποσύνθεσης, κρίνεται απαραίτητη η διευκρίνηση πως το σύστημα δεν ορίζει μία ρητή αυτόνομη δομή κόμβου “Node”, για το δέντρο αναζήτησης. Αντιθέτως, ο κάθε κόμβος αναπαρίσταται έμμεσα, μέσω των ορισμάτων της αναδρομικής μεθόδου `dfs()`, τα οποία είναι η τρέχουσα κατάσταση του κόσμου “state”, το προσωρινό πλάνο “plan”, το τρέχον βάθος “depth”, η λίστα των εκκρεμών εργασιών “tasks”, η οποία λειτουργεί ως μία δομή LIFO, καθώς οι νέες υπο-εργασίες τοποθετούνται πάντα στην αρχή της για να επιτευχθεί η αναζήτηση Πρώτα σε Βάθος και τέλος η ατζέντα “agenda”, η οποία λειτουργεί ως ένα ιστορικό των μεθόδων και των ενεργειών που επιλέχθηκαν για την παραγωγή του πλάνου. Η μετάβαση σε έναν θυγατρικό κόμβο γίνεται μέσω της εκάστοτε αναδρομικής κλήσης. Αξιοσημείωτο επίσης είναι το ότι η κατάσταση μεταβιβάζεται ως ανεξάρτητο αντίγραφο, μέσω της “`copyState()`”, διασφαλίζοντας έτσι την πλήρη απομόνωση κάθε κόμβου, ενώ αντιθέτως οι δομές “plan” και “agenda” διατηρούνται κοινές, και ανανεώνονται δυναμικά κατά την εμφάνιση και την οπισθοδρόμηση.

Ο “Planner” λαμβάνει την αρχική κατάσταση του κόσμου “initialState”, και τη λίστα των στόχων “goalTasks” προς επίλυση, τα οποία και μεταβιβάζονται στην κεντρική αναδρομική μέθοδο αναζήτησης “`dfs()`”. Κατά την έναρξη κάθε κύκλου

ανάλυσης, το σύστημα απομονώνει επιλεκτικά την πρώτη εργασία “Task” που εκκρεμεί προς επίλυση. Σε αυτό το στάδιο διακρίνεται το είδος της ενέργειας και το σύστημα προβαίνει στις κατάλληλες ενέργειες:

1. Σύνθετη Εργασία (Compound Task): Σε περίπτωση που η τρέχουσα αντιμετώπιση ενέργεια αποτελεί “Compound Task” το πρόγραμμα αναζητά τους διαθέσιμους τρόπους διάσπασης της μέσω Μεθόδων Αποσύνθεσης “Decomposition Methods”. Εδώ παρατηρείται άμεση διασύνδεση με εξωτερικές διαδικασίες ελέγχου, καθώς ο “Planner” δεν κρίνει ο ίδιος την εγκυρότητα μίας Μεθόδου. Αντιθέτως, απευθύνεται σε ένα εξειδικευμένο αντικείμενο “preconditionChecker” της κλάσης “Preconditions” το οποίο εξετάζει τον κανόνα. Μόνο στην περίπτωση που οι προϋποθέσεις αληθεύουν στην τρέχουσα κατάσταση, η Μέθοδος γίνεται αποδεκτή και ενσωματώνεται στην ατζέντα (agenda). Στην συνέχεια, οι νέες υπο-εργασίες (subtasks) που μεταφέρει η συγκεκριμένη Μέθοδος τοποθετούνται σειριακά μαζί με όσες εκκρεμούσαν ήδη, αντικαθιστώντας την θέση του “Task”. Έτσι δημιουργείται μια LIFO (Last-In First-Out) δομή, συνθέτοντας τη νέα λίστα στόχων που διαβιβάζεται στην επόμενη αναδρομική κλήση της μεθόδου αναζήτησης.
2. Πρωτογενής Ενέργεια (Primitive Action): Στην περίπτωση που η τρέχουσα εργασία είναι Πρωτογενής, το σύστημα επιχειρεί την άμεση προσομοίωση της. Επαναξιολογεί τις προϋποθέσεις της και εφόσον επιβεβαιωθούν επιτυχώς, χρησιμοποιεί ένα διαφορετικό εξωτερικό εργαλείο μεταβολής “effectsApplier”. Το συγκεκριμένο αντικείμενο, μέσω της μεθόδου “applyEffects” αναλαμβάνει να εφαρμόσει τις προσθετικές ή αφαιρετικές τροποποιήσεις της δράσης σε ένα ανεξάρτητο αντίγραφο της τρέχουσας κατάστασης “copyState()”. Η επιτυχής ολοκλήρωση της προσομοίωσης έχει διπλό αντίκτυπο, το όνομα της ενέργειας, συνοδευόμενο από τις παραμέτρους της, εισάγεται στην δομή της ατζέντας (agenda) και η ίδια η δομή της ενέργειας προστίθεται στο προσωρινό πλάνο (plan), προωθώντας τη ροή στην επόμενη αναδρομική κλήση.

4.3.2 Στρατηγική Οπισθοδρόμησης (Backtracking)

Κατά την εξερεύνηση των πολλαπλών πιθανών καταστάσεων (Search Space), ο εκάστοτε αλγόριθμος συναντά αρκετά αδιέξοδα. Αυτά προκύπτουν είτε διότι κάποια επιλεγμένη Μέθοδος Αποσύνθεσης οδήγησε σε μία πορεία δίχως λύση, είτε διότι οι προϋποθέσεις μίας κρίσιμης Πρωτογενούς Ενέργειας παραβιάζονται σε βαθύτερο επίπεδο της αναδρομής. Για την αντιμετώπιση αυτού του φαινομένου, ο Σχεδιαστής βασίζεται στην κλασική στρατηγική “Αναζήτηση Πρώτα-σε-Βάθος” (Depth-First

Search) η οποία ταυτοχρόνως εμπλουτίζεται με τον αναδρομικό μηχανισμό της Οπισθοδρόμησης (Backtracking).

Η λογική της συγκεκριμένης στρατηγικής έγκειται στην οργανωμένη αναίρεση των επιλογών. Οι δομές παρακολούθησης της “agenda” και του προσωρινού “plan” ενημερώνονται συνεχώς όσο ο βρόχος της “dfs” εμβαθύνει στο πρόβλημα. Όταν όμως μια Ενέργεια ή Μέθοδος δεν καταλήγει σε έγκυρο και ολοκληρωμένο αποτέλεσμα, η επόμενη αναδρομική κλήση της “dfs” επιστρέφει κενή τιμή (“null”). Έτσι σηματοδοτείται μία επιβεβαιωμένη αποτυχία εύρεσης πλάνου στο εκάστοτε κλαδί, οπότε το σύστημα επιστρέφει τον έλεγχο στο προηγούμενο επίπεδο. Σε αυτό το σημείο, το “Backtracking” αναλαμβάνει δράση, οι προηγούμενες αποφάσεις αφαιρούνται σχολαστικά. Σε περίπτωση προσθήκης μίας Πρωτογενούς Ενέργειας “Action” στο πλάνο “plan” ή και στην “agenda”, αυτό αφαιρείται άμεσα μέσω της “remove()” και από τις δύο δομές. Ομοίως, αν είχε επιλεχθεί προηγουμένως μία συγκεκριμένη Μέθοδος Αποσύνθεσης “Method”, η οποία εν τέλει απέτυχε, το ίχνος της διαγράφεται αυτομάτως από την ατζέντα “agenda”. Ελευθερωμένος από τα εσφαλμένα δεδομένα του αδιεξόδου, ο βρόχος επανάληψης του ίδιου επιπέδου προχωρά για να δοκιμάσει εναλλακτικές Μεθόδους. Η αξιοπιστία του μηχανισμού “Backtracking” βασίζεται αναμφίβολα στη χρήση της μεθόδου “copyState()”. Πριν από κάθε νέα αποσύνθεση ή εκτέλεση Πρωτογενούς Ενέργειας δημιουργείται ένα ανεξάρτητο αντίγραφο της τρέχουσας κατάστασης. Επομένως, όταν μια διαδρομή αποτύχει και χρειαστεί να αναιρεθεί, τα δεδομένα του περιβάλλοντος παραμένουν ανεπηρέαστα, επιτρέποντας την εξερεύνηση εναλλακτικών διαδρομών.

4.3.3 Σύνθεση και Εξαγωγή Τελικού Πλάνου

Η επιτυχία της προηγούμενης διαδικασίας αναγνωρίζεται άμεσα, όταν η λίστα των υπό διεκπεραίωση εργασιών αδειάσει. Αυτή η συνθήκη “tasks.isEmpty()” εξετάζεται στην έναρξη της κύριας συνάρτησης “dfs” και αποτελεί την ασφαλή ένδειξη ότι όλες οι αποσυνθέσεις έχουν τοποθετηθεί, επαληθευτεί και εκτελεστεί αρμονικά. Σε εκείνο το σημείο ανακόπτεται ο συνεχής βρόχος των επαναλήψεων και το σύστημα επιστρέφει την τελική μορφή του αλλαγμένου πλέον περιβάλλοντος “WorldState”. Ωστόσο, η σύνθεση του τελικού αποτελέσματος απαιτεί την ανάλογη συγκέντρωση και δέσμευση των δεδομένων στο ανώτατο επίπεδο του Σχεδιαστή “Planner”. Στο τέλος μιας εξ ολοκλήρου επιτυχούς εξερεύνησης, το προσωρινό “plan”, το οποίο μέχρι εκείνη τη στιγμή αυξομειωνόταν και διαμορφωνόταν δυναμικά, μετατρέπεται πλέον στο οριστικό, αμιγές πλάνο αποτελώντας μια αυστηρή, διαδοχική ακολουθία από αποκλειστικά Πρωτογενείς Ενέργειες (Primitive Actions). Αντίστοιχα, η παράλληλη δομή “agenda” εμπεριέχει πλέον την πλήρη και λεπτομερή αρχιτεκτονική της αποσύνθεσης, καταγράφοντας αλφαριθμητικά (Strings) όλες τις παρεμβαλλόμενες

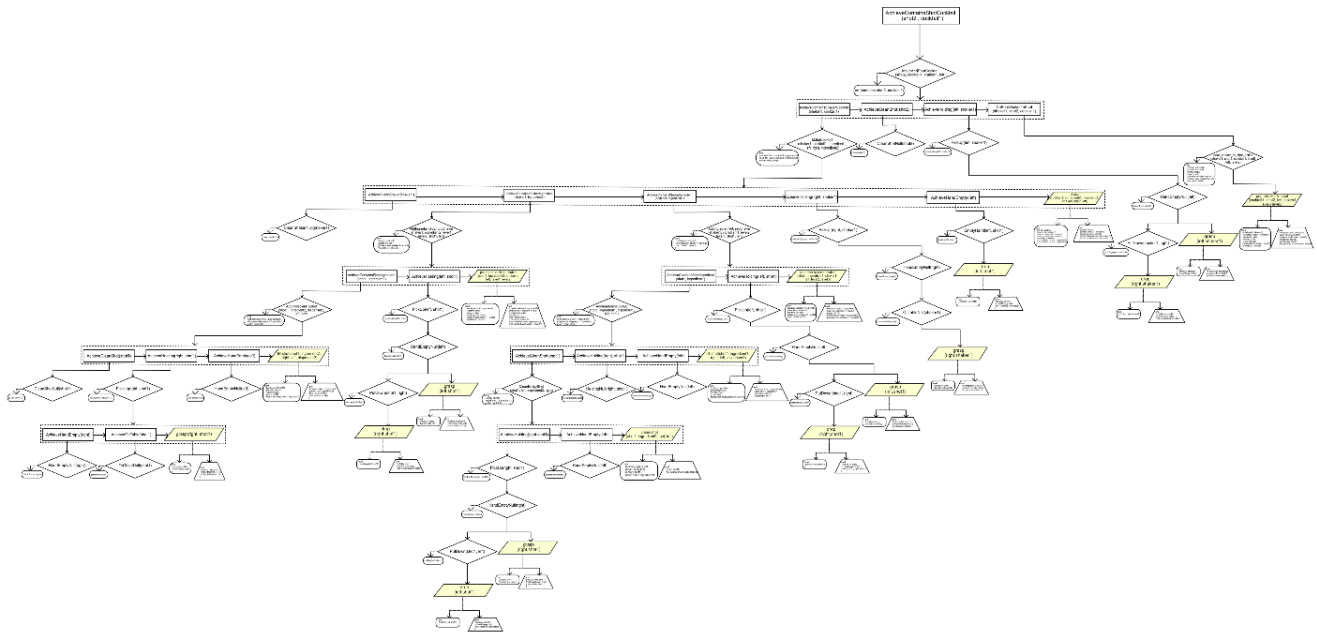
Μεθόδους, διευκολύνοντας έτσι την επιβεβαίωση της διεργασίας από τον χρήστη εάν αυτό ζητηθεί.

4.3.4 Ψευδοκώδικας και δενδρική αναπαράσταση του πρώτου προβλήματος

Ακολουθεί η περιγραφή του κώδικα της κλάσης Planner σε ψευδοκώδικα: [12]

Κλάση Planner
Ορισμός ΜέγιστοιΚόμβοι = 20000
Ορισμός ΜέγιστοΒάθος = 60
Αρχικοποίηση ελεγκτήςΠροϋποθέσεων
Αρχικοποίηση μετρητήςΚόμβων
Αρχικοποίηση τελικόΒάθος
Ορισμός Κλάσης Result
Ορισμός και Αρχικοποίηση πλάνο
Ορισμός και Αρχικοποίηση ατζέντα
Ορισμός και Αρχικοποίηση τελικήΚατάσταση
Ορισμός και Αρχικοποίηση μετρητήςΚόμβων
Ορισμός και Αρχικοποίηση βάθος
Συνάρτηση planFinder(αρχικήΚατάσταση, εργασίεςΣτόχοι)
Αρχικοποίηση πλήθοςΚόμβων
Αρχικοποίηση τελικόΒάθος
Αρχικοποίηση πλάνο
Αρχικοποίηση ατζέντα
// Έναρξη της αναδρομικής αναζήτησης σε βάθος από το βάθος 0
κατάστασηΑποτελέσματος = dfs(αρχικήΚατάσταση, εργασίεςΣτόχοι, πλάνο, ατζέντα, 0)
Αν η κατάστασηΑποτελέσματος = Επιτυχημένη
Τότε επέστρεψε (πλάνο, ατζέντα, πλήθοςΚόμβων, τελικόΒάθος)
Αλλιώς
επίστεψε Αποτυχία
Συνάρτηση dfs(κατάσταση, εργασίες, πλάνο, ατζέντα, βάθος)
Αν η πλήθοςΚόμβων + 1 > ΜέγιστοιΚόμβοι Ή βάθος > 60
Τότε επέστρεψε Αποτυχία
//Αν δεν υπάρχουν άλλες εργασίες, τότε πέτυχε η εύρεση πλάνου
Αν η λίστα με τις εργασίες είναι άδεια
Τότε τελικόΒάθος = βάθος και Επίστεψε την κατάσταση

τρέχουσαΕργασία = πρώτο στοιχείο από την λίστα εργασιών
υπόλοιπεςΕργασίες = τα υπόλοιπα στοιχεία της λίστας εργασιών
//Περίπτωση Α: Η εργασία είναι Πρωταρχική Ενέργεια
Αν η τρέχουσαΕργασία είναι Πρωταρχική Ενέργεια
Αν οι προϋποθέσεις της τρέχουσας εργασίας ικανοποιούνται στην κατάσταση
//Εφαρμογή των επιπτώσεων της εργασίας
επόμενηΚατάσταση = αντίγραφο(κατάσταση)
εφαρμογήΕπιπτώσεων(επόμενηΚατάσταση, τρέχουσαΕργασία,Επιπτώσεις)
προσθήκη της τρέχουσαΕργασία στο πλάνο και στην ατζέντα
//Αναδρομική επίλυση των υπόλοιπων εργασιών
αποτέλεσμα = dfs(επόμενηΚατάσταση, υπόλοιπεςΕργασίες, πλάνο, ατζέντα, βάθος+1)
Αν το αποτέλεσμα είναι επιτυχές
Τότε επέστρεψε αποτέλεσμα
//Οπισθοδρόμηση και αναίρεση αλλαγών αν η αναζήτηση απέτυχε
Αφαίρεση της τελευταίας εργασίας από το πλάνο και την ατζέντα
Επίστρεψε Αποτυχία
//Περίπτωση Β: Η εργασία είναι Σύνθετη Ενέργεια
Αν η τρέχουσαΕργασία είναι Σύνθετη Ενέργεια
Για κάθε μέθοδο που μπορεί να αποσυνθέσει την τρέχουσαΕργασία
Αν οι προϋποθέσεις της μεθόδου ικανοποιούνται στην κατάσταση
προσθήκη της μεθόδου στην ατζέντα
//Αντικατάσταση της Σύνθετης Εργασίας με τις Υποεργασίες της
νέεςΕργασίες = υποεργασίεςΜεθόδου + υπόλοιπεςΕργασίες
//Αναδρομική επίλυση της νέας λίστας Εργασιών
αποτέλεσμα = dfs(κατάσταση, νέεςΕργασίες, πλάνο, ατζέντα, βάθος+1)
Αν το αποτέλεσμα είναι επιτυχές
Τότε επέστρεψε αποτέλεσμα
//Οπισθοδρόμηση και αναίρεση μεθόδου αν η αναζήτηση απέτυχε
Αφαίρεση της μεθόδου από την ατζέντα



Σχήμα 4.3: Δενδρική αναπαράσταση της λύσης του πρώτου προβλήματος

4.4 Τεχνικές Προκλήσεις και Βελτιστοποιήσεις Κώδικα

Η μετάβαση από την θεωρητική προσέγγιση ενός ιεραρχικού συστήματος στην πρακτική υλοποίηση του συνοδεύεται αναμφίβολα από μια σειρά προκλήσεων, οι οποίες πηγάζουν τόσο από την εγγενή τεχνική φύση του προβλήματος όσο και από τις προκλήσεις που επιφέρει η υλοποίηση σε περιβάλλον Java. Η διαχείριση πολλαπλών πόρων, η σε βάθος εξερεύνηση στον χώρο καταστάσεων και η χαρτογράφηση της κάθε κίνησης της διαδικασίας αναζήτησης, καθιστούν την εύρεση πλάνου ως μία απαιτητική διαδικασία. Στο πλαίσιο αυτό, η συγκεκριμένη ενότητα αναλύει τις στρατηγικές και τις προγραμματιστικές βελτιστοποιήσεις που εφαρμόστηκαν στον κώδικα εστιάζοντας στην αποτελεσματική διαχείριση των πόρων και την επιτάχυνση των λειτουργιών ελέγχου, έτσι ώστε να διασφαλιστεί η εύρεση μίας έγκυρης λύσης.

4.4.1 Διαχείριση Μνήμης και Κλωνοποίηση Καταστάσεων (State Cloning)

Μία από τις πιο απαιτητικές διαδικασίες στην ανάπτυξη του εν λόγω συστήματος είναι η ανάγκη της συνεχούς αντιγραφής της κατάστασης του κόσμου “WorldState”, κατά την διάρκεια της αναζήτησης. Προκειμένου να διασφαλιστεί η ορθή λειτουργία του μηχανισμού οπισθοδρόμησης (backtracking), ο “Planner” οφείλει να κρατάει στην μνήμη αναλλοίωτες τις προηγούμενες καταστάσεις. Αυτό είναι κρίσιμο, καθώς αν ο αλγόριθμος βρεθεί σε μονοπάτι αδιεξόδου, πρέπει να είναι σε θέση να επανέλθει σε κάποια προηγούμενη, έγκυρη κατάσταση χωρίς αυτή να έχει υποστεί αλλοίωση από τις δοκιμαστικές ενέργειες που μεσολάβησαν. Για την επίτευξη της

διαδικασίας αυτής, εφαρμόστηκε η τεχνική “Deep Copying” στις μεταβαλλόμενες δομές δεδομένων που χρησιμοποιήθηκαν. Η στρατηγική αυτής της επιλεκτικής αντιγραφής μειώνει το υπολογιστικό κόστος, αποφεύγοντας την αντιγραφή στατικών δεδομένων που παραμένουν ίδια καθ’ όλη την διάρκεια του προβλήματος (π.χ. ονόματα αντικειμένων). Έτσι, αποφεύχθηκε η άσκοπη δέσμευση μνήμης για δεδομένα του προβλήματος μειώνοντας το υπολογιστικό κόστος.

4.4.2 Αντιμετώπιση Ατέρμονων Βρόχων (Infinite Recursion)

Η αναδρομική φύση σε έναν αλγόριθμο αναζήτησης Depth-First Search (DFS) εγκυμονεί τον κίνδυνο εγκλωβισμού σε ατέρμονους βρόχους, ένα φαινόμενο γνωστό και ως “Infinite Recursion”. Στο domain του προβλήματος, οι ενέργειες είναι συχνά κυκλικές, για παράδειγμα μπορεί ο “Barman” να πιάσει ένα σκεύος και αμέσως να το αφήσει χωρίς να κάνει κάποια ουσιαστική πρόοδο όντας παγιδευμένος στην συγκεκριμένη αλληλουχία ενεργειών. Επειδή η Java διαθέτει περιορισμένο μέγεθος στοίβας κλήσεων, μια τέτοια ανεξέλεγκτη αναδρομή μπορεί να οδηγήσει σε σφάλμα τύπου “StackOverflowError” διακόπτοντας την λειτουργία του προγράμματος. Για την αποφυγή αυτών, ενσωματώνονται στον Σχεδιαστή (Planner) όρια για τον μηχανισμό ελέγχου, ο περιορισμός σε βάθος και ο περιορισμός κόμβων. Και τα δύο αυτά όρια επιλέχθηκαν με γνώμονα την πολυπλοκότητα των προβλημάτων του πεδίου σε συνάρτηση με τις δυνατότητες του υπολογιστή.

- Ο περιορισμός σε βάθος (MaxDepth): Ορίζεται ένα ανώτατο όριο των εξήντα επιπέδων αναδρομής. Σε περίπτωση που η αποσύνθεση ενός “Task” προσπαθήσει να ξεπεράσει το όριο αυτό, ο αλγόριθμος θεωρεί το συγκεκριμένο μονοπάτι μη αποτελεσματικό και οπισθοχωρεί.
- Ο περιορισμός κόμβων (MaxNodes): Επιβάλλεται ένα φράγμα των είκοσι χιλιάδων επισκέψεων σε κόμβους του δέντρου αναζήτησης. Ο έλεγχος αυτός λειτουργεί σαν δικλείδα ασφαλείας σε περίπτωση που ο “Planner” κάνει μία υπερβολικά ευρεία εξερεύνηση εναλλακτικών Μεθόδων χωρίς αποτέλεσμα.

Οι παραπάνω περιορισμοί λειτουργούν συμπληρωματικά, διασφαλίζοντας την σταθερότητα του συστήματος. Δίνουν την δυνατότητα στον “Planner” να εγκαταλείπει μη παραγωγικές διαδρομές δίνοντας του ώθηση να στραφεί σε εναλλακτικές Μεθόδους Αποσύνθεσης, που είναι πιθανότερο να οδηγήσουν σε επιτυχημένη εύρεση πλάνου.

4.4.3 Επιλογή Αποδοτικών Δομών Δεδομένων στη Java

Η υπολογιστική απόδοση ενός “Planner” εξαρτάται άμεσα από την ταχύτητα πρόσβασης και τροποποίησης της πληροφορίας. Στην παρούσα υλοποίηση η επιλογή των δομών δεδομένων έγινε με γνώμονα την βελτιστοποίηση της χρονικής

πολυπλοκότητας των βασικών λειτουργιών αναζήτησης και την αποδοτική διαχείριση της μνήμης κατά την αναδρομική εξερεύνηση του χώρου καταστάσεων.

1. HashSet για τα μοναδιαία κατηγορήματα: Για την αναπαράσταση ιδιοτήτων που χαρακτηρίζουν μεμονωμένα αντικείμενα, όπως είναι το Predicate “clean shot1”, επιλέχθηκε η δομή “HashSet”. Με αυτόν τον τρόπο τα σύνολα εξασφαλίζουν ότι κάθε “fact” καταγράφεται μόνο μία φορά, αποφεύγοντας τυχόν πλεονασμού. Επιπλέον η εν λόγω δομή προσφέρει χρονική πολυπλοκότητα $O(1)$ για τις βασικές λειτουργίες προσθήκης/διαγραφής ή I/O και ελέγχου ύπαρξης. Αυτό επιτρέπει στο σύστημα να επιβεβαιώνει ακαριαία την κατάσταση ενός αντικειμένου χωρίς να απαιτεί γραμμική σάρωση λίστας.
2. HashMap για συσχετίσεις και σχέσεις εξαρτήσεων: Για την αναπαράσταση σχέσεων μεταξύ δύο οντοτήτων, όπως είναι το Predicate “shakerLevel shaker1 level2”, χρησιμοποιήθηκε η δομή “HashMap”. Οι χάρτες επιτρέπουν την χρήση ενός αντικειμένου ως μοναδικό κλειδί “Key” για την ανάκτηση μιας συσχετιζόμενης τιμής “Value”. Η αναζήτηση με βάση το κλειδί γίνεται σε καλύτερο και μέσο χρόνο $O(1)$, ενώ στην χειρότερη περίπτωση σε $O(\log_n)$. Η επιλογή αυτή είναι κρίσιμη για τη διαχείριση των μεταβλητών καταστάσεων, καθώς επιτρέπει άμεση πρόσβαση στη δυναμική πληροφορία του κόσμου.
3. Ειδική Περίπτωση εμφωλευμένης δομής HashMap<Type, Set<Type>>: Ιδιαίτερη αναφορά οφείλει να γίνει στη δομή του τύπου “Map<Container, Set<Beverage>> contains”, η οποία αναπαριστά ένα δοχείο και το περιεχόμενο του. Πρόκειται για μία εμφωλευμένη δομή δεδομένων “Nested Data Structure” όπου ένας χάρτης αντιστοιχίζει κάθε δοχείο “Container” σε ένα σύνολο από υλικά, τα οποία υπάγονται στην κατηγορία των ποτών “Set<Beverage>”. Η χρήση “Set” έναντι “List” επιλέχθηκε για την αυτόματη αποφυγή διπλότυπων εγγραφών, παρά τη σύνθετη φύση της η δομή διατηρεί την πολυπλοκότητα $O(1)$, καθώς ο μηχανισμός κατακερματισμού επιτρέπει τον εντοπισμό των στοιχείων σε σταθερό χρόνο.
4. ArrayList για τη διαχείριση πλάνου και οπισθοδρόμησης: Για τη διατήρηση της ορθής σειράς εκτέλεσης των ενεργειών και των υποεργασιών χρησιμοποιήθηκε η δομή “ArrayList”. Κατά τη διάρκεια της διαδικασίας οπισθοδρόμησης στον αλγόριθμο “DFS”, η αφαίρεση του τελευταίου στοιχείου από μία “ArrayList” εκτελείται σε χρόνο $O(1)$, γεγονός που καθιστά την αναδρομική αναζήτηση ιδιαίτερα γρήγορη.
5. Διαχείριση μνήμης και Deep Copying: Λόγω της αναδρομικής φύσης του “Planner”, εφαρμόζεται η στρατηγική της βαθιάς αντιγραφής στην κλάση “WorldState”. Κάθε φορά που ο αλγόριθμος του “Planner” εξερευνά ένα νέο μονοπάτι, δημιουργείται ένα νέο στιγμιότυπο όλων των συλλογών. Στην περίπτωση της ειδικής περίπτωσης της δομής “contains”, η αντιγραφή περιλαμβάνει τη δημιουργία νέων “HashSet” για κάθε εγγραφή του χάρτη.

Δομή Δεδομένων	Σκοπός που Εξυπηρετεί	Πολυπλοκότητα
HashSet	Μοναδιαίες Ιδιότητες (Unary Predicates)	$O(1)$
HashMap	Συσχετίσεις (Binary Predicates)	$O(1)$
Map<K, Set<V>>	Ειδική Περίπτωση "contains"	$O(1)$
ArrayList	Πλάνο, Ατζέντα, Οπισθοδρόμηση	$O(1)$

Πίνακας 4.1: Πολυπλοκότητα των επιλεγμένων δομών δεδομένων

4.4.4 Βελτιστοποίηση Ελέγχου Προϋποθέσεων (Precondition Checking)

Ο έλεγχος των προϋποθέσεων αποτελεί τη συχνότερα εκτελούμενη λειτουργία στον αλγόριθμο σχεδιασμού, καθώς κάθε απόφαση για την επιλογή μίας Μεθόδου Αποσύνθεσης ή την εφαρμογή μίας Πρωτογενούς Ενέργειας εξαρτάται από την επικύρωση της έναντι της τρέχουσας κατάστασης του κόσμου. Λόγω του μεγάλου αριθμού των ελέγχων που απαιτούνται κατά την αναζήτηση Πρώτα-σε-Βάθος (DFS), η υλοποίηση του μηχανισμού αυτού βελτιστοποιήθηκε ώστε να ελαχιστοποιεί το υπολογιστικό κόστος, διασφαλίζοντας ταυτόχρονα την ορθότητα της λογικής του πεδίου “Barman”.

Η κύρια βελτιστοποίηση του ελέγχου βασίζεται στον πολυμορφισμό που προσφέρεται από την γλώσσα Java μέσω της διεπαφής (Interface) “hasPreconditions”. Η διεπαφή αυτή επιτρέπει στον “Planner” να αντιμετωπίζει με ενιαίο τρόπο τις Πρωτογενείς Ενέργειες (Actions) και τις Μεθόδους Αποσύνθεσης (Methods), καλώντας τη μέθοδο “getPreconditions()” χωρίς να απαιτείται η γνώση της εσωτερικής δομής τους. Η εφαρμογή της αφαιρετικότητας είναι καίριας σημασίας, καθώς μέσω αυτής, η κλάση “Preconditions” λειτουργεί ως μηχανισμός επικύρωσης που δέχεται οποιοδήποτε αντικείμενο φέρει προϋποθέσεις και τις αντιπαραβάλλει με την “WorldState”.

Επιπλέον η υπολογιστική ταχύτητα του ελέγχου ενισχύεται από τη χρήση ενός κεντρικού μηχανισμού δρομολόγησης, ο οποίος μέσω της δομής “switch” κατευθύνει τον έλεγχο άμεσα στην αντίστοιχη δομή της “WorldState”, αποφεύγοντας δαπανηρές σειρές διαδοχικών ελέγχων “if-else”. Επιπλέον η άμεση διαχείριση του τελεστή άρνησης “not” και η ταχεία σύγκριση αντικειμένων για τον τελεστή ισότητας “=” επιτρέπουν στον “Planner” να εντοπίζει και να απορρίπτει έγκαιρα μη παραγωγικά μονοπάτια αναζήτησης, περιορίζοντας δραστικά τον χώρο αναζήτησης και βελτιστοποιώντας τη χρήση των διαθέσιμων υπολογιστικών πόρων.

5 Πειραματική Αξιολόγηση και Αποτελέσματα

5.1 Μεθοδολογία Πειραμάτων και Περιβάλλον

Στο παρόν κεφάλαιο παρουσιάζεται η πειραματική διαδικασία που ακολουθήθηκε για την αξιολόγηση του συστήματος σχεδιασμού. Στόχος της αξιολόγησης είναι η επαλήθευση της ορθότητας του “Planner”, η μέτρηση της υπολογιστικής του απόδοσης και η ικανότητα του να παράγει έγκυρα πλάνα για το σύνολο των προβλημάτων του domain “Barman” (IPC 2023).

5.1.1 Προδιαγραφές Συστήματος Εκτέλεσης

Για την διασφάλιση της αξιοπιστίας των μετρήσεων, όλα τα πειράματα διεξήχθησαν σε ένα σταθερό υπολογιστικό περιβάλλον. Οι τεχνικές προδιαγραφές του συστήματος εκτέλεσης συνοψίζονται ως:

Χαρακτηριστικό	Περιγραφή
Επεξεργαστής	AMD Ryzen 5 2600x
Μνήμη RAM	16GB DDR4
Λογισμικό / JDK	JDK 25
IDEA	IntelliJ IDEA 2025

Πίνακας 5.1: Τεχνικά Χαρακτηριστικά Συστήματος

5.1.2 Το Σύνολο των προβλημάτων Ελέγχου

Η αξιολόγηση βασίστηκε στα επίσημα αρχεία προβλημάτων του Διεθνούς Διαγωνισμού Σχεδιασμού IPC 2023 για το Total-Order Domain “Barman”. Χρησιμοποιήθηκαν συνολικά 20 προβλήματα (pfile01.hddl έως pfile20.hddl), τα οποία χαρακτηρίζονται από κλιμακούμενη πολυπλοκότητα. Αρχικά παρουσιάζονται προβλήματα χαμηλής πολυπλοκότητας που στην πορεία των προβλημάτων αυξάνεται, για παράδειγμα δίνεται ένα “Task” προς αποσύνθεση στο πρώτο πρόβλημα και πενήντα “Tasks” προς αποσύνθεση στο εικοστό και τελευταίο πρόβλημα.

5.1.3 Κριτήρια και Μετρικές Αξιολόγησης

Η απόδοση του Planner αξιολογείται βάσει των παρακάτω μετρικών:

- Ποσοστό Επιτυχίας: Το πλήθος των προβλημάτων που επιλύθηκαν εντός των ορίων ασφαλείας που θέσπισαν.
- Χρόνος Αναζήτησης: Ο καθαρός χρόνος που απαιτήθηκε από τον αλγόριθμο αναζήτησης για την εύρεση λύσης.

- Αριθμός Κόμβων: Ο αριθμός των κόμβων που εξερευνήθηκαν, ο οποίος αντικατοπτρίζει την αποτελεσματικότητα της επιλογής Μεθόδων Αποσύνθεσης και τη συχνότητα της οπισθοδρόμησης.
- Μήκος Πλάνου: Ο συνολικός αριθμός Πρωτογενών Ενεργειών που απαιτούνται για την τελική επίτευξη των στόχων.

5.1.4 Διαδικασία Επικύρωσης Πλάνου

Πέρα από την παραγωγή της γραμμικής αλληλουχίας Πρωτογενών Ενεργειών, η μεθοδολογία περιλαμβάνει ένα υποχρεωτικό στάδιο επικύρωσης. Μετά την ολοκλήρωση της αναζήτησης, το παραγόμενο πλάνο περνάει στο υποσύστημα “Validator”, ο μηχανισμός αυτός προσομοιώνει την εκτέλεση κάθε Ενέργειας στην αρχική κατάσταση του κόσμου επαληθεύοντας έτσι πως σε κάθε βήμα ικανοποιούνται οι προϋποθέσεις και εφαρμόζονται σωστά οι επιδράσεις. Η διαδικασία αυτή εγγυάται ότι τα αποτελέσματα που παρουσιάζονται είναι τεχνικά ορθά και εφαρμόσιμα στο πεδίο.

5.1.5 Εργαλεία και Τεχνολογίες Λογισμικού

Για την ανάπτυξη και υλοποίηση του συγκεκριμένου συστήματος επιλέχθηκαν σύγχρονα και αξιόπιστα εργαλεία λογισμικού, με στόχο τη διασφάλιση της ποιότητας του κώδικα, την ταχύτητα παραγωγής και την ευκολία συντήρησης. Η επιλογή των τεχνολογιών έγινε με γνώμονα τις ιδιαίτερες απαιτήσεις του προβλήματος “Barman” και την ανάγκη για αποδοτική διαχείριση πολύπλοκων δομών δεδομένων και λογικής κατά την εκτέλεση των αλγορίθμων σχεδιασμού. Η συνολική ανάπτυξη του έργου βασίστηκε αποκλειστικά στις εξής τεχνολογίες:

1. Ολοκληρωμένη Περιβάλλον Ανάπτυξης (IDE): IntelliJ IDEA 2025

Ως αποκλειστικό περιβάλλον ανάπτυξης κώδικα χρησιμοποιήθηκε το IntelliJ IDEA της Jet Brains στην έκδοση του 2025. Πρόκειται για ένα από τα κορυφαία και ευρέως γνωστά εργαλεία για την ανάπτυξη εφαρμογών σε Java, γεγονός το οποίο συνέβαλε καθοριστικά στην επιλογή περιβάλλοντος ανάπτυξης. Η επιλογή του συγκεκριμένου IDE προσέφερε σημαντικά πλεονεκτήματα, όπως, η έξυπνη υποστήριξη κώδικα και στατική ανάλυση, για παράδειγμα, η αυτόματη συμπλήρωση και η ανίχνευση σφαλμάτων σε πραγματικό χρόνο διευκόλυναν την ορθή συγγραφή των πολύπλοκων κλάσεων. Τα προηγμένα εργαλεία αποσφαλμάτωσης “Debugging”, όπως η δυνατότητα παρακολούθησης της ροής εκτέλεσης βήμα προς βήμα και η εξέταση των μεταβλητών κατάστασης “State Variables” αποδείχθηκαν εξαιρετικής σημασίας για τον εντοπισμό και την επίλυση αυστηρών λογικών σφαλμάτων κατά την ανάπτυξη του κώδικα και κυρίως στην ανάπτυξη του σχεδιαστή

“Planner” αναζήτηση στα δέντρα του “HTN Planning”. Καθώς επίσης και η αυτοματοποιημένη αναδόμηση “Refactoring”, η οποία επέτρεψε την ευέλικτη αναδιάρθρωση μερών του συστήματος καθώς αυτό γινόταν πιο περίπλοκο, διατηρώντας τον κώδικα καθαρό, κατανοητό και επεκτάσιμο.

2. Γλώσσα Προγραμματισμού και Πλατφόρμα: Java (JDK 25)

Η συγγραφή του κώδικα του συστήματος πραγματοποιήθηκε εξ ολοκλήρου στη γλώσσα προγραμματισμού Java, κάνοντας αποκλειστική χρήση της υπερσύγχρονης έκδοσης JDK 25. Η αρχιτεκτονική της Java, που βασίζεται στο αντικειμενοστρεφές πρότυπο “Object-Oriented Programming – OOP”, κρίθηκε ιδανική για τη μοντελοποίηση των εννοιών του Domain “Barman”. Η ενσωμάτωση του JDK 25 στο έργο εξασφάλισε αξιοποίηση νέων δυνατοτήτων της γλώσσας, όπως για παράδειγμα τα σύγχρονα προγραμματιστικά χαρακτηριστικά όπως τα “Flexible Constructor Bodies”, τα οποία οδήγησαν σε πιο εκφραστικό, ασφαλή και άμεσα κατανοητό κώδικα.

Συνοψίζοντας, η συνδυαστική και αποκλειστική χρήση του IntelliJ IDEA 2025 και του JDK 25 σύνθεσε ένα στιβαρό, σύγχρονο και εξαιρετικά αποδοτικό οικοσύστημα ανάπτυξης, το οποίο κάλυψε πλήρως τον κύκλο ζωής του λογισμικού, από την αρχική μοντελοποίηση έως και την τελική εκτέλεση του “Planner”.

5.2 Παρουσίαση και Ανάλυση Αποτελεσμάτων

Στην παρούσα ενότητα αναλύονται τα δεδομένα που προέκυψαν από την πειραματική αξιολόγηση του εν λόγω συστήματος. Το κύριο εύρημα της μελέτης είναι η επίτευξη της επιτυχημένης αποσύνθεσης όλων των προβλημάτων. Το σύστημα καταφέρνει να παράγει έγκυρα πλάνα και για τα είκοσι προβλήματα του πεδίου. Το αποτέλεσμα αυτό επιβεβαιώνει την ορθότητα της ιεραρχικής μοντελοποίησης και την αποτελεσματικότητα του συνδυασμού του αλγορίθμου DFS με τις εξειδικευμένες δομές δεδομένων.

5.2.1 Ανάλυση Υπολογιστικής Απόδοσης

Η πειραματική αξιολόγηση και η μελέτη των αποτελεσμάτων προσδίδουν πως ο “Planner” διατηρεί μία αξιοσημείωτη ισορροπία μεταξύ ταχύτητας και υπολογιστικής πολυπλοκότητας. Στα πρώτα προβλήματα (pfile01.hddl - pfile05.hddl) παρατηρείται σχετικά χαμηλός αριθμός επισκεπτόμενων κόμβων και η εύρεση λύσης είναι σχεδόν ακαριαία. Καθώς αναλύουμε τα τελευταία προβλήματα (pfile15.hddl - pfile20.hddl) η πολυπλοκότητα αυξάνεται και παρουσιάζεται μια αύξηση στον αριθμό των επισκεπτόμενων κόμβων. Το γεγονός ότι το σύστημα δεν άγγιξε το όριο των 20000

κόμβων σε κανένα σενάριο αποδεικνύει ότι η μοντελοποίηση των Μεθόδων είναι εξαιρετικά στοχευμένη, περιορίζοντας το άσκοπο backtracking.

5.2.2 Κλιμάκωση και Διαχείριση Μνήμης

Παρά την πολυπλοκότητα που αναμένεται να παρουσιάζεται σε ένα πρόβλημα Ιεραρχικού Σχεδιασμού, η χρήση της Java 25, των “flexible constructors” και των βελτιστοποιημένων δομών δεδομένων επέτρεψε τη σταθερή διαχείριση μνήμης. Η στρατηγική του “Deep Copying” στην “WorldState” δεν επιβάρυνε σημαντικά το χρόνο εκτέλεσης, ενώ διασφάλισε την ακεραιότητα των δεδομένων κατά την αναδρομή. Ακόμη και στο πρόβλημα με την μεγαλύτερη πολυπλοκότητα, όπου η αλυσίδα των Μεθόδων Αποσύνθεσης είναι αρκετά εκτενής, ο “Planner” δεν παρουσίασε σφάλματα υπερχείλισης στοίβας “StackOverflow”, επιβεβαιώνοντας τη στιβαρότητα της υλοποίησης.

5.2.3 Ποιότητα Πλάνων και Μήκος Λύσεων

Τα παραγόμενα πλάνα χαρακτηρίζονται ως “Satisficing”, καθώς παρέχουν έγκυρες λύσεις σε πολύ σύντομο χρόνο. Το μήκος των πλάνων ακολουθεί την πολυπλοκότητα των εισαγόμενων στόχων. Παρατηρήθηκε ότι ο μηχανισμός “Cleanup” παίζει καθοριστικό ρόλο στο τελικό μήκος, καθώς προσθέτει αυτόματα τις απαραίτητες ενέργειες επαναφοράς ώστε να επανέλθει στο αρχικό επίπεδο η κατάσταση του κόσμου. Η επιτυχής επικύρωση όλων των πλάνων από τον “Validator” επιβεβαιώνει ότι η ιεραρχική προσέγγιση παράγει λύσεις που είναι απόλυτα εφαρμόσιμες στο πεδίο “Barman”.

Αριθμός Προβλήματος	Tasks	Μήκος Πλάνου	Επισκεπτόμενοι Κόμβοι	Μέγιστο Βάθος Πλάνου	Χρόνος Εκτέλεσης
pfile01	1	18	53	52	0,051sec
pfile02	2	44	204	58	0.055sec
pfile03	3	68	165	58	0.058sec
pfile04	2	42	106	52	0.053sec
pfile05	3	68	165	58	0.058sec
pfile06	4	92	218	58	0.063sec
pfile07	2	42	106	52	0.063sec
pfile08	3	66	260	58	0.068sec
pfile09	4	90	212	52	0.071sec
pfile10	3	66	159	52	0.067sec
pfile11	6	132	428	58	0.086sec
pfile12	5	114	265	52	0.091sec

pfile13	9	212	483	58	0.107sec
pfile14	5	114	265	52	0.085sec
pfile15	10	234	530	52	0.113sec
pfile16	14	330	843	58	0.130sec
pfile17	20	456	1188	58	0.271sec
pfile18	30	686	1733	58	0.463sec
pfile19	40	890	2317	58	0.952sec
Pfile20	50	1194	2650	52	1.091sec

Πίνακας 5.2: Συγκεντρωτικά Αποτελέσματα Μήκους Πλάνων

5.3 Μελέτη Περίπτωσης

Προκειμένου να αποσαφηνιστεί ολοκληρωμένα ο τρόπος με τον οποίο υλοποιείται το σύστημα που αναπτύχθηκε στην παρούσα εργασία, παρατίθεται μια λεπτομερής μελέτη περίπτωσης. Ως αντιπροσωπευτικό παράδειγμα προσομοίωσης και ανάλυσης επιλέχθηκε το πρόβλημα δέκα (pfile10.hddl).

Η επιλογή του συγκεκριμένου σεναρίου έγινε με γνώμονα την ικανοποιητική πολυπλοκότητα του πεδίου, ιδανικό για να αναδείξει τις ικανότητες του συστήματος, χωρίς να προκύπτει ένα εκτενές αποτέλεσμα. Στο εν λόγω πρόβλημα το σύστημα καλείται να αποσυνθέσει τρεις διαφορετικές Εργασίες-Στόχους ακολουθιακά. Ωστόσο, ο βαθμός δυσκολίας αυξάνεται αισθητά, παρότι στο ευρύτερο περιβάλλον υπάρχουν πολλαπλά αντικείμενα κάθε τύπου, υπάρχει αποκλειστικά ένα αντικείμενο “shaker” και φυσικά δύο λειτουργικά χέρια, γεγονός το οποίο ωθεί στην επαναλαμβανόμενη δέσμευση και αποδέσμευση πόρων, καθώς επίσης και στον καθαρισμό τους.

Στις υποενότητες που ακολουθούν θα αναλυθεί βηματικά η ροή του κύκλου ζωής του προγράμματος. Αρχικά, περιγράφεται η εισαγωγή των δεδομένων παράλληλα με την αρχικοποίηση των υποσυστημάτων. Ύστερα, εξετάζεται ο τρόπος λειτουργίας του αλγορίθμου κατά την αποσύνθεση των Σύνθετων Εργασιών και η διαχείριση αδιεξόδων μέσω του μηχανισμού “backtracking”. Στην συνέχεια, εξηγείται η καίριας σημασίας φάση της επικύρωσης του προτεινόμενου πλάνου και η παρεμβολή της καθαριότητας των πόρων. Τέλος, αναλύεται η σύνθεση του τελικού αποτελέσματος και η ολοκληρωμένη παρουσίαση του κατά τον τερματισμό του προγράμματος.

5.3.1 Ανάλυση Εισόδου και Αρχικοποίηση Συστήματος

Την έναρξη όλου του συστήματος αποτελεί η εισαγωγή του προβλήματος. Αρχικά, το σύστημα περιμένει την απάντηση του χρήστη στην ερώτηση: "Enter the number of the problem file you want to solve (1-20) ", έπειτα αφού γίνουν οι κατάλληλοι έλεγχοι σχηματίζεται το “filepath”, το οποίο περνάει σαν παράμετρος στο αντικείμενο “ProblemParser”, στην κλάση του οποίου αναλύεται το αρχείο

προβλήματος. Κατά την ανάλυση αξιοποιείται η πληροφορία του αρχείου και με τις κατάλληλες μεθόδους:

- Εντοπίζονται και δημιουργούνται τα αντικείμενα του προβλήματος, τα οποία αποθηκεύονται ως “Map <String, Anything> objectsMap”.
- Αποθηκεύεται η αρχική κατάσταση του κόσμου σαν αντικείμενο της κλάσης “WorldState” ως “initialState”.
- Συντάσσονται οι ενέργειες προς αποσύνθεση σε μία λίστα της μορφής “List<String> goalTasks”.

Με αφετηρία την ολοκλήρωση της παραπάνω διαδικασίας, ο έλεγχος ροής επιστρέφει και πάλι στην κλάση “Main”. Σε αυτό το σημείο το κομβικό ενδιαμέσο στάδιο λαμβάνουν χώρα οι εξής ενέργειες αρχικοποίησης (Initialization):

- Ανάκτηση και Ρύθμιση Συστήματος: Αφού εξάγονται οι πληροφορίες από τον “Parser”, οι καίριες δομές αποθηκεύονται στην δομή “DomainHelper”. Έτσι προσφέρεται άμεση και στατική πρόσβαση σε όλη την πληροφορία από όλα τα τμήματα του προγράμματος.
- Μετατροπή Στόχων μέσω Task Factory: Οι απλές συμβολοσειρές με τις οποίες αποθηκεύονται οι ενέργειες “goalTasks” απαρτίζονται από το όνομα τις εργασίας και τις παραμέτρους τους. Αυτές οι πληροφορίες τροφοδοτούνται στο σχεδιαστικό πρότυπο εργοστασίου με την μέθοδο Factory.createTask(), το οποίο αναλαμβάνει δυναμικά την κατασκευή πολυμορφικών αντικειμένων τύπου “Task”. Τα αντικείμενα πλέον αντικαθιστούν τις εκκρεμείς εργασίες του δέντρου προς επίλυση.
- Αρχικοποίηση Υποσυστημάτων: Τελικά, δημιουργούνται οι βασικοί μηχανισμοί του συστήματος, ο μηχανισμός αναζήτησης πλάνου “Planner”, ο επικυρωτής των παραγόμενων σεναρίων “Validator” και ο μηχανισμός καθαριότητας “Cleanup”.

Από αυτό το σημείο και έπειτα, η κεντρική δομή ακολουθεί μια επαναληπτική ροή, επιλύοντας σειριακά τις ενέργειες προς αποσύνθεση ξεχωριστά.

5.3.2 Μηχανισμός Αποσύνθεσης και Διαχείρισης Αδιεξόδων

Με την ολοκλήρωση του προηγούμενου βήματος, κάθε επιμέρους στόχος της λίστας εισάγεται σε συνδυασμό με την τρέχουσα κατάσταση στον αλγόριθμο του “Planner”. Ο “Planner” λειτουργεί αναδρομικά, αξιοποιώντας μια συστηματική στρατηγική αναζήτησης σε βάθος, η οποία υλοποιείται μέσω της μεθόδου “dfs()”

Σημείο εκκίνησης κάθε επαναληπτικού βήματος αποτελεί η εξαγωγή της πρώτης εργασίας προς εκτέλεση (tasks.get(0)). Αν αποτελεί Σύνθετη Εργασία (Compound Task), το σύστημα αναζητά διαδοχικά όλες τις πιθανές Μεθόδους Αποσύνθεσης της (Decomposition Method). Για κάθε μία από αυτές, ελέγχεται μέσω του αντικειμένου

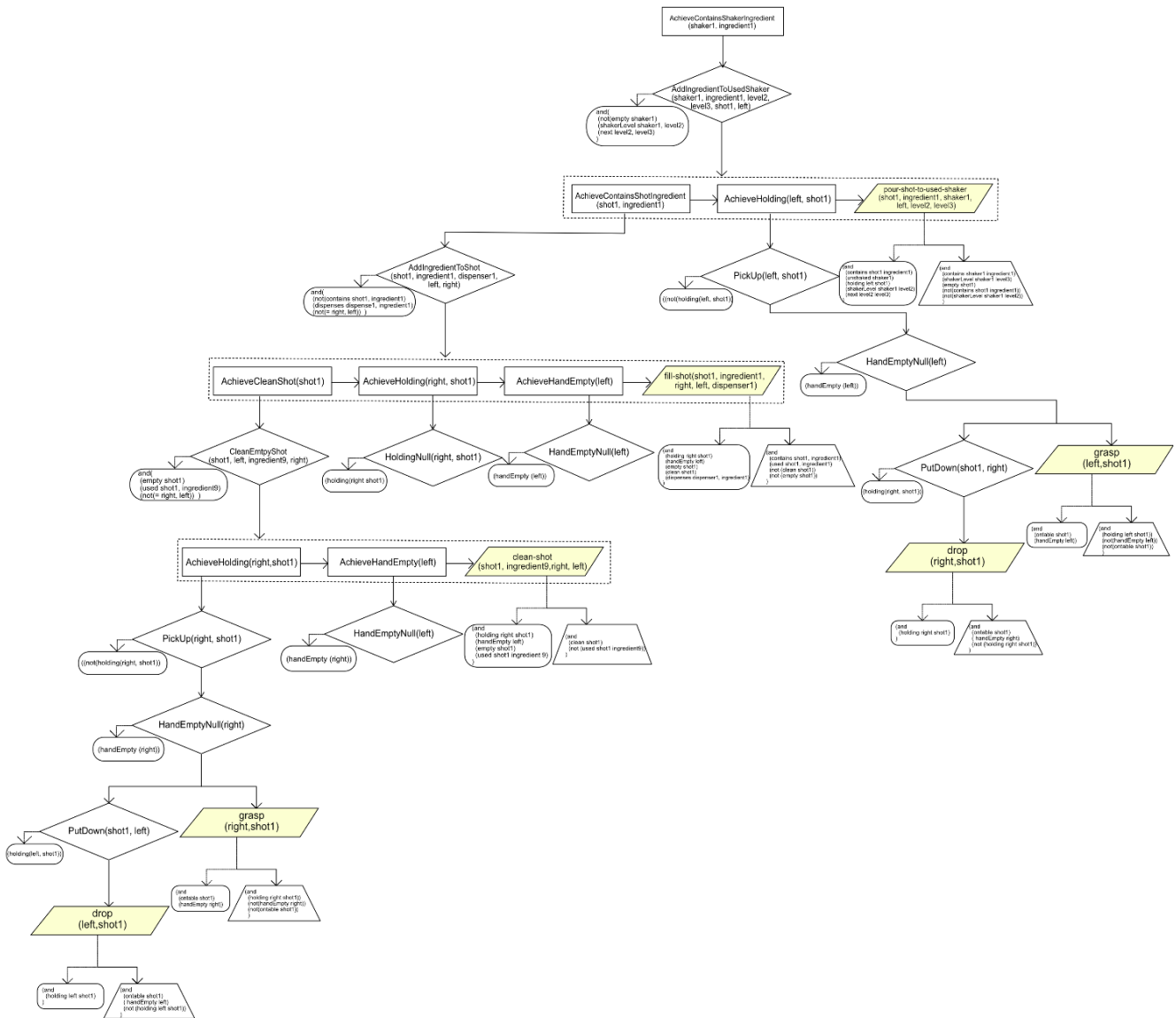
“preconditionChecker” αν ικανοποιούνται οι προϋποθέσεις της “Preconditions” πάνω στην τρέχουσα κατάσταση του κόσμου “WorldState”. Σε περίπτωση που βρεθεί μία έγκυρη μέθοδος, καταγράφεται άμεσα στο γενικό ιστορικό, την λεγόμενη ατζέντα “agenda”, διατηρώντας το όνομα και τις παραμέτρους της. Επιπλέον, η αρχική Σύνθετη Εργασία αντικαθίσταται από τις νέες, πιο αναλυτικές υποεργασίες “subtasks”, τα οποία δόθηκαν από την Μέθοδο. Αυτές εισάγονται στην κορυφή της στοίβας των εναπομεινάντων εργασιών “nextTasks.addAll()”.

Η διαδικασία της αποσύνθεσης επαναλαμβάνεται, καθώς εμβαθύνει σε όλο και βαθύτερα επίπεδα του δέντρου, έως ότου ο αλγόριθμος εξάγει μία Πρωτογενής Ενέργεια (Primitive Action). Λόγω της αυστηρότητας που προσδίδουν τα προβλήματα Ιεραρχικού Σχεδιασμού (HTN), για κάθε στοιχειώδη πράξη ελέγχονται εξαρχής οι προϋποθέσεις εφαρμογής. Σε περίπτωση επιτυχούς επιβεβαίωσης, το σύστημα δεν μεταβάλλει άμεσα τον κόσμο, αλλά κατασκευάζει ένα αντίγραφο της κατάστασης “state.copyState()” ώστε να προσθέτει με ασφάλεια τις επιδράσεις της “effects” πάνω σε αυτό. Εν συνέχεια, η ενέργεια εισάγεται στο προσχέδιο του τελικού πλάνου “plan”.

Ωστόσο, σε περιορισμένα περιβάλλοντα σαν αυτό που αντιμετωπίζεται, στην παρούσα εργασία, είναι αναμενόμενο ο αλγόριθμος να έρθει αντιμέτωπος με αδιέξοδα. Για παράδειγμα, κατά τη διαδικασία ανάμειξης, ο “Planner” μπορεί ατυχώς να δεσμεύσει και τα δύο λειτουργικά χέρια κρατώντας για παράδειγμα ποτήρια αντί να τα τοποθετήσει στον πάγκο “ontable”, με αποτέλεσμα να είναι αδύνατη η χρήση των διανομέων “dispenser” για τα επόμενα βήματα παρασκευής.

Όταν το σύστημα αντιληφθεί πως καμία εναπομένουσα Μέθοδος ή Ενέργεια δεν ικανοποιεί τις αναγκαίες προϋποθέσεις, η αναδρομική κλήση της “dfs” αποτυγχάνει επιστρέφοντας “null”. Αυτό το σήμα πυροδοτεί άμεσα τον καταλυτικό μηχανισμό οπισθοδρόμησης “backtracking”, ο οποίος αναιρεί με ασφάλεια το εσφαλμένο εξερευνητικό μονοπάτι. Η τελευταία επιλογή διαγράφεται από την “agenda” μέσω της “agenda.remove()” και αντίστοιχα, η εσφαλμένη ενέργεια αφαιρείται από την ακολουθία του “plan”. Επιπλέον, το σύστημα επιστρέφει ομαλά στον κόμβο επιλογής για να δοκιμάσει την επόμενη εναλλακτική Μέθοδο της ουράς. Εδώ αναδεικνύεται η μεγάλη αξία της τεχνικής του “copyState()”, καθώς το σύστημα ανακαλεί και επιστρέφει ασφαλώς στην προηγούμενη ανεπηρέαστη κατάσταση, χωρίς να παρατηρούνται αλλοιώσεις από τα προσδοκόμενα “effects” του αποτυχημένου κλαδιού.

Αυτός ο κύκλος ελέγχων, αποσύνθεσης και οπισθοδρόμησης, συνεχίζεται διαρκώς μέχρι η λίστα εναπομεινάντων εργασιών “Tasks” να αδειάσει πλήρως. Η κενή αναπαράσταση της λίστας λειτουργεί ως η βασική συνθήκη τερματισμού της Μεθόδου, ενημερώνοντας την κεντρική κλάση πως τακτοποιήθηκε ένα λειτουργικό πλάνο βημάτων έτοιμο να επικυρωθεί και να σταθεροποιηθεί στο σύστημα.



Σχήμα 5.1: Δείγμα Διαγράμματος Ροής του Δέκατου Προβλήματος

5.3.3 Επικύρωση και Ενδιάμεσος Καθαρισμός Πόρων

Μόλις ο αλγόριθμος “Planner” πλοηγηθεί επιτυχώς στο δέντρο επιλογών και επιστρέψει ένα έγκυρο πλάνο για τον εκάστοτε επιμέρους στόχο, το εν λόγω προσχέδιο εισέρχεται στη Φάση Επικύρωσης “Validation Phase” προτού αφομοιωθεί επίσημα. Μέσω της κλάσης “Main”, η ακολουθία πλάνου συναντά το αντικείμενο “Validator” της κλάσης “Validate”, το οποίο λειτουργεί ως αντικειμενικός ανεξάρτητος προσομοιωτής περιβάλλοντος.

Ο επικυρωτής λαμβάνεται το προτεινόμενο πλάνο και επιχειρεί να προσομοιώσει κάθε Πρωτογενή Ενέργεια του σειριακά, σε συνθήκες πραγματικής εκτέλεσης. Κατά τη διάρκεια της επικύρωσης, εάν έστω και μία Ενέργεια προσπαθήσει να εκτελεστεί χωρίς να επικυρώσει τις προϋποθέσεις “Preconditions” κατά την

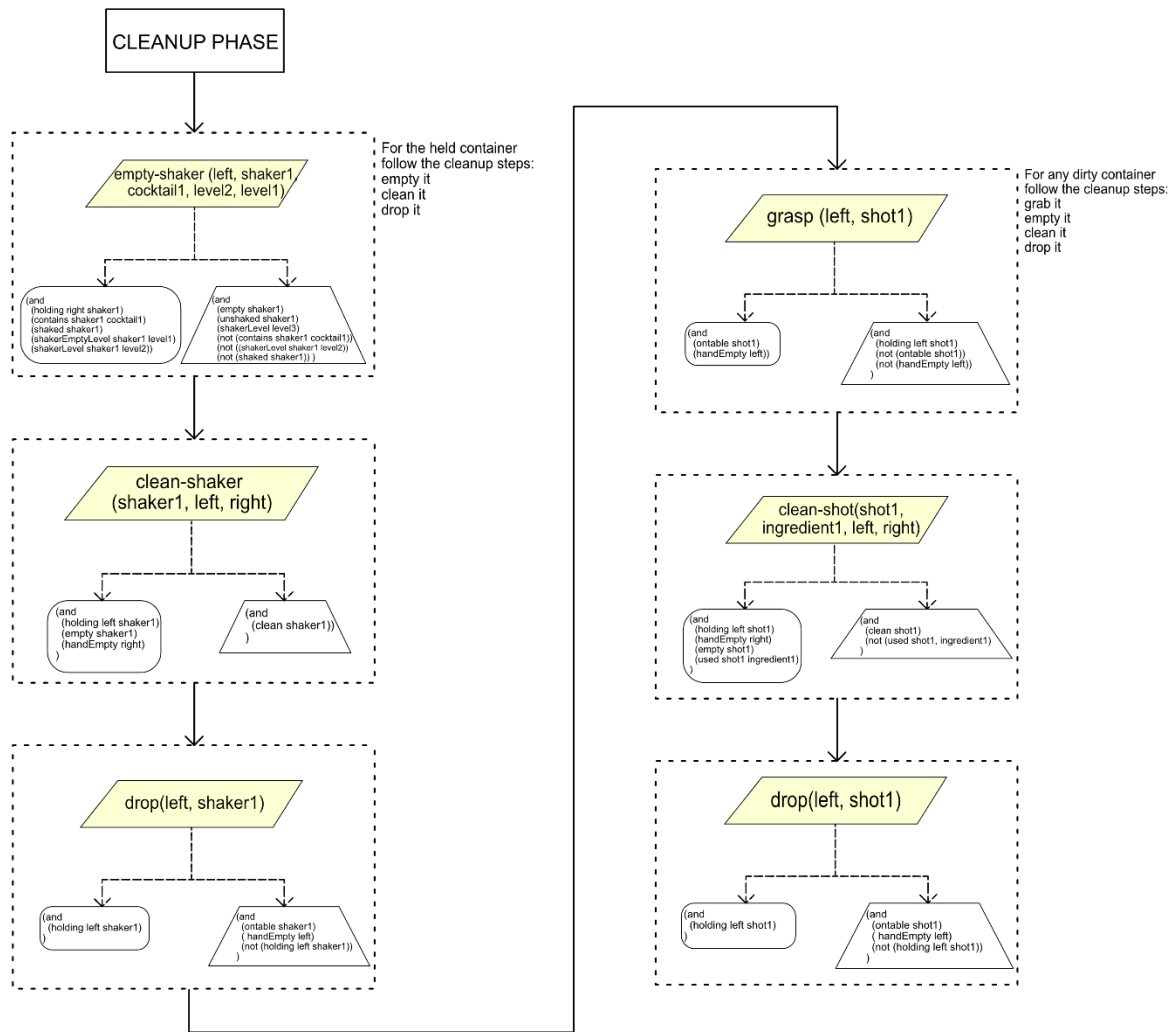
αντίστοιχη χρονική στιγμή, η διαδικασία αναστέλλεται εμφανίζοντας σφάλμα. Αν όμως όλες οι προϋποθέσεις ικανοποιούνται, η συνδεδεμένη κλάση “ValidationResult” ανταποκρίνεται θετικά. Μόνο υπό αυτή τη συνθήκη το αποτέλεσμα ενσωματώνεται στον κεντρικό έλεγχο, ενημερώνοντας πλέον οριστικά τη γενική κατάσταση του κόσμου “currentState = validation.finalState.copyState()”.

Σε αυτό ακριβώς το σημείο αναδεικνύεται η δυσκολία του προβλήματος “pfile10”, η ύπαρξη ενός μοναδικού λειτουργικού “shaker”. Με το πέρας της παρασκευής και του σερβιρίσματος του πρώτου “cocktail”, τα χέρια παραμένουν δεσμευμένα, κρατώντας άδεια ποτήρια ή το ίδιο το “shaker”, ενώ ταυτόχρονα το ίδιο το “shaker” μένει λερωμένο από τα προηγούμενα υλικά που χρησιμοποιήθηκαν. Αυτό σημαίνει πως αν το σύστημα μεταβεί απευθείας στην επίλυση του επόμενου στόχου, ο “Planner” θα αποτύγχανε εξ’ αρχής να ικανοποιήσει τις δικές του αρχικές προϋποθέσεις, βαφτίζοντας το συνολικό πλάνο ανέφικτο, οδηγώντας διαρκώς σε αδιέξοδα.

Για την αντιμετώπιση αυτής της ιδιοτροπίας, έχει αναπτυχθεί ο μηχανισμός Καθαρισμού και Αποδέσμευσης “Cleanup Phase”. Προτού το σύστημα αναθέσει στον “Planner” την επόμενη βασική Ενέργεια-Στόχο της λίστας, η κεντρική ροή εκτέλεσης του προγράμματος διακόπτεται, δίνοντας έλεγχο στο αντικείμενο “Cleanup”. Ο μηχανισμός αυτός αξιολογεί την τρέχουσα κατάσταση του κόσμου και επιτηρεί μια σειρά εσωτερικών τακτοποιήσεων, δημιουργώντας ένα εξειδικευμένο πλάνο ρύθμισης.

1. Αρχικά, κάνοντας του κατάλληλους ελέγχους, αναγνωρίζει και εξασφαλίζει σε μία ειδική λίστα “goalShots” τα ποτήρια τα οποία ήδη περιέχουν τα ήδη ολοκληρωμένα κοκτέιλ, αποφεύγοντας την κατά λάθος απόρριψη ή άδειασμα τους.
2. Στην συνέχεια, δίνει εντολή στο σύστημα για την αποδέσμευση των περιττών αντικειμένων, που κρατούν τα χέρια (left/right) τοποθετώντας τα αντικείμενα στον πάγκο, ελευθερώνοντας τα χέρια.
3. Τέλος της διαδικασίας αποτελεί η εισαγωγή εμβόλιμων ενεργειών καθαρισμού, όπως το “clean” και το “empty”, με τις οποίες το βασικό εργαλείο εξυπηρέτησης πλένεται και επαναφέρεται άδειο και καθαρό στο τραπέζι “ontable”.

Αυτή η αυτοματοποιημένη ακολουθία πράξεων καθαρισμού περνάει και αυτή με τη σειρά της από τον αυστηρό ελεγκτή επικύρωσης. Μετά την επιτυχία της ενσωμάτωσης, το σύστημα επανέρχεται ευέλικτο, διαθέτοντας έναν καθαρό εικονικό πάγκο και ελευθέρα άκρα, διαδικασία που επιτρέπει στον βασικό “Planner” να εστιάσει ανεμπόδιστα στην εύρεση του επόμενου απαιτητικού πλάνου παρασκευής του επόμενου “cocktail”.



Σχήμα 5.2: Διαδικασία Καθαρισμού και Επαναφοράς του Κόσμου

5.3.4 Σύνθεση Τελικού Πλάνου και Αποτελέσματα

Καθώς η αυστηρή επαναληπτική ροή της κλάσης “Main” ολοκληρώνει τον κύκλο της, ο αλγόριθμος αναζήτησης και η δυναμική φάση καθαρισμού αναλύουν, συρράπτουν και ικανοποιούν επιτυχώς όλους τους ζητούμενους επιμέρους στόχους. Καθ’ όλη τη διάρκεια αυτού του κύκλου, το σύστημα φροντίζει να παρακολουθεί και να αθροίζει προσεκτικά τα επιμέρους επικυρωμένα σχέδια δράσης. Πιο συγκεκριμένα, αποθηκεύονται τακτικά στις ευρύτερες δομές “fullAgenda” ή οποία καταγράφει όλο το ιεραρχικό ιστορικό των μεθόδων και της αποσύνθεσης, καθώς και το “fullPlan”, η οποία περιέχει αποκλειστικά το αθροιστικό πλάνο των εκτελέσιμων στοιχειωδών ενεργειών.

Με τον επιτυχή τερματισμό και του τελευταίου ιεραρχικό στόχου, το κεντρικό υποσύστημα αξιολογεί τη μεταβλητή τύπου boolean “allSuccess”. Εφόσον όλοι οι στόχοι έχουν επιλυθεί δίχως καμία εμφανή ανωμαλία σε κάποια φάση της προσομοίωσης, επικυρώνεται η καθολική αξιοπιστία της τελικής ενιαίας διαδρομής.

Σε αυτό το σημείο ολοκλήρωσης το πρόγραμμα εισέρχεται στη φάση της Παρουσίασης Αποτελεσμάτων “Final Results Phase”, όπου μέσω της κονσόλας παραδίδεται στον χρήστη η τελική πλήρης εικόνα. Αρχικά, εκτυπώνεται ολόκληρο το ενοποιημένο Ιστορικό Ενημέρωσης “Full Agenda”, παρέχοντας διαφάνεια στην σειρά και στον τρόπο με τον οποίο επέλεξε ο αλγόριθμος να δομήσει τις κινήσεις του. Αμέσως μετά, ανακτάται το Τελικό Πλάνο “List fullPlan” και εκτυπώνεται στον χρήστη μια απλή, συμπυκνωμένη και αριθμημένη σειριακή ακολουθία στενευμένων ενεργειών του τύπου “1.(grasp left shaker 1)”. Αυτό το τελικό “output” είναι ουσιαστικά το «Τελικό Προϊόν» του “Planner”, μια αλυσίδα εντολών έτοιμη θεωρητικά να εισαχθεί σε ένα πραγματικά λειτουργικό σύστημα.

Τέλος, προκειμένου να αποδειχθεί ποσοτικά η αναλυτική ικανότητα, η ταχύτητα και η αποδοτικότητα του συνολικού “HTN” αλγορίθμου, το σύστημα συλλέγει τα δεδομένα του ενσωματωμένου χρονομέτρου “System.currentTimeMillis()”, το οποίο εκκινείται ταυτόχρονα με την αρχικοποίηση του “ProblemParser”. Υπολογίζει τη συνολική διαφορά του χρόνου και παρουσιάζει στον χρήστη τον καθαρό χρόνο εκτέλεσης σε δευτερόλεπτα “Total Execution Time”. Για απαιτητικά σενάρια διαχείρισης και περιορισμού κοινών πόρων, όπως το “rfile10”, η αμεσότητα εύρεσης και επιστροφής αυτού του ενιαίου πλάνου επιβεβαιώνει την στιβαρότητα όλης της προγραμματιστικής προσέγγισης, τερματίζοντας ομαλά και επιτυχώς τον κύκλο ζωής του αυτοματοποιημένου συστήματος.

6 Συμπεράσματα και Μελλοντική Εργασία

6.1 Συμπεράσματα

Η παρούσα πτυχιακή εργασία ολοκληρώνεται με την επιτυχή σχεδίαση και υλοποίηση ενός Ιεραρχικού Σχεδιαστή “HTN Planner” στην αντικειμενοστρεφή γλώσσα Java, εξειδικευμένου στο πεδίο “Barman” του Διεθνούς Διαγωνισμού Σχεδιασμού (IPC 2023). Μέσα από την πειραματική διαδικασία, αποδείχθηκε ότι η Ιεραρχική προσέγγιση είναι εξαιρετικά αποδοτική για τα προβλήματα που απαιτούν σύνθετη διαχείριση πόρων και καταστάσεων.

Η αρχιτεκτονική του υλοποιημένου συστήματος κατάφερε να γεφυρώσει αρμονικά το χάσμα μεταξύ θεωρητικών αλγορίθμων Τεχνητής Νοημοσύνης και μιας πρακτικής, εκτελέσιμης εφαρμογής. Η ενσωμάτωση του μηχανισμού αναζήτησης σε βάθος (Depth-First Search), σε άμεση συνεργασία με τους αναπτυχθέντες, εξειδικευμένους μηχανισμούς επικύρωσης (Validation) και εμβόλιμου καθαρισμού (Cleanup Phase), προσέδωσε στο σύστημα τη δυνατότητα ασφαλούς απεγκλωβισμού από αδιέξοδα (dead-ends). Ταυτόχρονα, η αμιγώς αντικειμενοστρεφής σχεδίαση εξασφάλισε υψηλή αναγνωσιμότητα, ασφάλεια και δυνατότητα εύκολης συντήρησης, επιβεβαιώνοντας πως η Java αποτελεί ένα ισχυρό, ακαδημαϊκό εργαλείο για την ανάπτυξη στοχευμένων λύσεων σε ρομποτικές εφαρμογές.

6.2 Περιορισμοί Υλοποίησης

Παρά την επιτυχή επίλυση των σεναρίων, όπως αναδείχθηκε μέσω του “rfile10”, ο αναπτυχθείς σχεδιαστής “Planner” παρουσιάζει ορισμένους τεχνικούς περιορισμούς που τέθηκαν συνειδητά με γνώμονα τη χρονική βιωσιμότητα και την εστίαση του έργου στην αλγοριθμική επίλυση. Πιο συγκεκριμένα:

1. Στατική Αναπαράσταση Πεδίου (Hardcoded Domain Rules): Το μεγαλύτερο ίσως μειονέκτημα της τρέχουσας υλοποίησης εντοπίζεται στο γεγονός ότι, ενώ τα δομικά προβλήματα “rfile.hddl” διαβάζονται και μεταφράζονται δυναμικά στον χρόνο εκτέλεσης “runtime” χάρη στον “ProblemParser”, οι νόμοι του πεδίου παραμένουν στατικοί. Η εσωτερική ιεραρχία, οι προϋποθέσεις (preconditions) και οι επιδράσεις (effects) των Μεθόδων και Ενεργειών υλοποιήθηκαν ως αυτόνομες κλάσεις (π.χ. AchieveContainsShotCocktail, PourShakerToShot κ.ο.κ.). Συνεπώς, κάθε πιθανή μελλοντική προσθήκη μίας νέας ιδιότητας απαιτεί συγγραφή νέου κώδικα και μεταγλώττιση του προβλήματος.
2. Απουσία Ευρετικών Μηχανισμών (Heuristics): Η περιήγηση στο δέντρο αναζήτησης συντελείται αποκλειστικά από τον κλασσικό αλγόριθμο “DFS”. Παρότι η διαδικασία εγγυάται την εύρεση λύσης αν αυτή υπάρχει, η απουσία ευρετικών συναρτήσεων καθιστά τον αλγόριθμο «τυφλό». Ως αποτέλεσμα, σε

προβλήματα τέτοιας πολυπλοκότητας, ο “Planner” κινδυνεύει να ξοδέψει υπολογιστικό χρόνο αναλύοντας λανθασμένα μονοπάτια προτού ενεργοποιήσει το “backtracking”, αντί να κατευθύνεται έξυπνα προς τις πιο “υποσχόμενες” επιλογές εξαρχής.

6.3 Προτάσεις για Μελλοντική Έρευνα και Επέκταση

Λαμβάνοντας υπόψη τα συμπεράσματα και τους περιορισμούς της παρούσας μελέτης, διακρίνονται πολλαπλά περιθώρια περαιτέρω ανάπτυξης και ερευνητικής επέκτασης της εφαρμογής. Ορισμένες από τις σημαντικότερες κατευθύνσεις εντοπίζονται στις εξής:

1. Δυναμική Ανάλυση Πεδίου (Dynamic Domain Parsing): Η ενσωμάτωση ενός καθολικού αναγνώστη για τη δυναμική επεξεργασία του αρχείου “domain.hddl” θα μετέτρεπε το σύστημα σε έναν πλήρως ανεξάρτητο “Planner” (Domain Independent HTN Planner). Έτσι, η εφαρμογή θα μπορούσε να υποστηρίξει εντελώς διαφορετικά πεδία πέραν του “Barman”.
2. Υποστήριξη Μερικής Διάταξης (Partial-Order Planning): Η μετάβαση από τα σειριακά πλάνα στο πολυνηματικό σχεδιασμό θα επέτρεπε την παράλληλη εκτέλεση δράσεων από τα δύο αυτόνομα χέρια, γεγονός το οποίο θα μπορούσε να αποσυνθέτει ταυτόχρονα πολλές Σύνθετες Εργασίες.
3. Ανάπτυξη Γραφικού Περιβάλλοντος Διεπαφής (GUI): Η δημιουργία ενός σύγχρονου γραφικού περιβάλλοντος θα αντικαθιστούσε τη διεπαφή στο “terminal”. Η προσομοίωση των κινήσεων σε πραγματικό χρόνο θα προσέφερε στον χρήστη μια σαφή και άμεση εικόνα των λειτουργιών του συστήματος.

Βιβλιογραφία

- [1] Russell, S. and Norvig, P. (2021). *Τεχνητή Νοημοσύνη: Μια Σύγχρονη Προσέγγιση*. 4^η Αμερικανική Έκδοση. Αθήνα: Εκδόσεις Κλειδάριθμος.
- [2] Ghallab, M., Knoblock, C., Wilkins, D., Barrett, A., Christianson, D., Friedman, M., Kwok, C., Golden, K., Penberthy, S., Smith, D., Sun, Y., & Weld, D.(1998) *PDDL - The Planning Domain Definition Language*.
https://planning.wiki/_citedpapers/pddl1998.pdf
- [3] Höller, D., Behnke, G., Bercher, P., Biundo, S., Fiorino, H., Pellier, D., & Alford, R. (2020). HDDL: An Extension to PDDL for Expressing Hierarchical Planning Problems *.Proceedings of the AAAI Conference on Artificial Intelligence, 34(06)*, 9883-9891. <https://doi.org/10.1609/aaai.v34i06.6542>
- [4] McDermott, D. M. (2000). The 1998 AI Planning Systems Competition. *AI Magazine, 21(2)*, 35.
<https://doi.org/10.1609/aimag.v21i2.1506>
- [5] International Planning Competition 2023 – IPC 2023: <https://ipc2023.github.io/>
- [6] IPC 2023 - Classical Tracks: <https://ipc2023-classical.github.io/>
- [7] IPC 2023 - Learning Tracks: <https://ipc2023-learning.github.io/>
- [8] IPC 2023 - Probabilistic Tracks: <https://ataitler.github.io/IPPC2023/>
- [9] IPC 2023 – Numeric Tracks: <https://ipc2023-numeric.github.io/>
- [10] IPC 2023 - HTN Tracks: <https://ipc2023-htn.github.io/>
- [11] IPC 2023 – Domain “Barman”: <https://github.com/ipc2023-htn/ipc2023-domains/blob/main/total-order/Barman-BDI/domain.hddl>
- [12] IPC 2023 – First Problem of Domain “Barman”: <https://github.com/ipc2023-htn/ipc2023-domains/blob/main/total-order/Barman-BDI/pfile01.hddl>
- [13] Bercher, P. and Höller, D. (2018). *Tutorial: An Introduction to Hierarchical Task Network (HTN) Planning*. International Conference on Automated Planning and Scheduling (ICAPS 2018), Delft, Netherlands, June 25.

[14] Kvarnström, J. (2022). *Planning domains and problem instances in PDDL*. Department of Computer and Information Science, Linköping University.
<https://www.ida.liu.se/~TDDD48/labs/2022/pddl.en.shtml>

[15] Ilche Georgievski and Marco Aiello. (2014). *An Overview of Hierarchical Task Network Planning*. Distributed Systems Group Johann Bernoulli Institute for Mathematics and Computer Science University of Groningen.
<https://doi.org/10.48550/arXiv.1403.7426>

ΠΑΡΑΡΤΗΜΑ Α

Δείγματα Αρχείων HDDL

A.1 Πλήρης Κώδικας του Πεδίου “Barman”

Ακολουθεί ο πλήρης κώδικας του πεδίου “Barman” σε γλώσσα HDDL, όπως αυτός ορίζεται από τον Διεθνή Διαγωνισμό Σχεδιασμού και ορίζει το περιβάλλον του domain, με τους κανόνες και τις οντότητες του.

```
1(define (domain barman_htn)
2(:requirements :negative-preconditions
3 :hierarchy
4 :typing
5 :equality
6 :method-preconditions
7)
8(:types
9 container dispenser level beverage hand - anything
10 shot shaker - container
11 ingredient cocktail - beverage
12)
13(:predicates
14 (clean ?p0 - container)
15 (cocktailPart1 ?p0 - cocktail ?p1 - ingredient)
16 (cocktailPart2 ?p0 - cocktail ?p1 - ingredient)
17 (contains ?p0 - container ?p1 - beverage)
18 (dispenses ?p0 - dispenser ?p1 - ingredient)
19 (empty ?p0 - container)
20 (handEmpty ?p0 - hand)
21 (holding ?p0 - hand ?p1 - container)
22 (ingredient ?p0 - ingredient)
23 (next ?p0 - level ?p1 - level)
24 (ontable ?p0 - container)
25 (shaked ?p0 - shaker)
26 (shakerEmptyLevel ?p0 - shaker ?p1 - level)
27 (shakerLevel ?p0 - shaker ?p1 - level)
28 (unshaked ?p0 - shaker)
29 (used ?p0 - container ?p1 - beverage)
30)
31(:task AchieveContainsShakerIngredient :parameters (?x_0-
shaker ?x_1- ingredient))
32(:task AchieveCleanShaker :parameters (?x_0 - shaker))
33(:task AchieveHandEmpty :parameters (?x_0 - hand))
```



```

34 (:task AchieveContainsShotIngredient :parameters (?x_0 - shot ?x_1 -
ingredient))
35 (:task AchieveContainsShakerCocktail :parameters (?x_0 - shaker ?x_1
- cocktail))
36 (:task DoPourShakerToShot :parameters (?x_0 - shaker ?x_1 - shot ?x_2
- cocktail))
37 (:task AchieveOnTable :parameters (?x_0 - container))
38 (:task AchieveHolding :parameters (?x_0 - hand ?x_1 - container))
39 (:task AchieveCleanShot :parameters (?x_0 - shot))
40 (:task AchieveContainsShotCocktail :parameters (?x_0 - shot ?x_1 -
cocktail))
41
42 (:method MakeAndPourCocktail
43 :parameters (?x_0 - shot ?x_1 - cocktail ?x_2 - shaker ?x_3 - hand)
44 :task (AchieveContainsShotCocktail ?x_0 ?x_1)
45 :precondition (and
46 (not (contains ?x_0 ?x_1))
47 )
48 :ordered-subtasks (and
49 (AchieveContainsShakerCocktail ?x_2 ?x_1 )
50 (AchieveCleanShot ?x_0)
51 (AchieveHolding ?x_3 ?x_2)
52 (DoPourShakerToShot ?x_2 ?x_0 ?x_1)
53 )
54 )
55 (:method MakeAndPourCocktailNull
56 :parameters (?x_0 - shot ?x_1 - cocktail)
57 :task (AchieveContainsShotCocktail ?x_0 ?x_1)
58 :precondition (and
59 (contains ?x_0 ?x_1)
60 )
61 :ordered-subtasks (and
62 )
63 )
64 (:method MakeCocktail
65 :parameters (?x_0 - shaker ?x_1 - cocktail ?x_2 - ingredient ?x_3 -
hand ?x_4 - hand ?x_5 - ingredient)
66 :task (AchieveContainsShakerCocktail ?x_0 ?x_1)
67 :precondition (and
68 (cocktailPart1 ?x_1 ?x_5)
69 (cocktailPart2 ?x_1 ?x_2)
70 (not (= ?x_4 ?x_3))
71 )

```

```

72 :ordered-subtasks (and
73   (AchieveCleanShaker ?x_0)
74   (AchieveContainsShakerIngredient ?x_0 ?x_5)
75   (AchieveContainsShakerIngredient ?x_0 ?x_2)
76   (AchieveHolding ?x_4 ?x_0)
77   (AchieveHandEmpty ?x_3)
78   (shake ?x_1 ?x_5 ?x_2 ?x_0 ?x_4 ?x_3)
79 )
80)
81(:method MakeCocktailNull
82 :parameters (?x_0 - shaker ?x_1 - cocktail)
83 :task (AchieveContainsShakerCocktail ?x_0 ?x_1)
84 :precondition (and
85   (contains ?x_0 ?x_1)
86 )
87 :ordered-subtasks (and
88 )
89)
90(:method AddIngredientToEmptyShaker
91 :parameters (?x_0 - shaker ?x_1 - ingredient ?x_2 - level ?x_3 -
  level ?x_4 - shot ?x_5 - hand)
92 :task (AchieveContainsShakerIngredient ?x_0 ?x_1)
93 :precondition (and
94   (empty ?x_0)
95   (clean ?x_0)
96   (shakerLevel ?x_0 ?x_2)
97   (next ?x_2 ?x_3)
98 )
99 :ordered-subtasks (and
100   (AchieveContainsShotIngredient ?x_4 ?x_1)
101   (AchieveHolding ?x_5 ?x_4)
102   (pour-shot-to-clean-shaker ?x_4 ?x_1 ?x_0 ?x_5 ?x_2 ?x_3)
103 )
104)
105(:method AddIngredientToUsedShaker
106 :parameters (?x_0 - shaker ?x_1 - ingredient ?x_2 - level ?x_3 -
  level ?x_4 - shot ?x_5 - hand)
107 :task (AchieveContainsShakerIngredient ?x_0 ?x_1)
108 :precondition (and
109   (not (empty ?x_0))
110   (shakerLevel ?x_0 ?x_2)
111   (next ?x_2 ?x_3)

```

```

112 )
113 :ordered-subtasks (and
114   (AchieveContainsShotIngredient ?x_4 ?x_1)
115   (AchieveHolding ?x_5 ?x_4)
116   (pour-shot-to-used-shaker ?x_4 ?x_1 ?x_0 ?x_5 ?x_2 ?x_3)
117 )
118 )
119 (:method AddIngredientToShakerNull
120   :parameters (?x_0 - shaker ?x_1 - ingredient)
121   :task (AchieveContainsShakerIngredient ?x_0 ?x_1)
122   :precondition (and
123     (contains ?x_0 ?x_1)
124   )
125   :ordered-subtasks (and
126   )
127 )
128 (:method AddIngredientToShot
129   :parameters (?x_0 - shot ?x_1 - ingredient ?x_2 - dispenser ?x_3 -
hand ?x_4 - hand)
130   :task (AchieveContainsShotIngredient ?x_0 ?x_1)
131   :precondition (and
132     (not (contains ?x_0 ?x_1))
133     (dispenses ?x_2 ?x_1)
134     (not (= ?x_4 ?x_3))
135   )
136   :ordered-subtasks (and
137     (AchieveCleanShot ?x_0)
138     (AchieveHolding ?x_4 ?x_0)
139     (AchieveHandEmpty ?x_3)
140     (fill-shot ?x_0 ?x_1 ?x_4 ?x_3 ?x_2)
141   )
142 )
143 (:method AddIngredientToShotNull
144   :parameters (?x_0 - shot ?x_1 - ingredient)
145   :task (AchieveContainsShotIngredient ?x_0 ?x_1)
146   :precondition (and
147     (contains ?x_0 ?x_1)
148   )
149   :ordered-subtasks (and
150   )
151 )
152 (:method CleanFullShot

```

```

153 :parameters (?x_0 - shot ?x_1 - hand ?x_2 - beverage ?x_3 - hand)
154 :task (AchieveCleanShot ?x_0 )
155 :precondition (and
156   (contains ?x_0 ?x_2)
157   (not (= ?x_3 ?x_1))
158 )
159 :ordered-subtasks (and
160   (AchieveHolding ?x_3 ?x_0)
161   (empty-shot ?x_3 ?x_0 ?x_2 )
162   (AchieveHandEmpty ?x_1)
163   (clean-shot ?x_0 ?x_2 ?x_3 ?x_1)
164 )
165 )
166 (:method CleanEmptyShot
167 :parameters (?x_0 - shot ?x_1 - hand ?x_2 - beverage ?x_3 - hand)
168 :task (AchieveCleanShot ?x_0)
169 :precondition (and
170   (empty ?x_0)
171   (used ?x_0 ?x_2)
172   (not (= ?x_3 ?x_1))
173 )
174 :ordered-subtasks (and
175   (AchieveHolding ?x_3 ?x_0)
176   (AchieveHandEmpty ?x_1)
177   (clean-shot ?x_0 ?x_2 ?x_3 ?x_1)
178 )
179 )
180 (:method CleanShotNull
181 :parameters (?x_0 - shot)
182 :task (AchieveCleanShot ?x_0)
183 :precondition (and
184   (clean ?x_0)
185 )
186 :ordered-subtasks (and
187 )
188 )
189 (:method CleanEmptyShaker
190 :parameters (?x_0 - shaker ?x_1 - hand ?x_2 - hand)
191 :task (AchieveCleanShaker ?x_0)
192 :precondition (and
193   (not (clean ?x_0))
194   (empty ?x_0)

```

```

195     (not (= ?x_2 ?x_1))
196   )
197   :ordered-subtasks (and
198     (AchieveHolding ?x_2 ?x_0)
199     (AchieveHandEmpty ?x_1)
200     (clean-shaker ?x_0 ?x_2 ?x_1)
201   )
202 )
203 (:method CleanFullShaker
204   :parameters (?x_0 - shaker ?x_1 - level ?x_2 - cocktail ?x_3 -
hand ?x_4 - hand ?x_5 - level )
205   :task (AchieveCleanShaker ?x_0)
206   :precondition (and
207     (contains ?x_0 ?x_2)
208     (shaked ?x_0)
209     (shakerEmptyLevel ?x_0 ?x_1)
210     (shakerLevel ?x_0 ?x_5)
211     (not (= ?x_4 ?x_3))
212   )
213   :ordered-subtasks (and
214     (AchieveHolding ?x_4 ?x_0)
215     (empty-shaker ?x_4 ?x_0 ?x_2 ?x_5 ?x_1)
216     (AchieveHandEmpty ?x_3)
217     (clean-shaker ?x_0 ?x_4 ?x_3)
218   )
219 )
220 (:method CleanShakerNull
221   :parameters (?x_0 - shaker)
222   :task (AchieveCleanShaker ?x_0)
223   :precondition (and
224     (clean ?x_0)
225   )
226   :ordered-subtasks (and
227   )
228 )
229 (:method Pickup
230   :parameters (?x_0 - hand ?x_1 - container)
231   :task (AchieveHolding ?x_0 ?x_1)
232   :precondition (and
233     (not (holding ?x_0 ?x_1))
234   )
235   :ordered-subtasks (and

```

```

236     (AchieveHandEmpty ?x_0)
237     (AchieveOnTable ?x_1)
238     (grasp ?x_0 ?x_1)
239 )
240 )
241 (:method HoldingNull
242   :parameters (?x_0 - hand ?x_1 - container)
243   :task (AchieveHolding ?x_0 ?x_1)
244   :precondition (and
245     (holding ?x_0 ?x_1)
246   )
247   :ordered-subtasks (and
248   )
249 )
250 (:method EmptyHand
251   :parameters (?x_0 - hand ?x_1 - container)
252   :task (AchieveHandEmpty ?x_0)
253   :precondition (and
254     (holding ?x_0 ?x_1)
255   )
256   :ordered-subtasks (and
257     (drop ?x_0 ?x_1)
258   )
259 )
260 (:method HandEmptyNull
261   :parameters (?x_0 - hand)
262   :task (AchieveHandEmpty ?x_0)
263   :precondition (and
264     (handEmpty ?x_0)
265   )
266   :ordered-subtasks (and
267   )
268 )
269 (:method PutDown
270   :parameters (?x_0 - container ?x_1 - hand)
271   :task (AchieveOnTable ?x_0)
272   :precondition (and
273     (holding ?x_1 ?x_0)
274   )
275   :ordered-subtasks (and
276     (drop ?x_1 ?x_0)
277   )

```

```

278 )
279
280 (:method OnTableNull
281   :parameters (?x_0 - container)
282   :task (AchieveOnTable ?x_0)
283   :precondition (and
284     (ontable ?x_0)
285   )
286   :ordered-subtasks (and
287   )
288 )
289 (:method pour_shaker_to_shot_action
290   :parameters (?x_0 - shaker ?x_1 - shot ?x_2 - cocktail ?x_3 -
291     level ?x_4 - hand ?x_5 - level)
292   :task (DoPourShakerToShot ?x_0 ?x_1 ?x_2)
293   :precondition (and
294     (holding ?x_4 ?x_0)
295     (shaked ?x_0)
296     (empty ?x_1)
297     (clean ?x_1)
298     (contains ?x_0 ?x_2)
299     (shakerLevel ?x_0 ?x_3)
300     (next ?x_5 ?x_3)
301   )
302   :ordered-subtasks (and
303     (pour-shaker-to-shot ?x_2 ?x_1 ?x_4 ?x_0 ?x_3 ?x_5)
304   )
305 )
306
307
308 (:action clean-shaker
309   :parameters (?x_0 - shaker ?x_1 - hand ?x_2 - hand)
310   :precondition (and
311     (holding ?x_1 ?x_0)
312     (empty ?x_0)
313     (handEmpty ?x_2)
314   )
315   :effect (and
316     (clean ?x_0)
317   )
318 )

```

```

319(:action clean-shot
320  :parameters (?x_0 - shot ?x_1 - beverage ?x_2 - hand ?x_3 - hand )
321  :precondition (and
322    (holding ?x_2 ?x_0)
323    (handEmpty ?x_3)
324    (empty ?x_0)
325    (used ?x_0 ?x_1)
326  )
327  :effect (and
328    (clean ?x_0)
329    (not (used ?x_0 ?x_1))
330  )
331)
332(:action drop
333  :parameters (?x_0 - hand ?x_1 - container)
334  :precondition (and
335    (holding ?x_0 ?x_1)
336  )
337  :effect (and
338    (ontable ?x_1)
339    (handEmpty ?x_0)
340    (not (holding ?x_0 ?x_1))
341  )
342)
343(:action empty-shaker
344  :parameters (?x_0 - hand ?x_1 - shaker ?x_2 - cocktail ?x_3 -
level ?x_4 - level)
345  :precondition (and
346    (holding ?x_0 ?x_1)
347    (contains ?x_1 ?x_2)
348    (shaked ?x_1)
349    (shakerEmptyLevel ?x_1 ?x_4)
350    (shakerLevel ?x_1 ?x_3)
351  )
352  :effect (and
353    (empty ?x_1)
354    (unshaked ?x_1)
355    (shakerLevel ?x_1 ?x_4)
356    (not (contains ?x_1 ?x_2))
357    (not (shakerLevel ?x_1 ?x_3))
358    (not (shaked ?x_1))
359  )

```



```

360 )
361 (:action empty-shot
362   :parameters (?x_0 - hand ?x_1 - shot ?x_2 - beverage)
363   :precondition (and
364     (holding ?x_0 ?x_1)
365     (contains ?x_1 ?x_2)
366   )
367   :effect (and
368     (empty ?x_1)
369     (not (contains ?x_1 ?x_2))
370   )
371 )
372 (:action fill-shot
373   :parameters (?x_0 - shot ?x_1 - ingredient ?x_2 - hand ?x_3 -
hand ?x_4 - dispenser)
374   :precondition (and
375     (holding ?x_2 ?x_0)
376     (handEmpty ?x_3)
377     (empty ?x_0)
378     (clean ?x_0)
379     (dispenses ?x_4 ?x_1)
380   )
381   :effect (and
382     (contains ?x_0 ?x_1)
383     (used ?x_0 ?x_1)
384     (not (clean ?x_0))
385     (not (empty ?x_0))
386   )
387 )
388 (:action grasp
389   :parameters (?x_0 - hand ?x_1 - container)
390   :precondition (and
391     (ontable ?x_1)
392     (handEmpty ?x_0)
393   )
394   :effect (and
395     (holding ?x_0 ?x_1)
396     (not (handEmpty ?x_0))
397     (not (ontable ?x_1))
398   )
399 )
400 (:action pour-shaker-to-shot

```

```

401 :parameters (?x_0 - cocktail ?x_1 - shot ?x_2 - hand ?x_3 -
shaker ?x_4 - level ?x_5 - level)
402 :precondition (and
403   (holding ?x_2 ?x_3)
404   (contains ?x_3 ?x_0)
405   (shaked ?x_3)
406   (clean ?x_1)
407   (empty ?x_1)
408   (shakerLevel ?x_3 ?x_4)
409   (next ?x_5 ?x_4)
410 )
411 :effect (and
412   (contains ?x_1 ?x_0)
413   (used ?x_1 ?x_0)
414   (shakerLevel ?x_3 ?x_5)
415   (not (clean ?x_1))
416   (not (empty ?x_1))
417   (not (shakerLevel ?x_3 ?x_4))
418 )
419)
420(:action pour-shot-to-clean-shaker
421 :parameters (?x_0 - shot ?x_1 - ingredient ?x_2 - shaker ?x_3 -
hand ?x_4 - level ?x_5 - level)
422 :precondition (and
423   (contains ?x_0 ?x_1)
424   (empty ?x_2)
425   (clean ?x_2)
426   (holding ?x_3 ?x_0)
427   (shakerLevel ?x_2 ?x_4)
428   (next ?x_4 ?x_5)
429 )
430 :effect (and
431   (contains ?x_2 ?x_1)
432   (shakerLevel ?x_2 ?x_5)
433   (unshaked ?x_2)
434   (empty ?x_0)
435   (not (clean ?x_2))
436   (not (empty ?x_2))
437   (not (contains ?x_0 ?x_1))
438   (not (shakerLevel ?x_2 ?x_4))
439 )
440)

```

```

441(:action pour-shot-to-used-shaker
442  :parameters (?x_0 - shot ?x_1 - ingredient ?x_2 - shaker ?x_3 -
443    hand ?x_4 - level ?x_5 - level)
444  :precondition (and
445    (contains ?x_0 ?x_1)
446    (unshaked ?x_2)
447    (holding ?x_3 ?x_0)
448    (shakerLevel ?x_2 ?x_4)
449    (next ?x_4 ?x_5)
450  )
451  :effect (and
452    (contains ?x_2 ?x_1)
453    (shakerLevel ?x_2 ?x_5)
454    (empty ?x_0)
455    (not (contains ?x_0 ?x_1))
456    (not (shakerLevel ?x_2 ?x_4))
457  )
458(:action shake
459  :parameters (?x_0 - cocktail ?x_1 - ingredient ?x_2 - ingredient ?x_3
460    - shaker ?x_4 - hand ?x_5 - hand)
461  :precondition (and
462    (handEmpty ?x_5)
463    (holding ?x_4 ?x_3)
464    (contains ?x_3 ?x_1)
465    (contains ?x_3 ?x_2)
466    (unshaked ?x_3)
467    (cocktailPart1 ?x_0 ?x_1)
468    (cocktailPart2 ?x_0 ?x_2)
469  )
470  :effect (and
471    (shaked ?x_3)
472    (contains ?x_3 ?x_0)
473    (not (unshaked ?x_3))
474    (not (contains ?x_3 ?x_1))
475    (not (contains ?x_3 ?x_2))
476  )
477)

```

A.2 Κώδικας Προβλήματος Μέσης Δυσκολίας (pfile10.hddl)

Ακολουθεί ο πλήρης κώδικας του δέκατου προβλήματος, το οποίο αξιολογείται ως μέσης δυσκολίας και μπορεί να αντικατοπτρίσει την πολυπλοκότητα του πεδίου “Barman” σε γλώσσα HDDL.

```
1 (define (problem p-3-9-8)
2 (:domain barman_htn)
3 (:objects
4   left right - hand
5   shaker1 - shaker
6   shot1 shot2 shot3 shot4 shot5 shot6 shot7 shot8 - shot
7   ingredient1 ingredient2 ingredient3 ingredient4 ingredient5
8   ingredient6 ingredient7 ingredient8 ingredient9 - ingredient
9   dispenser1 dispenser2 dispenser3 dispenser4 dispenser5 dispenser6
10  dispenser7 dispenser8 dispenser9 - dispenser
11  cocktail1 cocktail2 cocktail3 - cocktail
12 level1 level2 level3 - level
13 )
14 (:htn
15   :parameters ( )
16   :ordered-subtasks (and
17     (AchieveContainsShotCocktail shot6 cocktail1)
18     (AchieveContainsShotCocktail shot8 cocktail2)
19     (AchieveContainsShotCocktail shot3 cocktail3)
20   )
21 )
22 (:init
23   (ontable shaker1)
24   (ontable shot1)
25   (ontable shot2)
26   (ontable shot3)
27   (ontable shot4)
28   (ontable shot5)
29   (ontable shot6)
30   (ontable shot7)
31   (ontable shot8)
32   (clean shaker1)
33   (clean shot1)
34   (clean shot2)
35   (clean shot3)
36   (clean shot4)
37   (clean shot5)
38   (clean shot6)
```

37	(clean shot7)
38	(clean shot8)
39	(empty shaker1)
40	(empty shot1)
41	(empty shot2)
42	(empty shot3)
43	(empty shot4)
44	(empty shot5)
45	(empty shot6)
46	(empty shot7)
47	(empty shot8)
48	(dispenses dispenser1 ingredient1)
49	(dispenses dispenser2 ingredient2)
50	(dispenses dispenser3 ingredient3)
51	(dispenses dispenser4 ingredient4)
52	(dispenses dispenser5 ingredient5)
53	(dispenses dispenser6 ingredient6)
54	(dispenses dispenser7 ingredient7)
55	(dispenses dispenser8 ingredient8)
56	(dispenses dispenser9 ingredient9)
57	(handEmpty left)
58	(handEmpty right)
59	(shakerEmptyLevel shaker1 level1)
60	(shakerLevel shaker1 level1)
61	(next level1 level2)
62	(next level2 level3)
63	(cocktailPart1 cocktail1 ingredient9)
64	(cocktailPart2 cocktail1 ingredient1)
65	(cocktailPart1 cocktail2 ingredient3)
66	(cocktailPart2 cocktail2 ingredient2)
67	(cocktailPart1 cocktail3 ingredient7)
68	(cocktailPart2 cocktail3 ingredient6)
69	)
70	)